GALAXITECH
DDD BELGIUM · WORKSHOP 2026
SEE THE SYSTEM
Wardley mapping, one click at a time

HOW THE EVENING RUNS

Talk, group work, discussion

TALK (5-10 MIN)	THEN, IN YOUR GROUP
01 Cup of tea	Exercise 1 · Who needs what? Exercise 2 · Build the chain
02 Evolution: the map wakes up	Exercise 3 · Spread it out
03 Tea shop at rush hour	Exercise 4A · Find the constraint Exercise 4B · Draw three pipelines
04 Strategy and the OODA loop Knowledge architecture	Exercise 5 · Simplify and move
05 A real map: EM tooling	Event modeling tooling map

01

Cup of tea

Tea shop

Customer

Tea shop
Customer
Cup of tea

Tea shop
Customer
Cup of tea
Cup

Tea shop
Customer
Cup of tea
Cup
Tea leaves

Tea shop
Customer
Cup of tea
Cup
Hot water
Tea leaves

Tea shop
Customer
Cup of tea
Cup
Hot water
Tea leaves
Kettle

Tea shop
Customer
Cup of tea
Cup
Tea leaves
Hot water
Kettle
Cold water

Tea shop
Customer
Cup of tea
Cup
Tea leaves
Hot water
Kettle
Cold water
Tap

Tea shop
Customer
Cup of tea
Cup
Tea leaves
Hot water
Kettle
Cold water
Power
Tap

Visibility
Tea shop
Customer
Cup of tea
Cup
Tea leaves
Hot water
Kettle
Cold water
Power
Tap

Problem domain

Visibility

Problem domain

Solution domain

Visibility

Reading the value chain

- Upper components are visible to the user. Lower components are hidden from the user
- Read down to ask how: how is this need met?
- Read up to ask why: why do we need this?
- Each link is a dependency and an exchange: the component above pays money or effort, and gets a capability back
- So every component has a cost, and provides value to the one above it
- A map is a chain of needs

What goes on a map

- Activities: what we do. Making a cup of tea
- Practices: how we do it. Hand brewing
- Data: facts we keep. The menu and the prices
- Knowledge: what we understand. How strong tea should be
- Things we use: the cup, the kettle

Wardley Mapping Canvas

1. Purpose

What is your purpose? Why does this organization or project exist?

2. Scope

What is it that you are mapping? What does it include? What does it *not* include?

3. Users

Who uses or interacts with the thing you are mapping?

4. User Needs

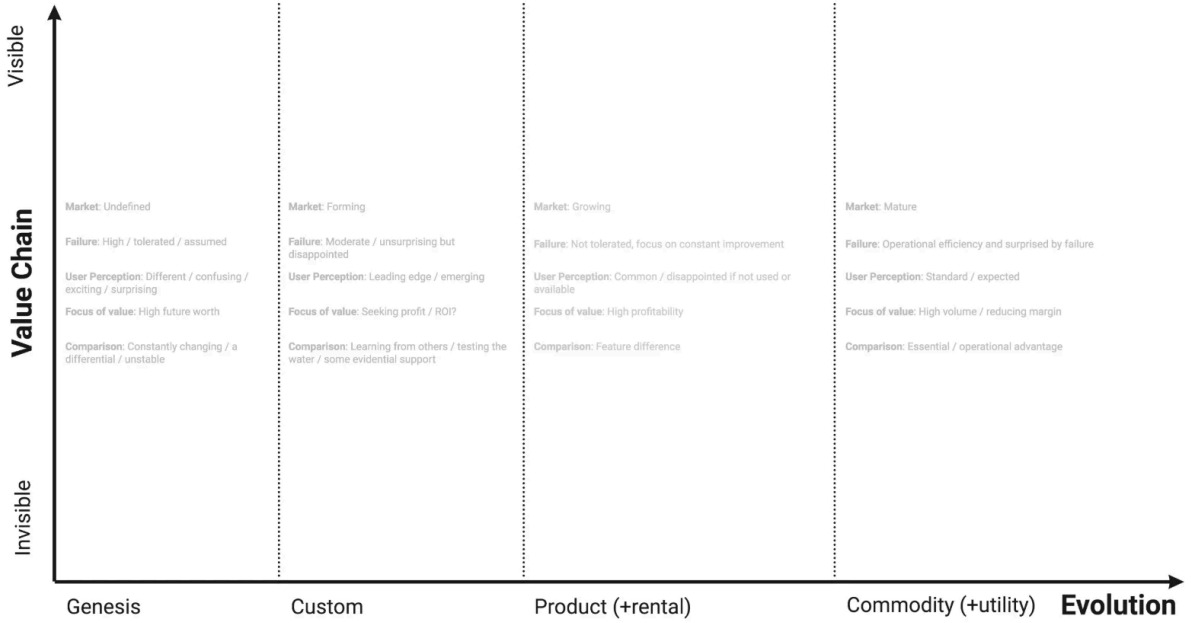
Copy your users over. What do they need from you? What is each user's journey?

5. Value Chain

Copy the users over, with their needs underneath. What sorts of things do you need to be doing to fulfill those needs? Arrange them according to dependence — create a value chain.

6. Map

Copy the value chain over. Use the evolutionary characteristics to decide where to place each component along the horizontal axis (Evolution).



Wardley Mapping, Evolutionary Characteristics Cheat Sheet courtesy of Simon Wardley, CC BY-SA 4.0.
Canvas designed by Ben Mosior. Visit https://hiredthought.com/wardley-mapping for more information.

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License, CC BY-SA 4.0.
To view a copy of the license, visit https://creativecommons.org/licenses/by-sa/4.0/

Six steps to a Wardley map

- 1 Purpose: why are we mapping? (exercise 1)
- 2 Scope: what is in, what is out? (exercise 1)
- 3 Users: who is it for? (exercise 1)
- 4 User needs: what do they need? (exercise 1)
- 5 Value chain: what do the needs depend on? (exercise 2)
- 6 Map: place it all on the evolution axis (exercise 3)
- Tonight we do steps 1-6 across exercises 1, 2, and 3

Exercise 1: Harbour Insurance

- Imagine you are a product team at Harbour Insurance, a fictional company
- Every team now has AI coding agents. Code arrives in hours, not days
- Yet customers get new features no faster than before
- Management asks: we pay for AI coding agents, but we don't see faster delivery. **What is happening?**
- Your domain expert knows more about the company. Ask them
- Your task: identify users and their needs (canvas steps 1-4)

Exercise 1: the company and its purpose

- Harbour Insurance insures ships, cargo and the businesses around the port of Antwerp
- About 2,000 employees and 300,000 policies. Quotes, policies and claims all run through its own software
- The product and engineering organisation: 6 product teams, about 40 developers, one architect
- Its purpose: get new insurance products and changes to customers quickly and safely
- Its scope for today: from a product hypothesis to customers using the change

YOUR GROUP

Exercise 1: four people, four roles

- Domain expert: has the final word and decides
- Facilitator: starts the talk, makes sure all are heard
- Timekeeper: keeps time and keeps you moving
- Clerk: materials, the canvas, runs to CJ with questions
- Feel free to swap roles after each exercise

Exercise 1: who needs what?

- Follow the canvas, steps 1 to 4
- Write the purpose and the scope first
- Then identify the users
- Under each user, write their needs on stickies
- **Stop here** – value chain comes in Exercise 2

Exercise 1: discuss your findings

- How many different users did you identify?
- Which user has the most needs? The fewest?
- Are any needs shared across multiple users?
- Next: we'll pick ONE anchor to build our map from

4. User Needs

Copy your users over. What d from you? What 's journey?

management person

predictability

measurable outcomes

alignment

profit and happy customers and stakeholders

progress

BI

product person

Hyphothesises

Prototypes

Fast feedback

Software Architect

Governance

overview

code following target architecture

Sustainable system/ ways of working

only eccential complexity

avoid becomming a bottleneck

Software Developer

Feedback

Validation

Learning

avoid borring work

Work with exiting problems

optimised flow

QA Person

Validation

overview

Quality

provide feedback

Pick your anchor

- You've identified multiple users – great!
- To build **one map**, we need **one anchor**
- Ask: **what question is this map answering?**
- For Harbour Insurance: "Why is delivery slow?"
- Pick the user whose need matters for that question
- Suggestion: Management + "Time to market"
- The other needs become **components** in your chain

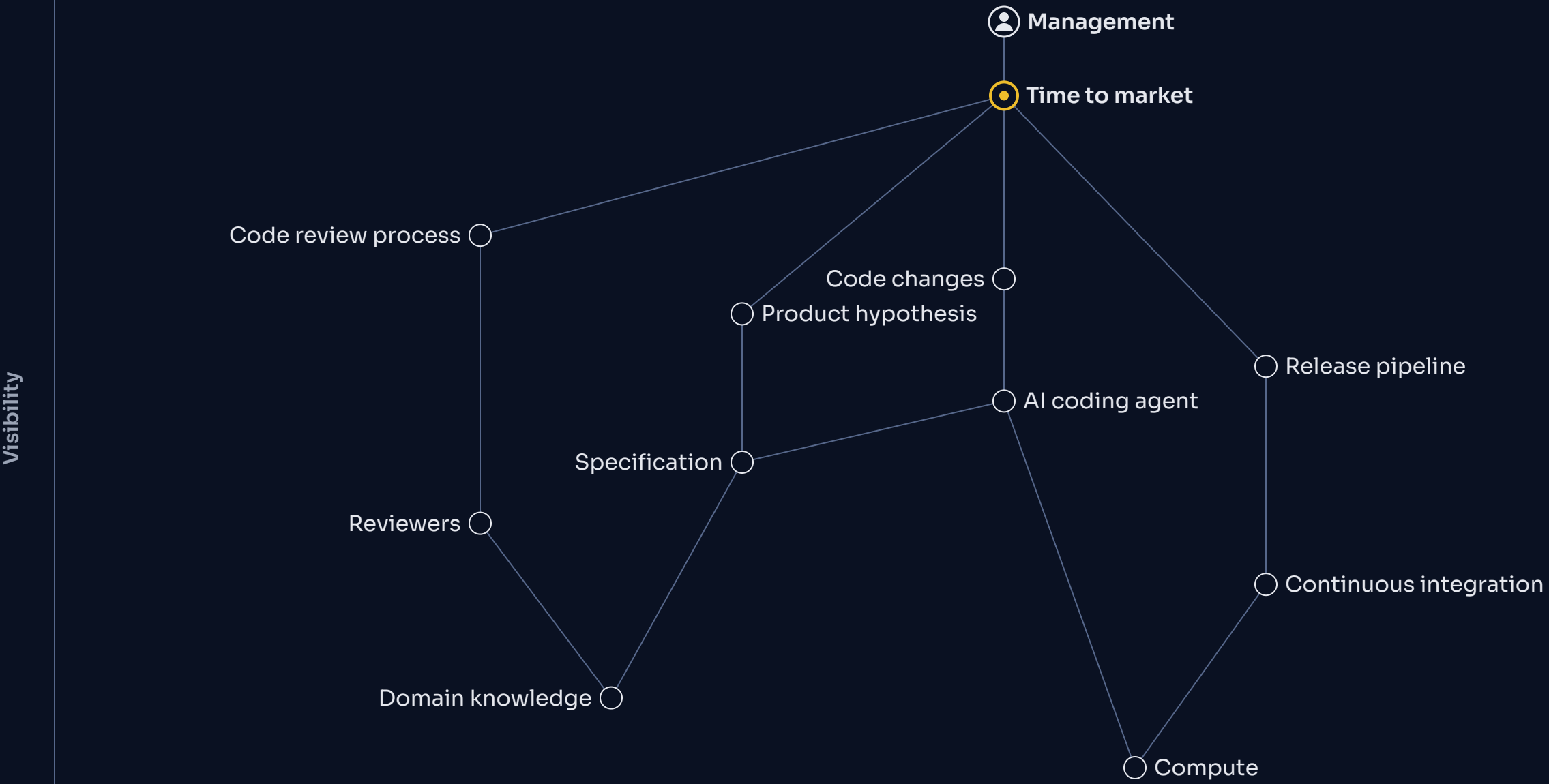
Exercise 2: build the chain

- Start from your chosen anchor (e.g., Management + Time to market)
- Build the value chain: what does that need depend on?
- Keep asking "what does X depend on?" until you reach infrastructure
- Stack components by visibility: visible to users at top, hidden at bottom
- **No evolution axis yet** – that comes in Exercise 3

Exercise 2: discuss your chain

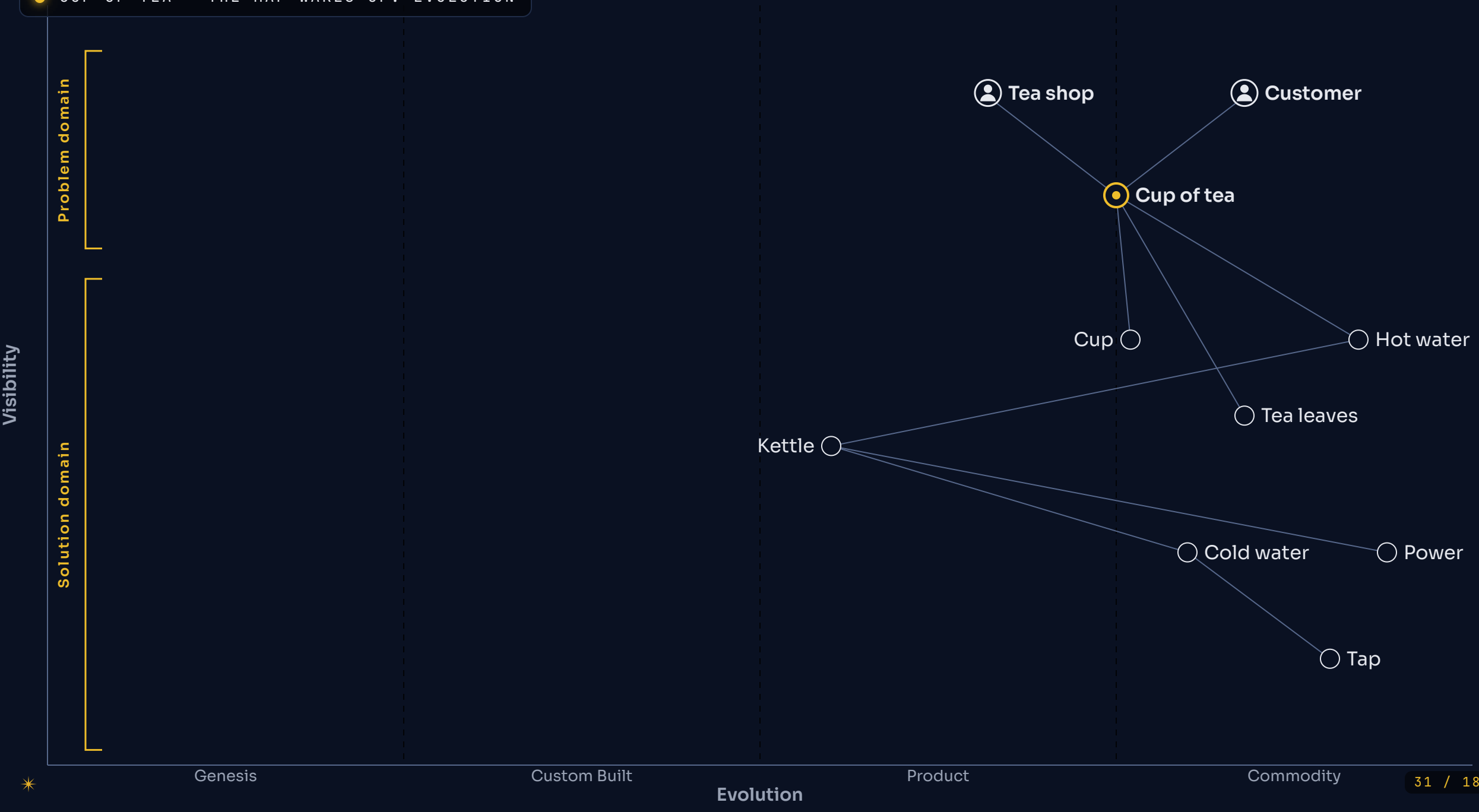
- Where is the line between problem domain and solution domain?
- Which dependency was hardest to identify?
- Did you reach infrastructure, or stop somewhere in the middle?

EXERCISE 2 · ONE WAY TO MAP IT



02

Evolution: the map wakes up


Problem domain
Solution domain
Visibility
Tea shop
Customer
Cup of tea
Cup
Hot water
Tea leaves
Kettle
Cold water
Power
Tap
Genesis
Custom Built
Product
Commodity
Evolution

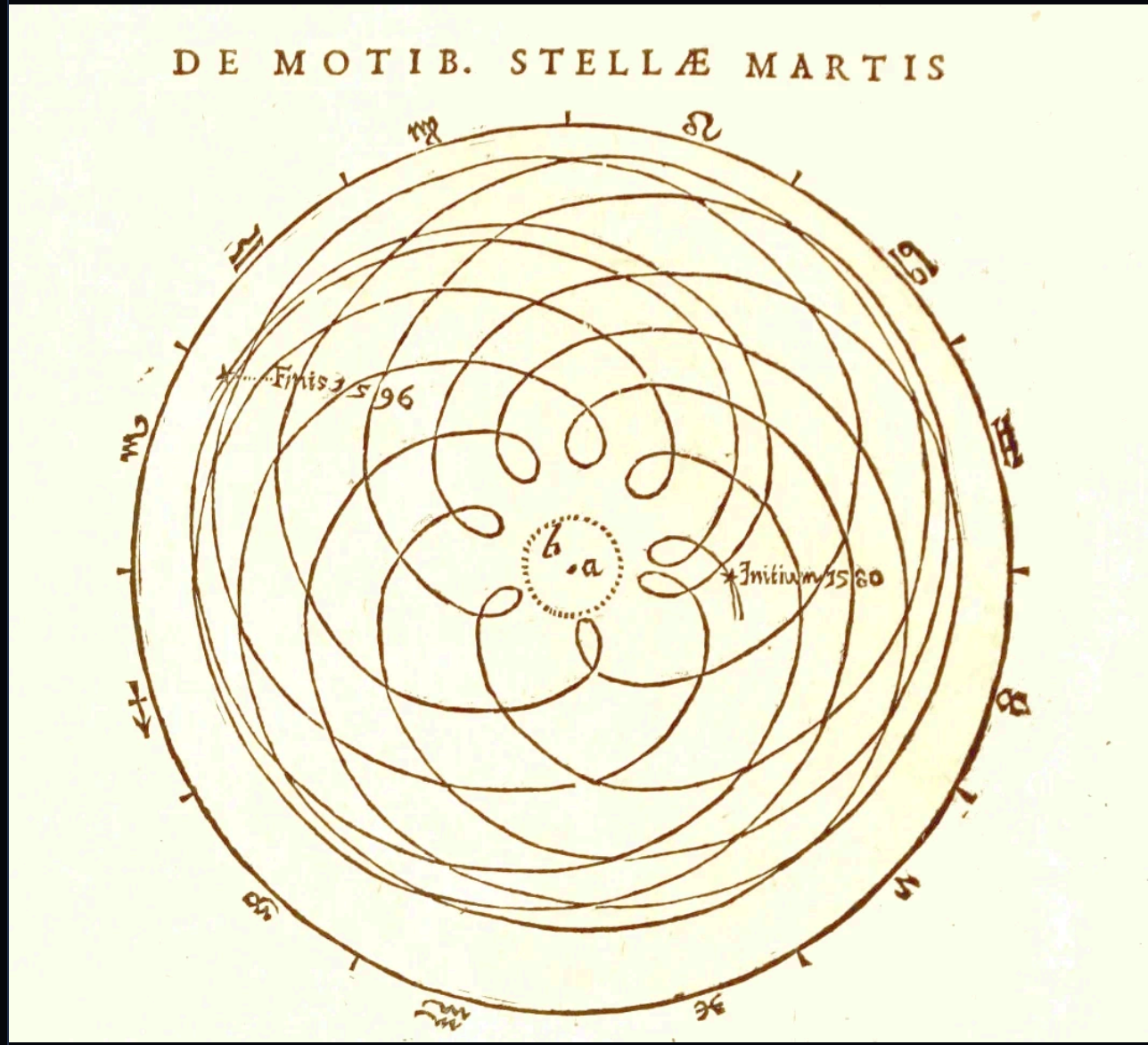
Every kind of component evolves

ACTIVITIES	GENESIS	CUSTOM BUILT	PRODUCT (+ RENTAL)	COMMODITY (+ UTILITY)
Practices	Novel	Emerging	Good	Best
Data	Unmodelled	Divergent	Convergent	Modelled
Knowledge	Concept	Hypothesis	Theory	Accepted

Simon Wardley’s names for the four stages of each kind. Knowledge moves from poorly understood to well understood.

Circles on circles

Seen from the Earth, Mars draws loops. The models kept adding circles and still could not predict it.

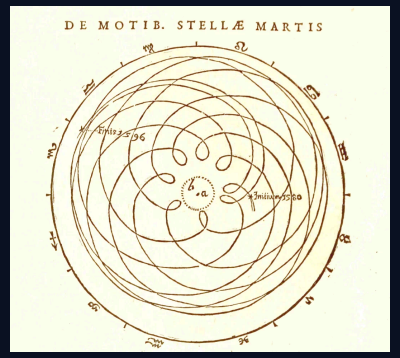


EVOLUTION · KNOWLEDGE

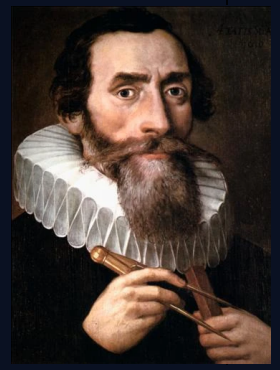
Kepler removed the accidental complexity

He found a simple rule for how the planets move. Our model of the sky went from fuzzy to well understood, and useful.

$$P^2 = \frac{4\pi^2}{G(M_1 + M_2)} a^3$$



Solar system model

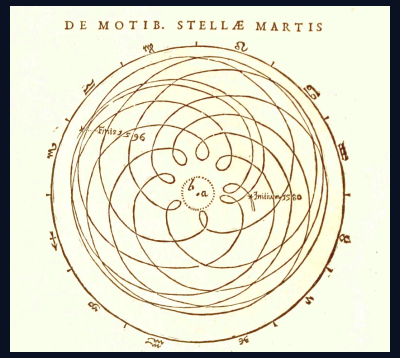


$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

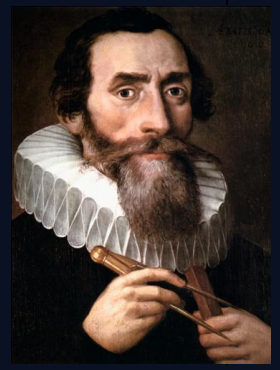
Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity



Solar system model

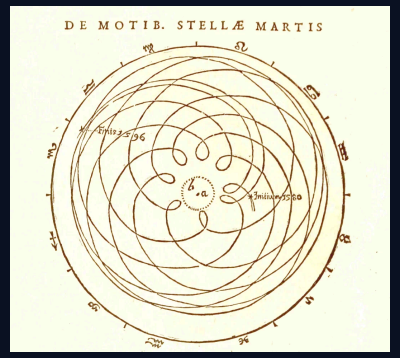


$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

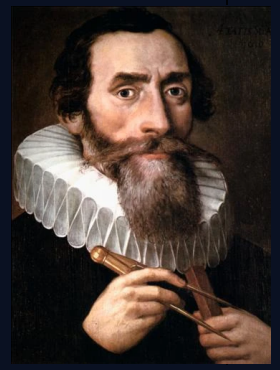
Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity



Solar system model

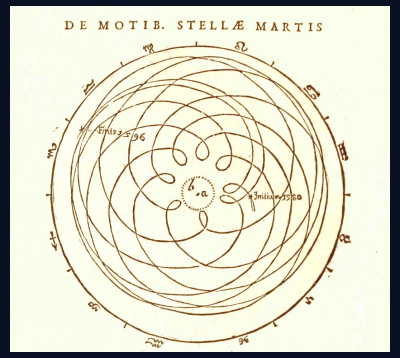


$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

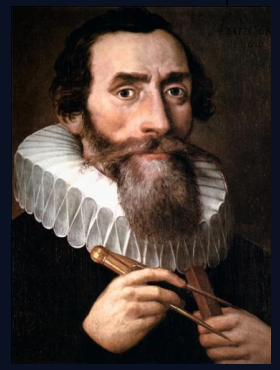
Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity



Solar system model

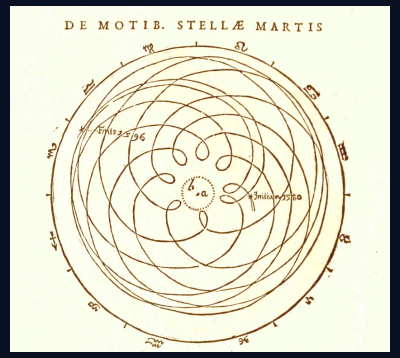


$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

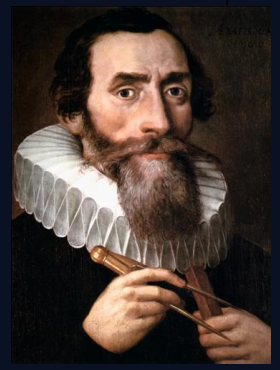
Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity



Solar system model



$P^2 = 4\pi^2 a^3 / G(M_1 + M_2)$

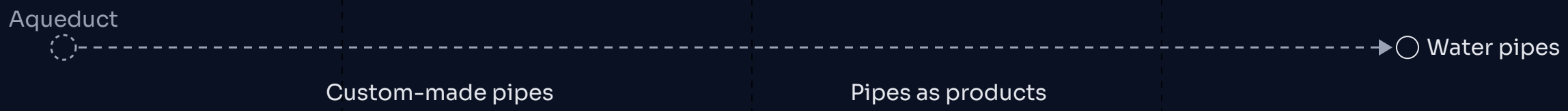
Before: circles on circles. Complex for lack of knowledge

Kepler's equation: the model became clear

He removed the accidental complexity

Like most products, water pipes went from invention to commodity

Everyone has them, and nobody thinks about them



CLIMATE

Things you can't stop

- Everything evolves: genesis to commodity
- Success breeds inertia
- Efficiency enables innovation
- You can anticipate these, not avoid them

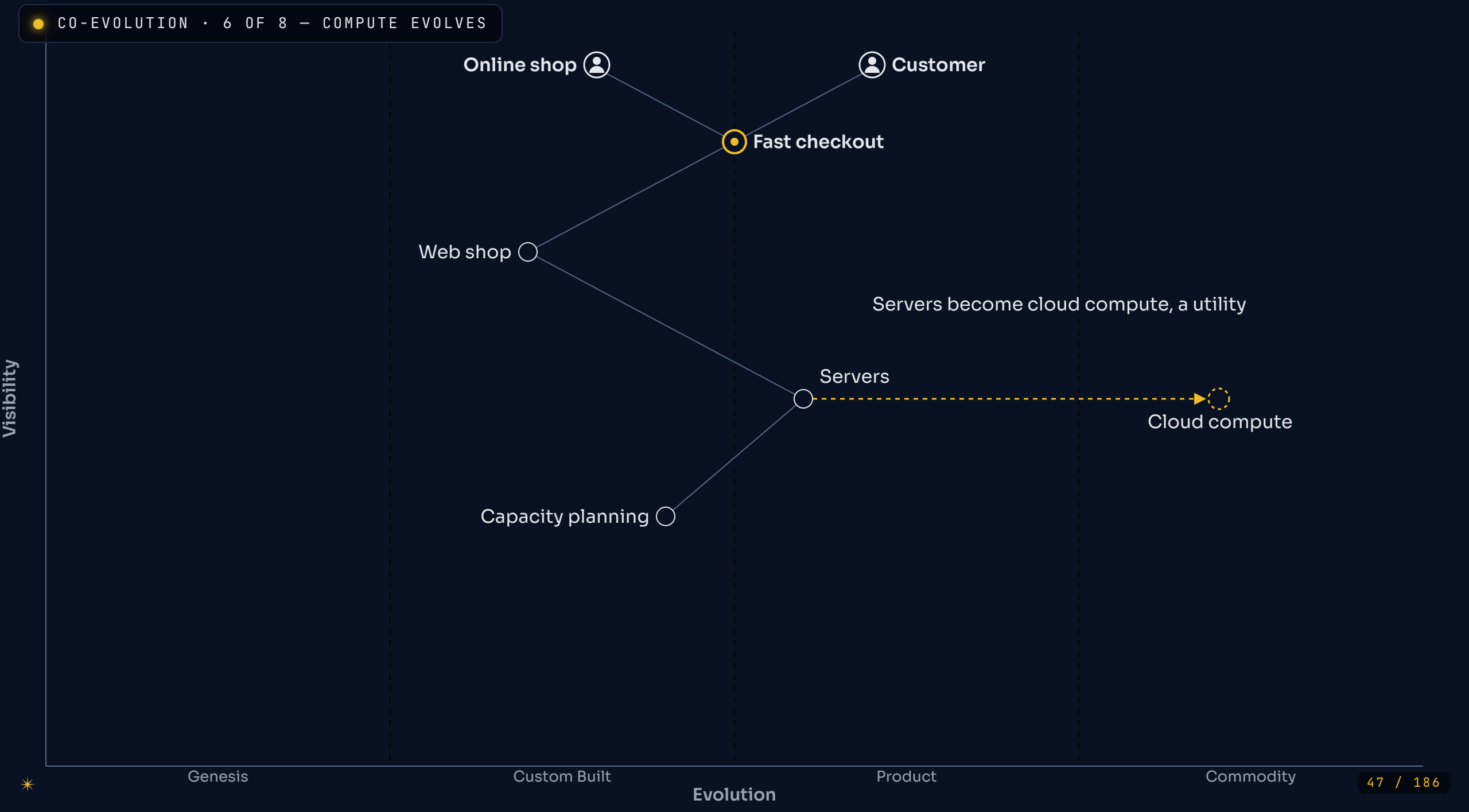
CO-EVOLUTION · 1 OF 8 — THE USERS

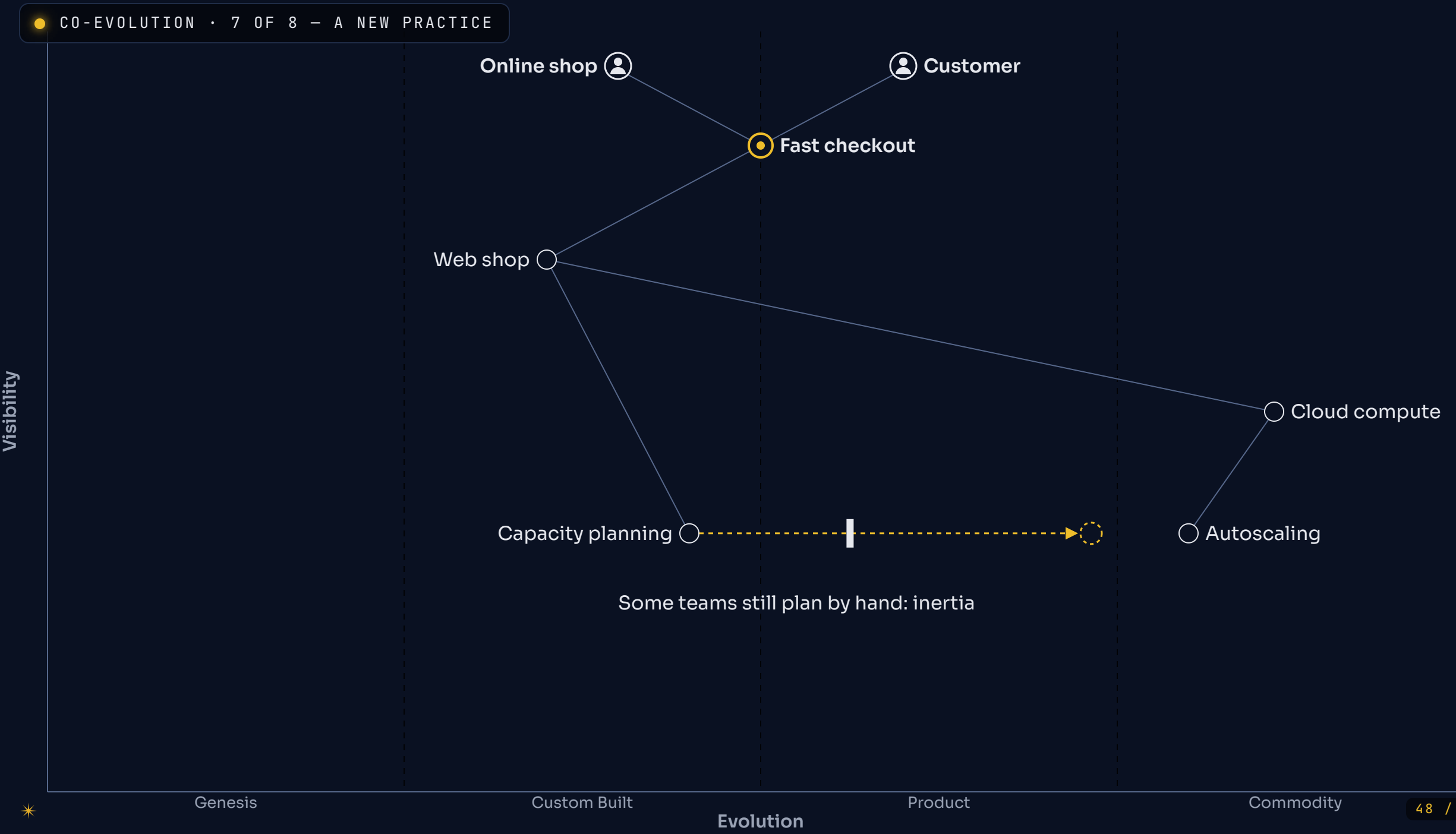
Online shop
Customer
Fast checkout
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

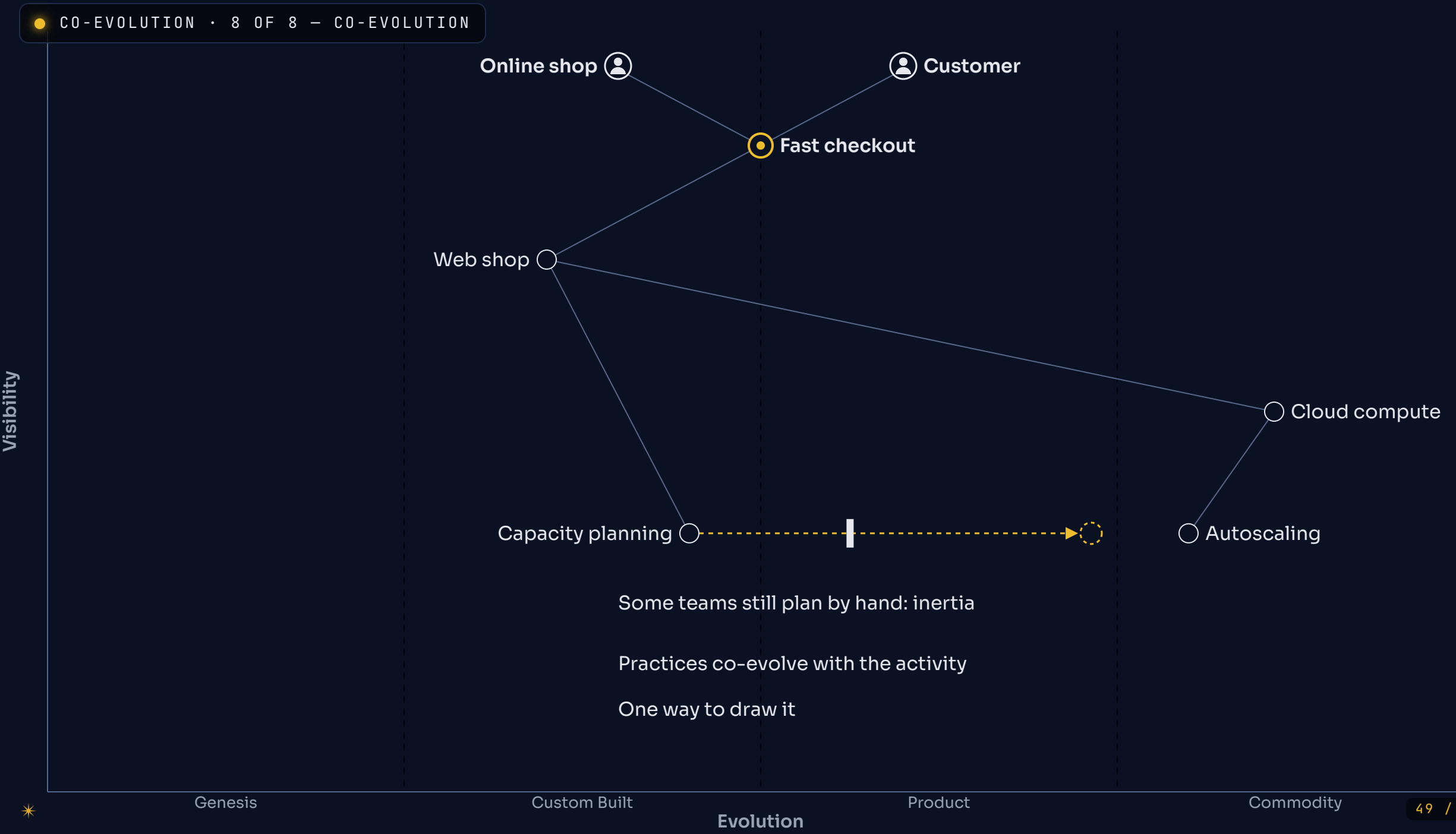
Online shop
Customer
Fast checkout
Web shop
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Online shop
Customer
Fast checkout
Web shop
Servers
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Online shop
Customer
Fast checkout
Web shop
Servers
Capacity planning
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Online shop
Customer
Fast checkout
Web shop
Servers become cloud compute, a utility
Servers
Cloud compute
Capacity planning
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Online shop
Customer
Fast checkout
Web shop
Cloud compute
Capacity planning
Autoscaling
Some teams still plan by hand: inertia
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Online shop
Customer
Fast checkout
Web shop
Cloud compute
Capacity planning
Autoscaling
Some teams still plan by hand: inertia
Practices co-evolve with the activity
One way to draw it
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Practices evolve with the activity

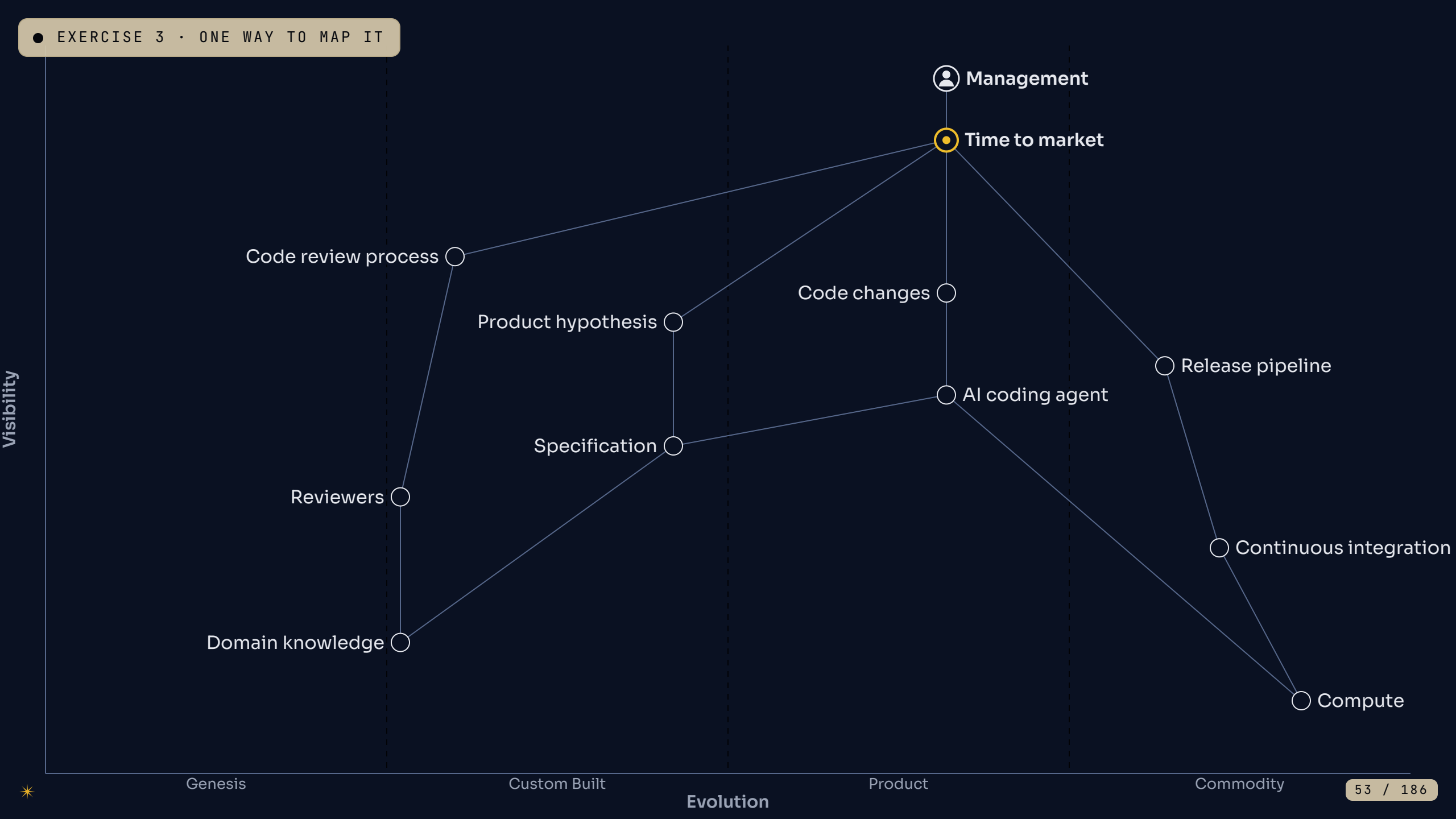
- Servers as a product: you need capacity planning
- Compute as a utility: autoscaling replaces it
- Some teams still plan capacity by hand. That is inertia
- Review is old: punch cards made mistakes costly
- Open source made review a daily practice
- Code is now a commodity. Review will evolve too

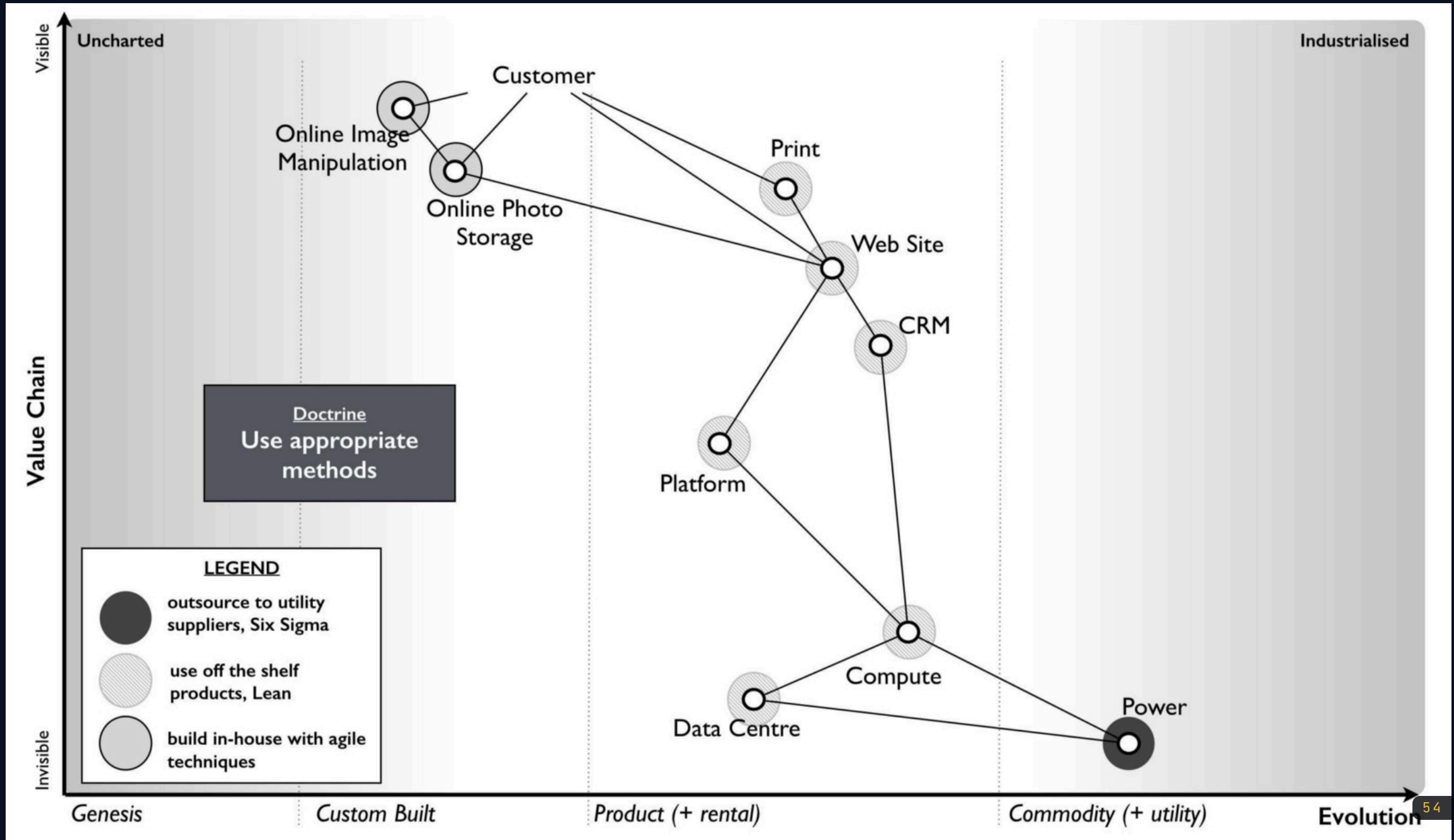
Exercise 3: spread it out!

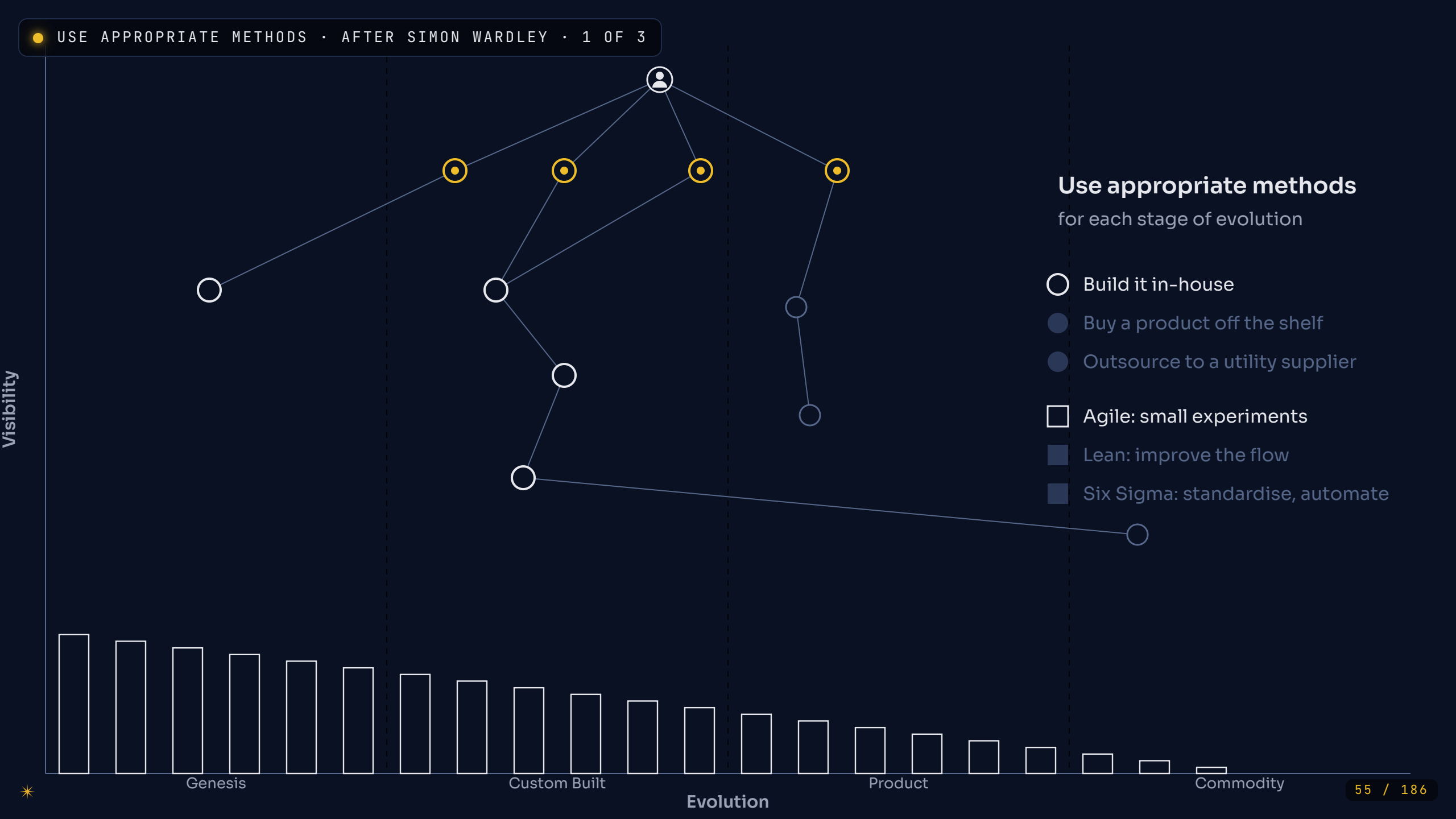
- Take your value chain from Exercise 2
- Now add the evolution axis
- Place every component: Genesis → Custom → Product → Commodity
- Users and needs stay at the top
- Stuck on a placement? The domain expert decides

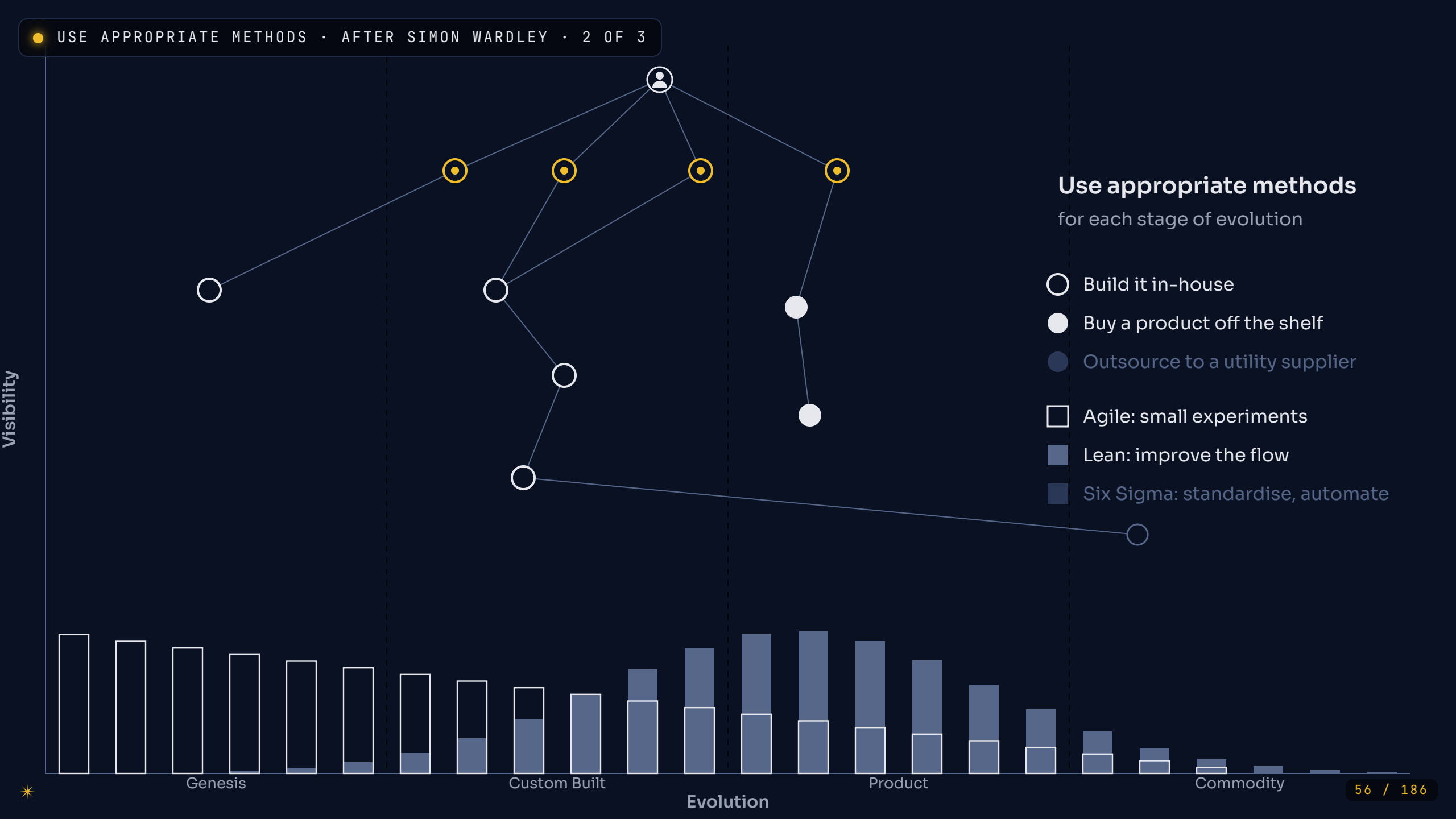
Exercise 3: where did your group disagree?

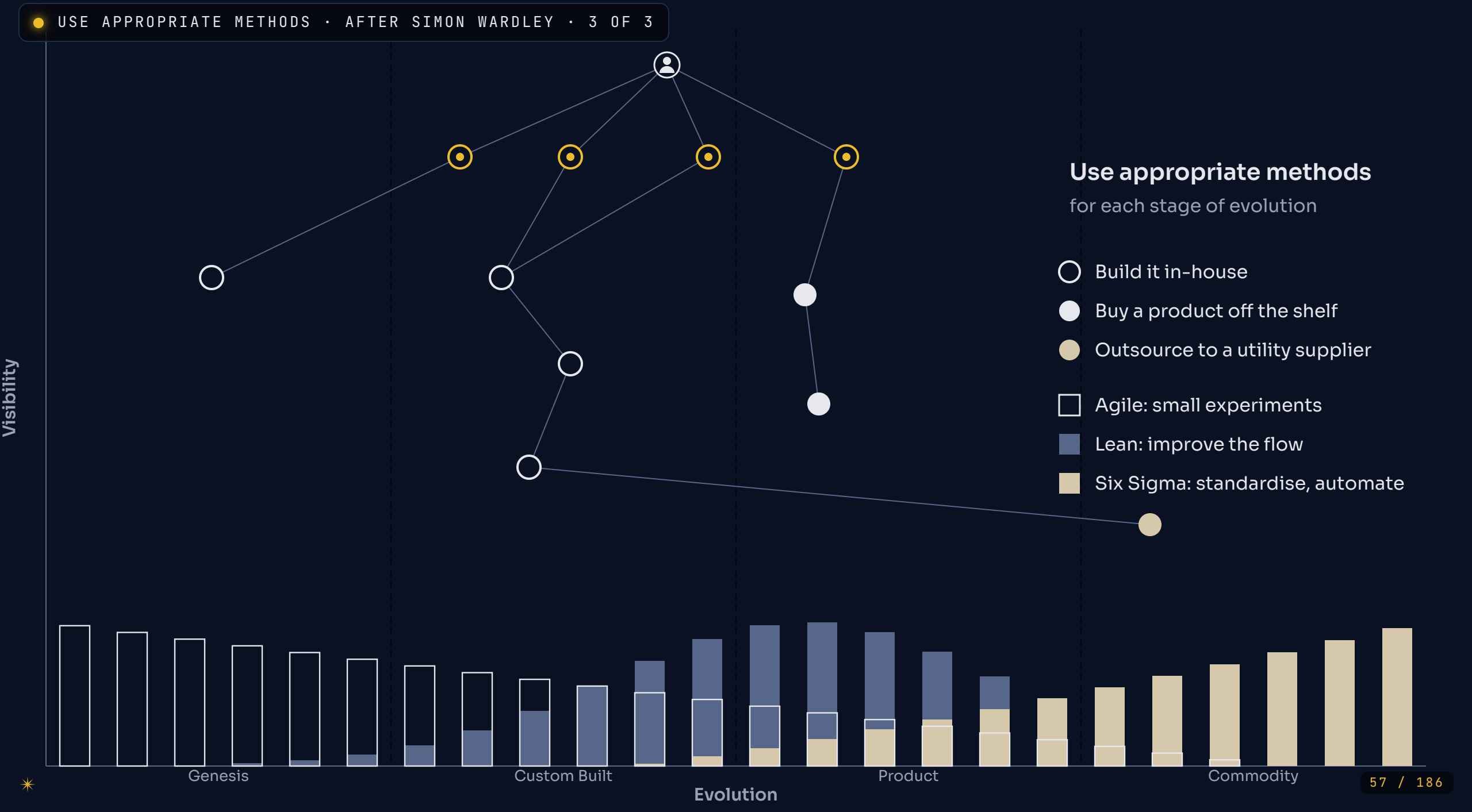
- What is a commodity now that was not two years ago?
- Which placement surprised you?
- Next round: carry on with your map, or start from ours


Management
Time to market
Code review process
Code changes
Product hypothesis
Release pipeline
AI coding agent
Specification
Reviewers
Continuous integration
Domain knowledge
Compute
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Visible
Invisible
Value Chain
Uncharted
Industrialised
Customer
Online Image Manipulation
Online Photo Storage
Print
Web Site
CRM
Platform
Compute
Data Centre
Power
Doctrine
Use appropriate methods
LEGEND
outsource to utility suppliers, Six Sigma
use off the shelf products, Lean
build in-house with agile techniques
Genesis
Custom Built
Product (+ rental)
Commodity (+ utility)
Evolution


Use appropriate methods
for each stage of evolution
Build it in-house
Buy a product off the shelf
Outsource to a utility supplier
Agile: small experiments
Lean: improve the flow
Six Sigma: standardise, automate
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Use appropriate methods
for each stage of evolution
Build it in-house
Buy a product off the shelf
Outsource to a utility supplier
Agile: small experiments
Lean: improve the flow
Six Sigma: standardise, automate
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Use appropriate methods
for each stage of evolution
Build it in-house
Buy a product off the shelf
Outsource to a utility supplier
Agile: small experiments
Lean: improve the flow
Six Sigma: standardise, automate
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

One way of working does not fit the whole map

	GENESIS, CUSTOM BUILT	PRODUCT	COMMODITY, UTILITY
The goal	Learn fast	Get it right	Cut cost and variation
How we work	Agile: small experiments	Lean: improve the flow	Six Sigma: standardise, automate
How we get it	Build it ourselves	Buy it and fit it in	Rent it as a utility
At Harbour Insurance	The product hypothesis	Event model tooling	AI coding agents, CI

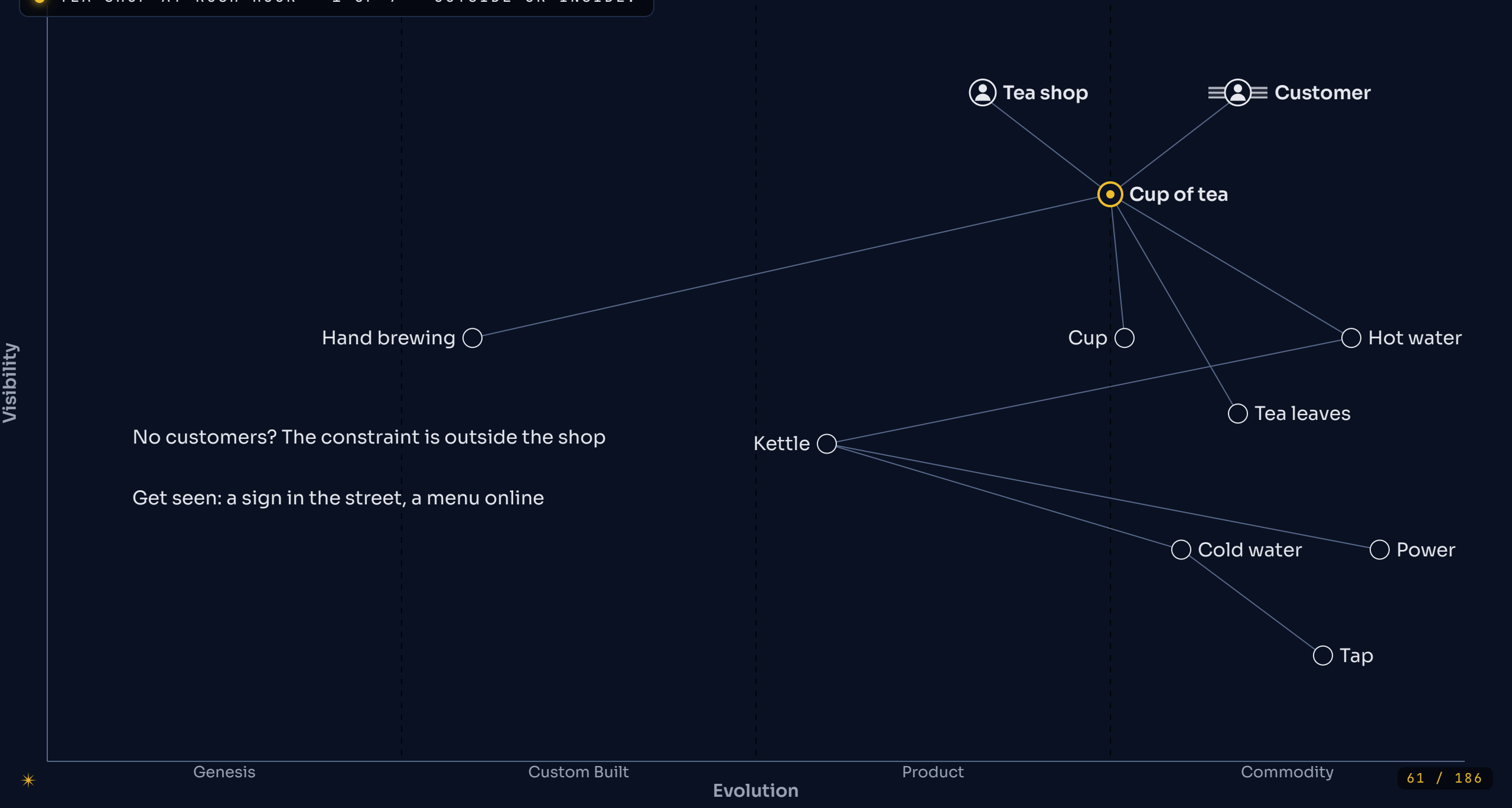
Simon Wardley's doctrine use appropriate methods: pick the way of working by the stage you just placed it in, not by habit.

03

Tea shop at rush hour

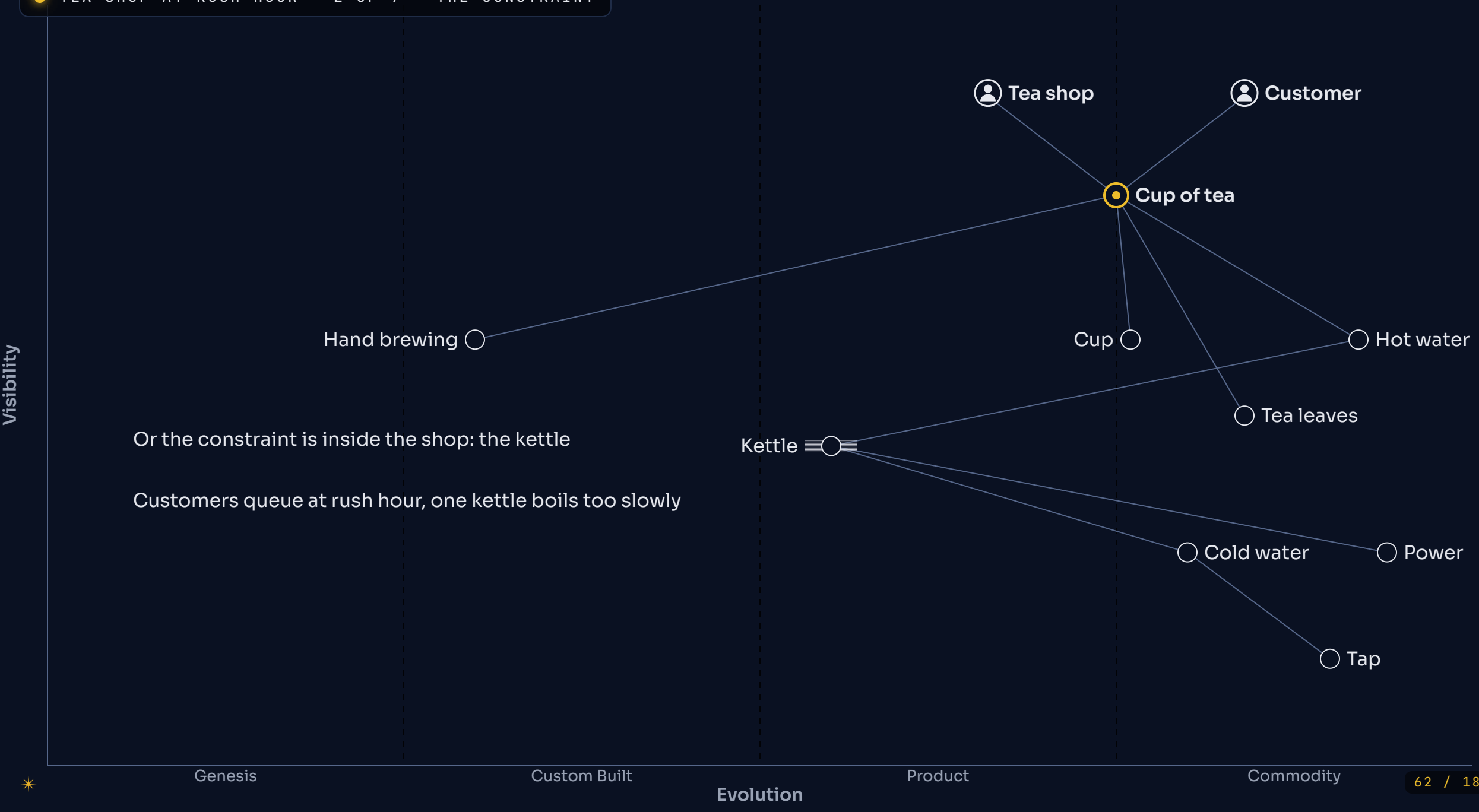
How much should we map?

- Maps grow until they stop saying anything
- Ask: what is the primary constraint here?
- Map the user, the need, the path to it
- Map around the constraint, as Tristan Slominski teaches it



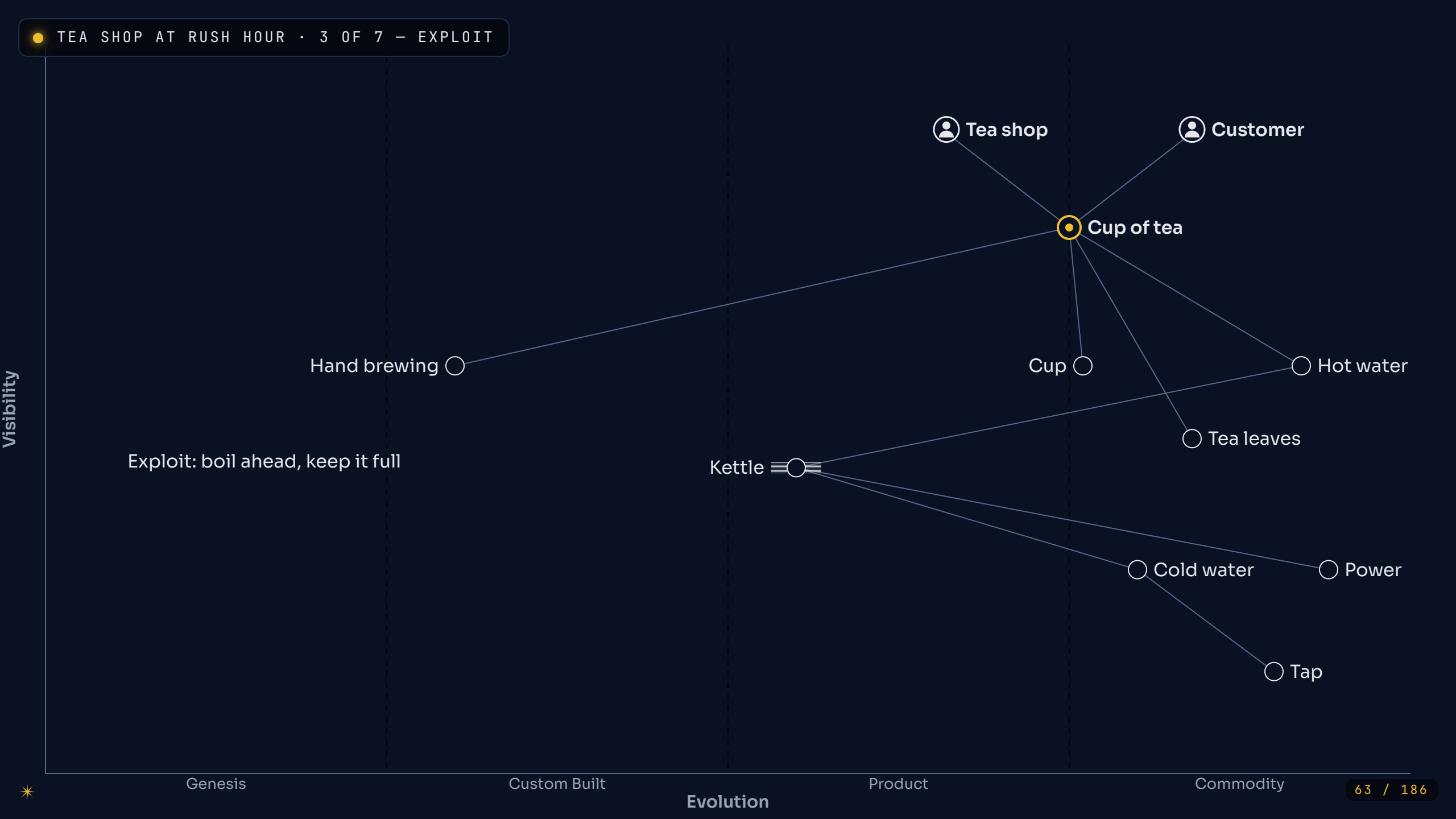
No customers? The constraint is outside the shop

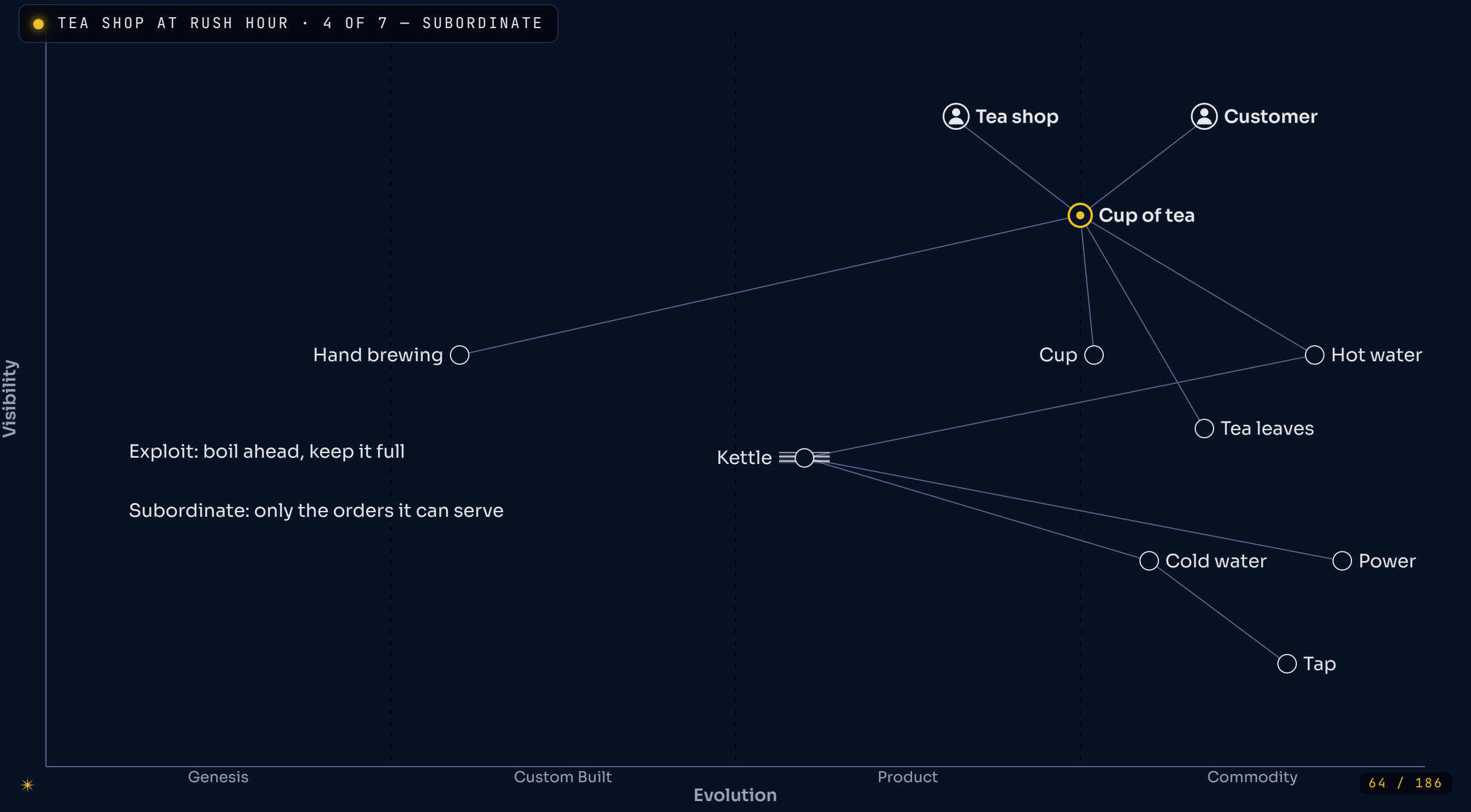
Get seen: a sign in the street, a menu online



Or the constraint is inside the shop: the kettle

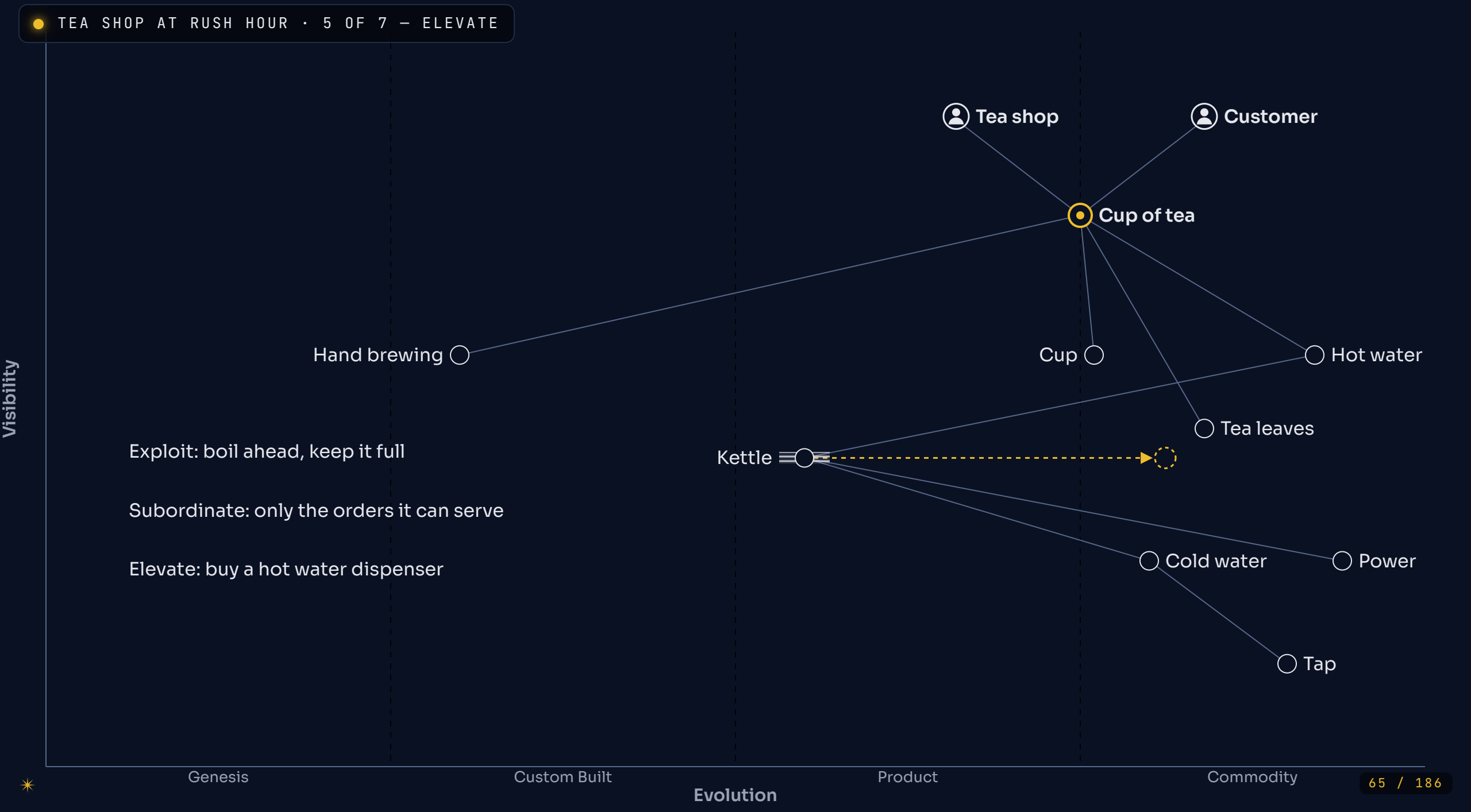
Customers queue at rush hour, one kettle boils too slowly

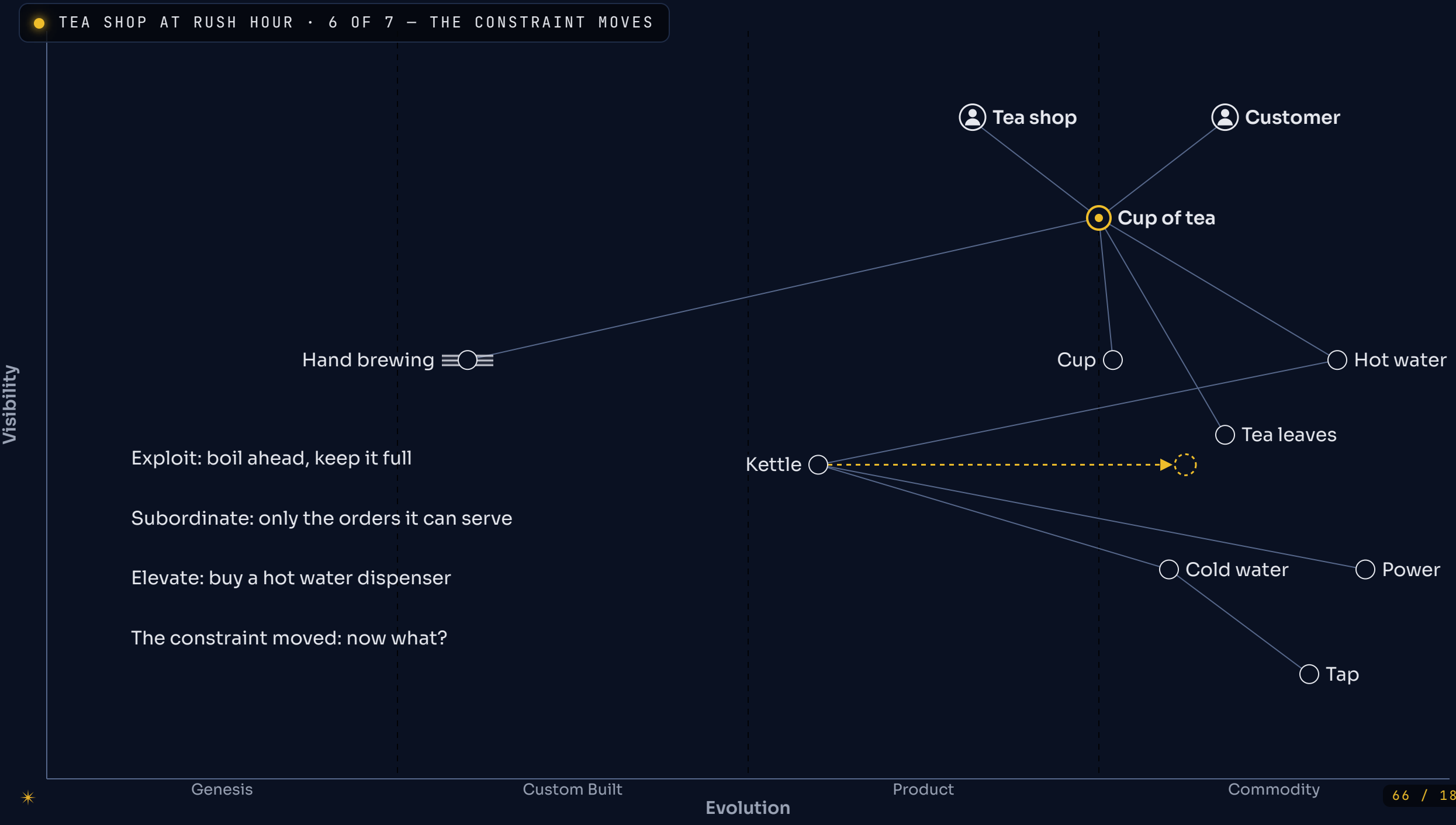

Tea shop
Customer
Cup of tea
Hand brewing
Cup
Hot water
Tea leaves
Exploit: boil ahead, keep it full
Kettle
Cold water
Power
Tap
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

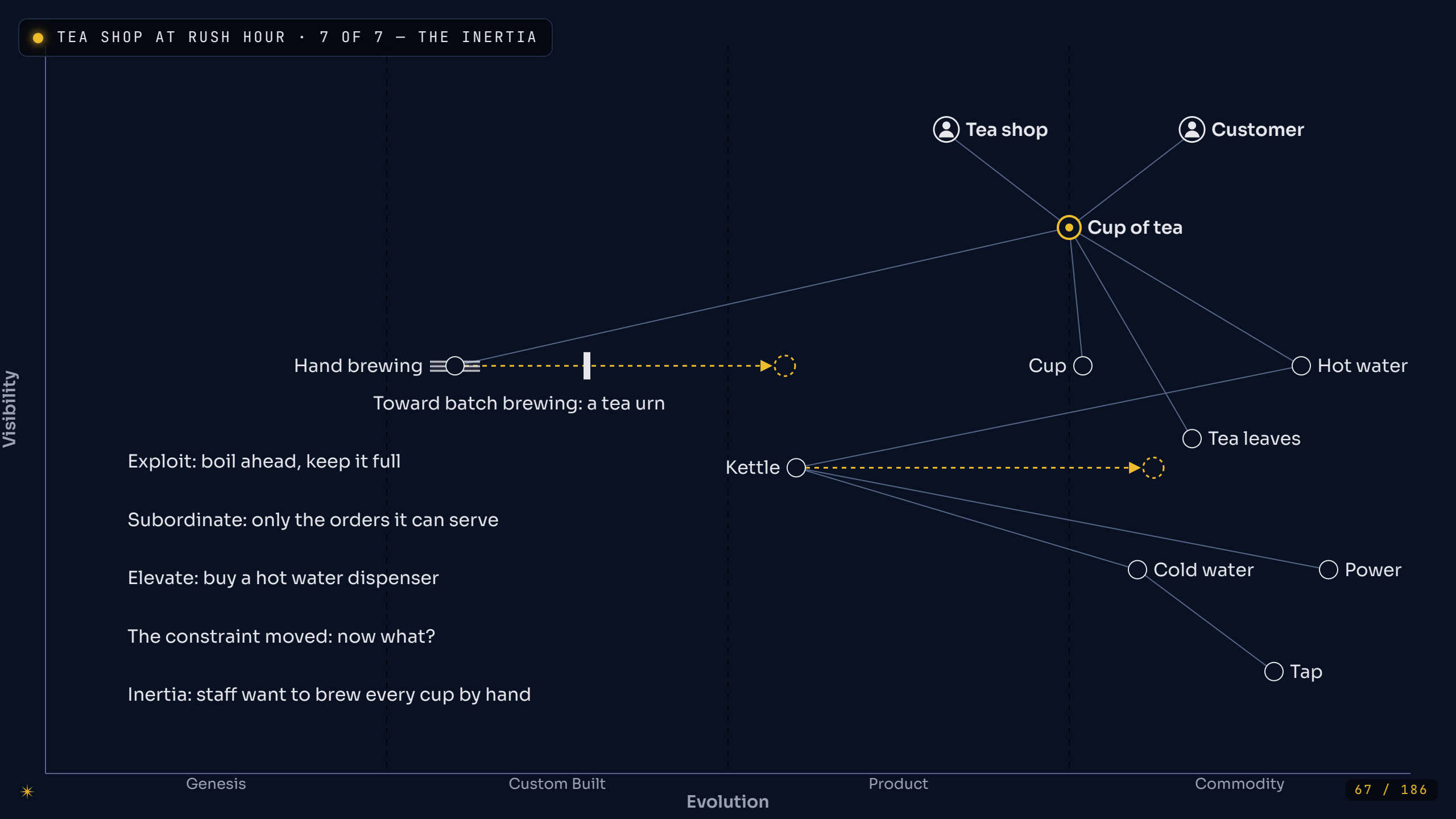


Exploit: boil ahead, keep it full

Subordinate: only the orders it can serve


Tea shop
Customer
Cup of tea
Hand brewing
Cup
Hot water
Tea leaves
Kettle
Cold water
Power
Tap
Exploit: boil ahead, keep it full
Subordinate: only the orders it can serve
Elevate: buy a hot water dispenser
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Tea shop
Customer
Cup of tea
Hand brewing
Cup
Hot water
Tea leaves
Kettle
Cold water
Power
Tap
Exploit: boil ahead, keep it full
Subordinate: only the orders it can serve
Elevate: buy a hot water dispenser
The constraint moved: now what?
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Tea shop
Customer
Cup of tea
Hand brewing
Toward batch brewing: a tea urn
Cup
Hot water
Tea leaves
Kettle
Cold water
Power
Tap
Exploit: boil ahead, keep it full
Subordinate: only the orders it can serve
Elevate: buy a hot water dispenser
The constraint moved: now what?
Inertia: staff want to brew every cup by hand
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Two kinds of friction

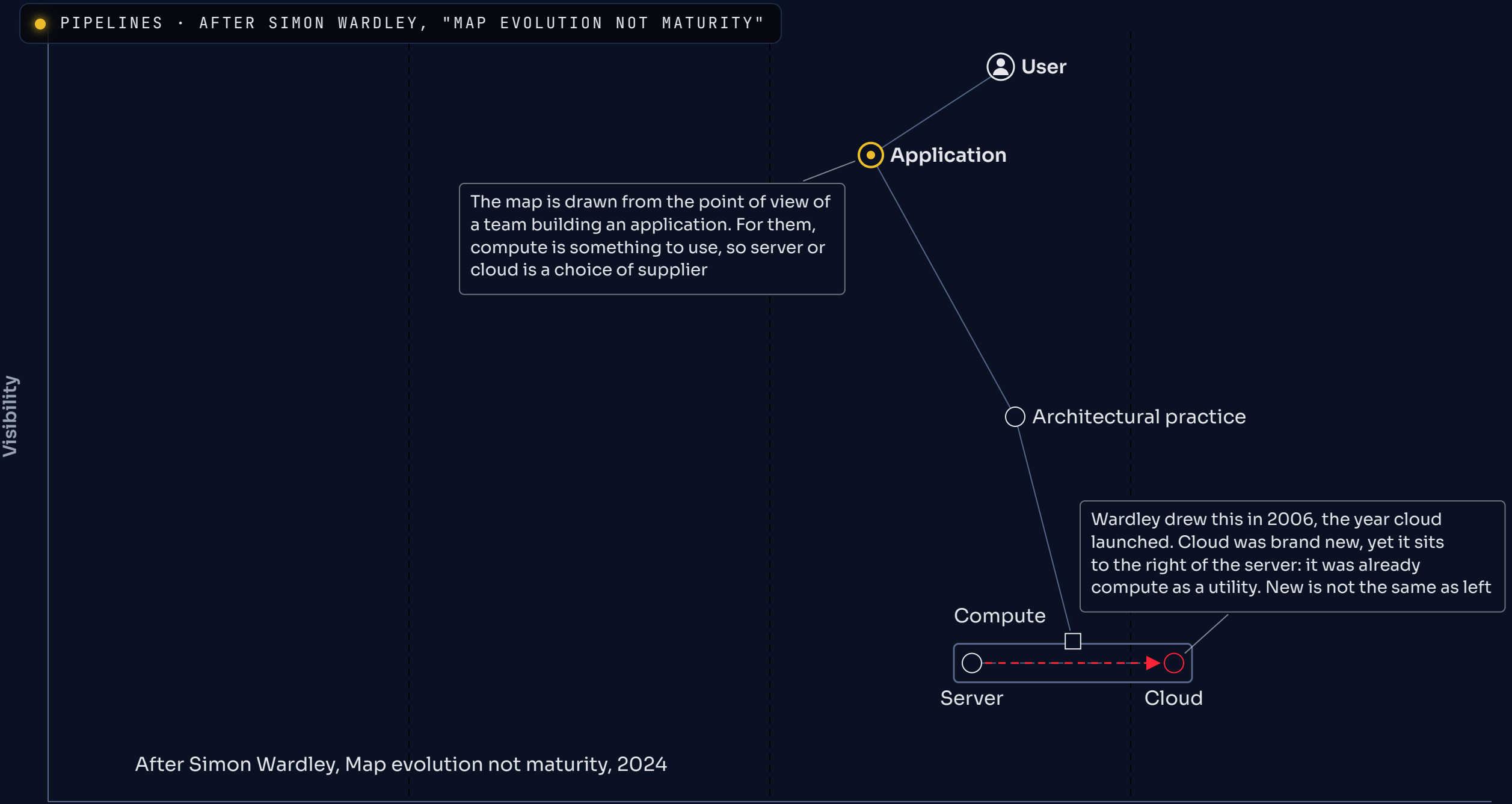
- Constraint: a capacity limit. One kettle, a queue
- Inertia: resistance to change. "We brew by hand"
- A constraint caps the flow; inertia blocks the move
- Work on the constraint first. The rest waits on it

Goldratt: exploit before you elevate

- First ask: is the constraint outside the shop or inside?
- Exploit: get the most out of the constraint as it is
- Subordinate: pace everything else to it
- Elevate: invest only when that is not enough
- Repeat: the constraint moves, often into a practice

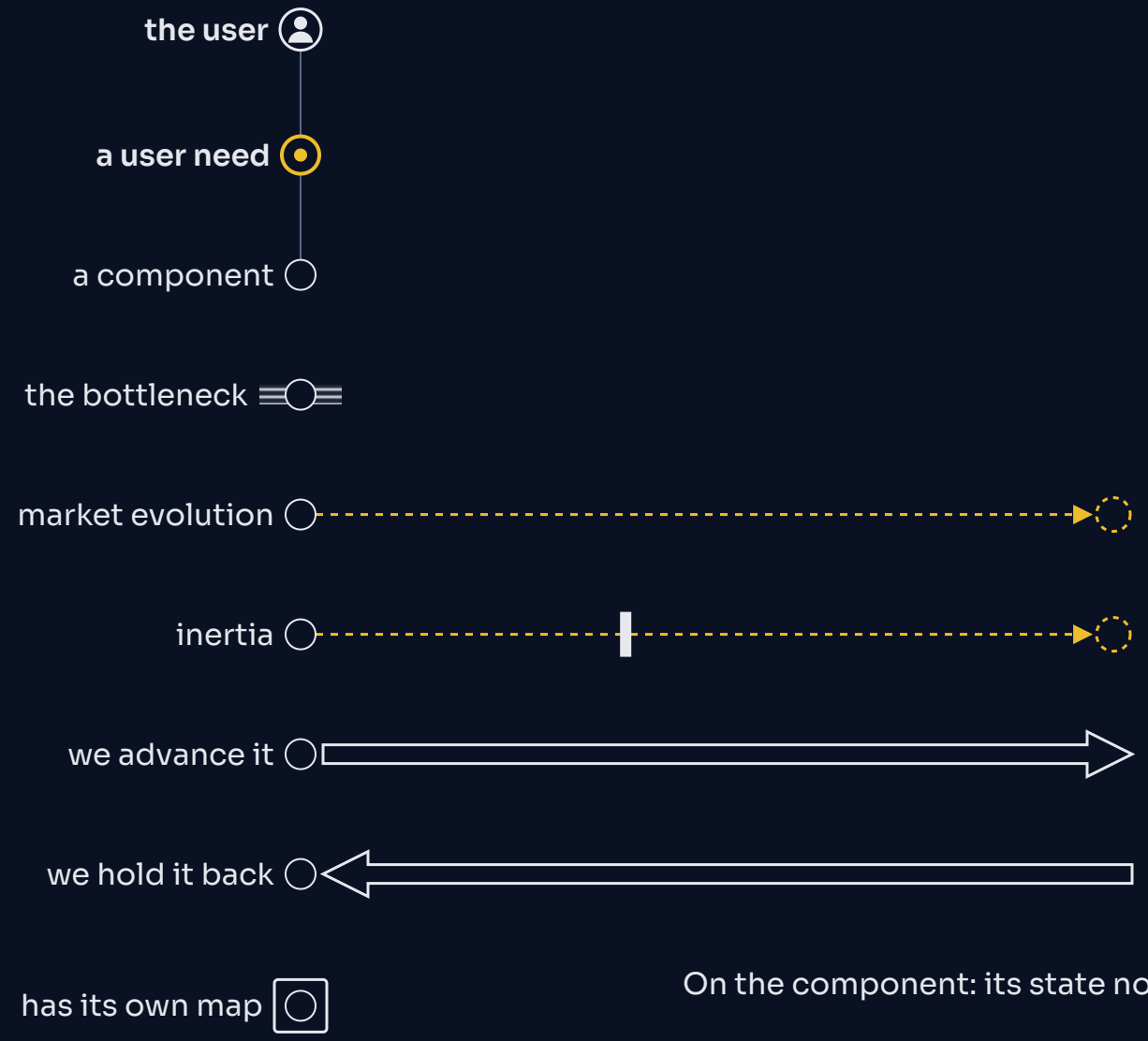
A pipeline: several ways to do one thing

- One component, several ways to do it, side by side in a box
- Each way sits at its own stage of evolution
- A new invention starts on the left
- The industrialised form of an old activity goes on the right, however new it is
- If people move from one way to another, draw an arrow
- Not a delivery pipeline: build, test and release are steps, not ways



After Simon Wardley, Map evolution not maturity, 2024

LEGEND



On the component: its state now

On the evolution line: forces on its move

Exercise 4A: find the constraint

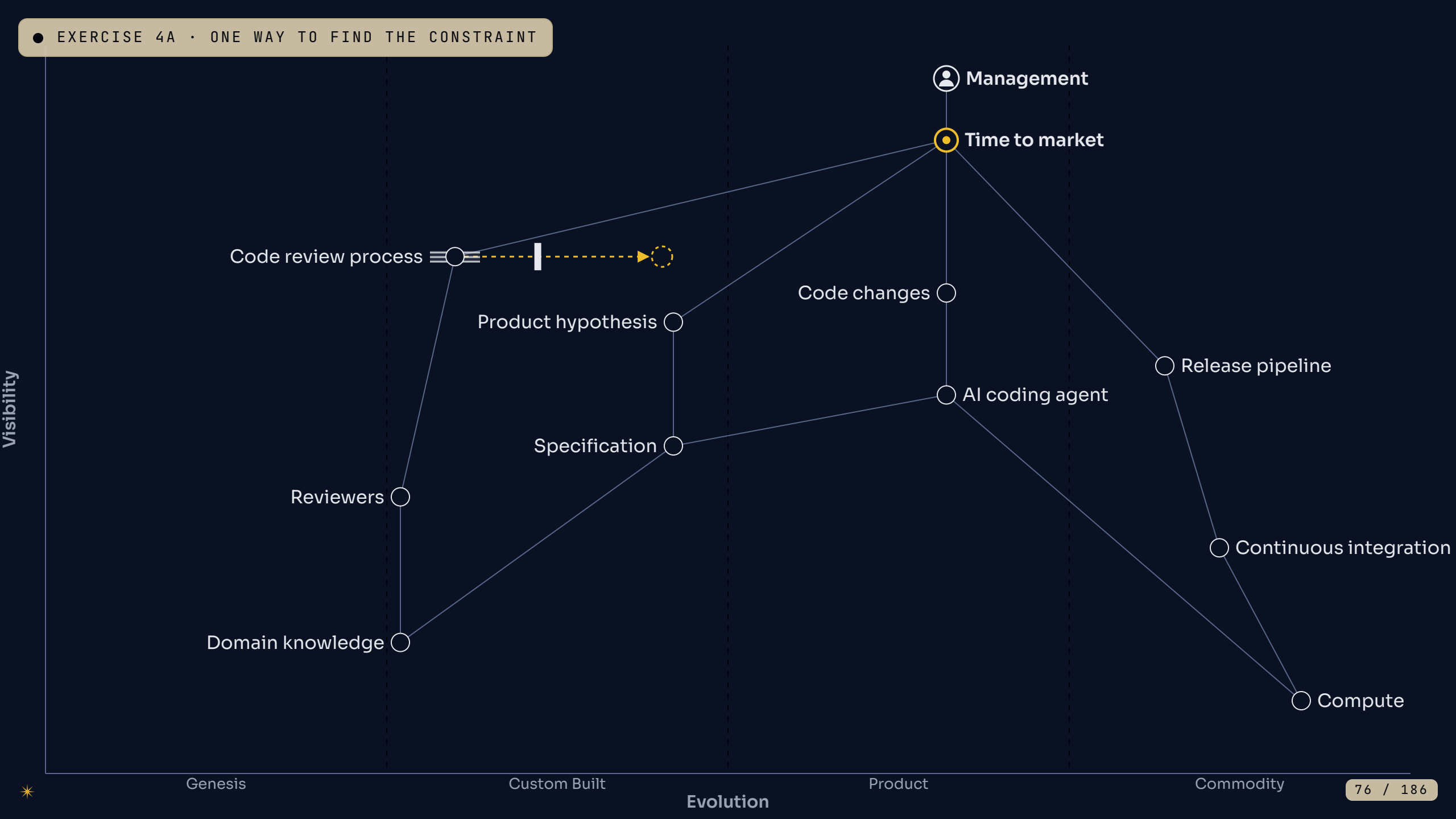
- Keep working on your map
- Mark the constraint: where does work wait longest?
- Mark one inertia: what resists change?
- Exploit the constraint first: what can you do with no budget?
- Subordinate: make the rest of the work serve the constraint
- Then elevate it: what would you invest in?

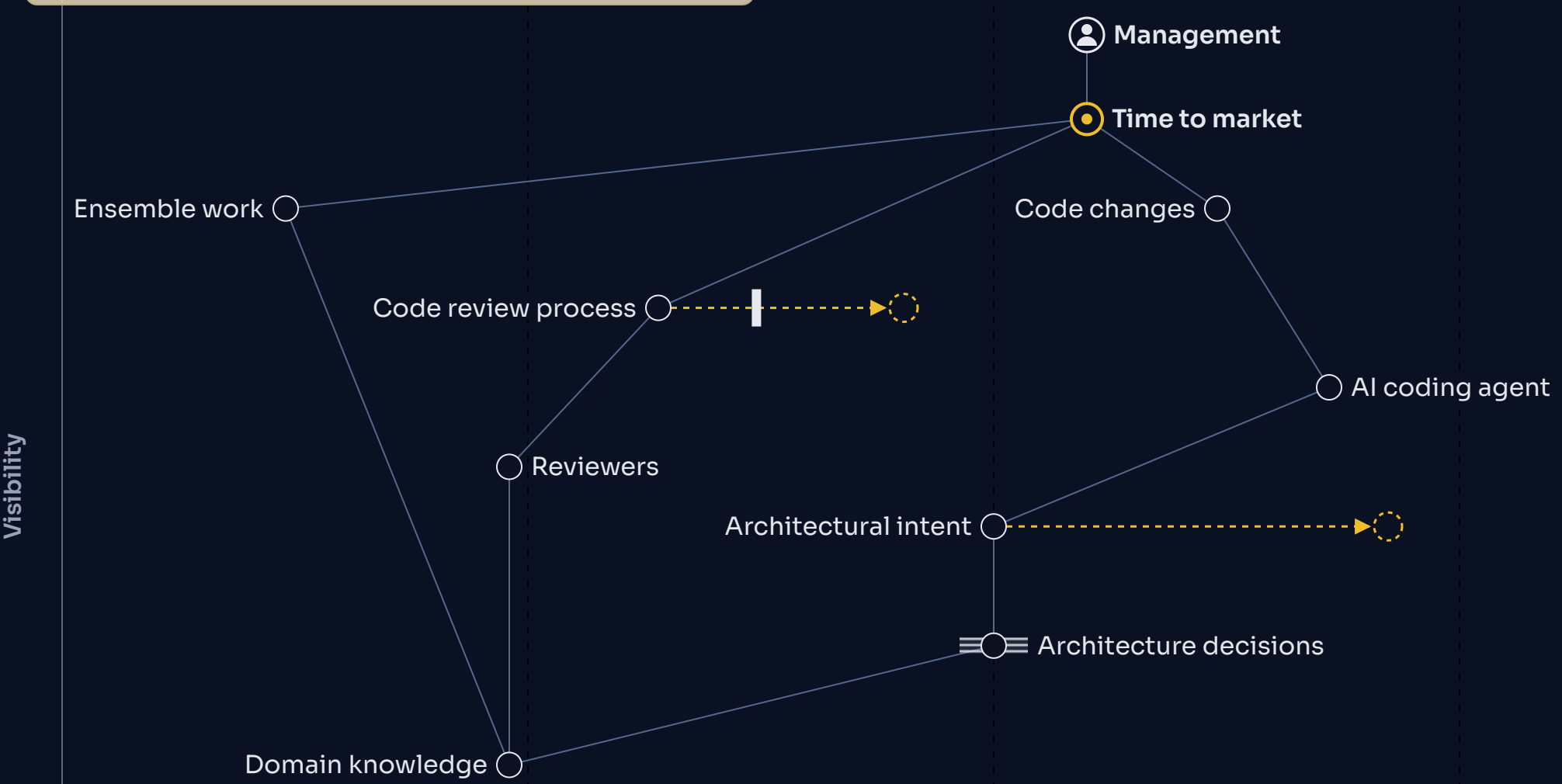
Exercise 4B: draw three pipelines

- A pipeline in Wardley's sense: one component, done several ways
- Writing code: by hand, with an AI assistant, by AI agents
- Checking quality: human review, pairing, automated checks
- Releasing: change board, scripted deploy, continuous deployment
- Place each way at its own stage of evolution, in one box
- Which pipeline moved right? Which one holds the constraint?

Exercise 4: discuss the constraint and the pipelines

- Is your constraint a practice or a technology?
- Which of your exploit moves needs no budget?
- Where does the constraint move after you elevate it?
- Which way does your team use today, in each pipeline?
- Did a newer way remove the constraint, or only move it?

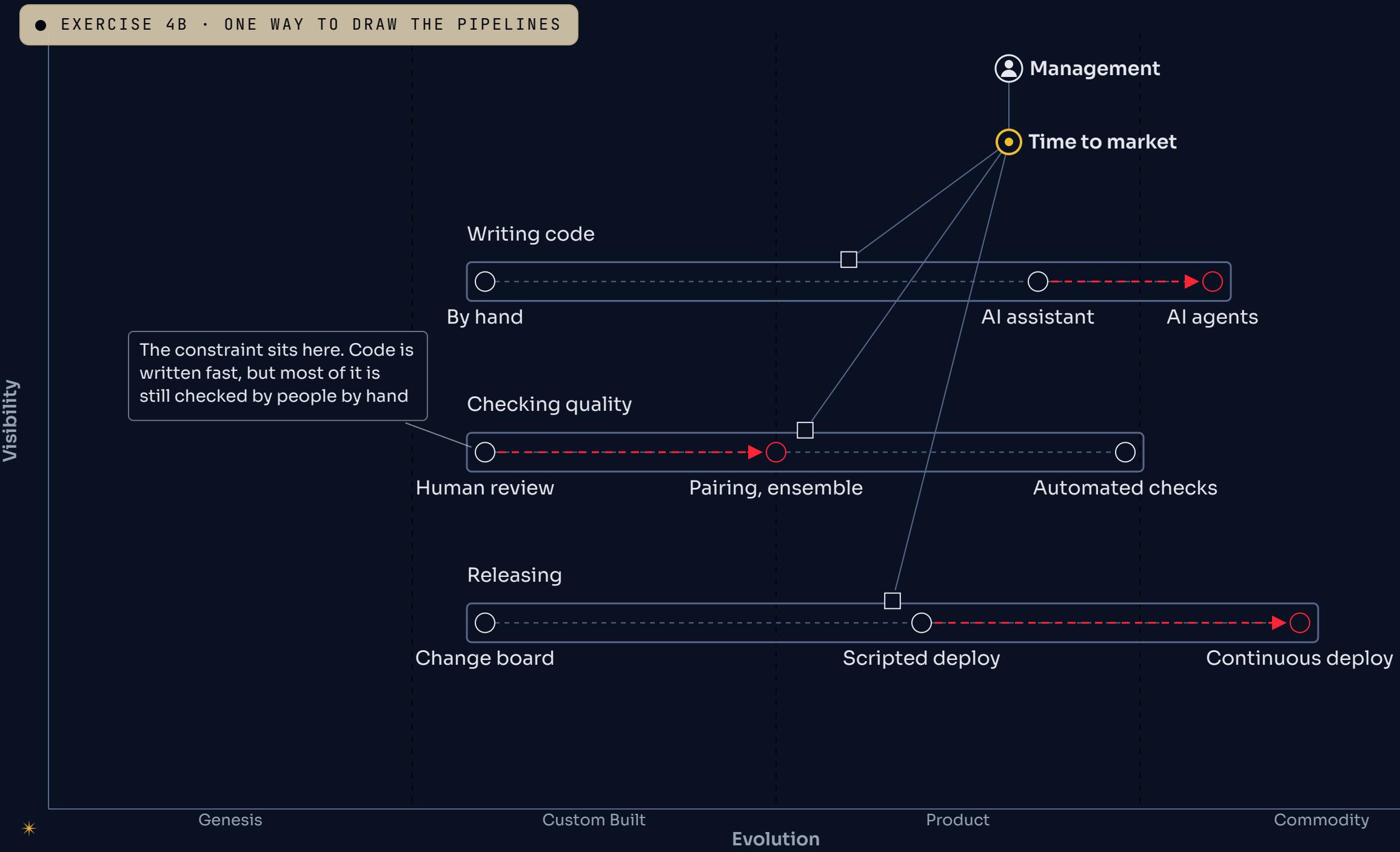

Management
Time to market
Code review process
Code changes
Product hypothesis
Release pipeline
AI coding agent
Specification
Reviewers
Continuous integration
Domain knowledge
Compute
Visibility
Genesis
Custom Built
Product
Commodity
Evolution



Exploit: small pull requests, review before starting new work

Elevate: build it together, and turn the architecture rules into automated checks

The constraint moves to architecture decisions: the architect cannot decide fast enough


Management
Time to market
Writing code
By hand
AI assistant
AI agents
The constraint sits here. Code is written fast, but most of it is still checked by people by hand
Checking quality
Human review
Pairing, ensemble
Automated checks
Releasing
Change board
Scripted deploy
Continuous deploy
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

04

Strategy and the OODA loop

STRATEGY

Strategy and tactics

"Tactics without strategy is the noise before defeat."

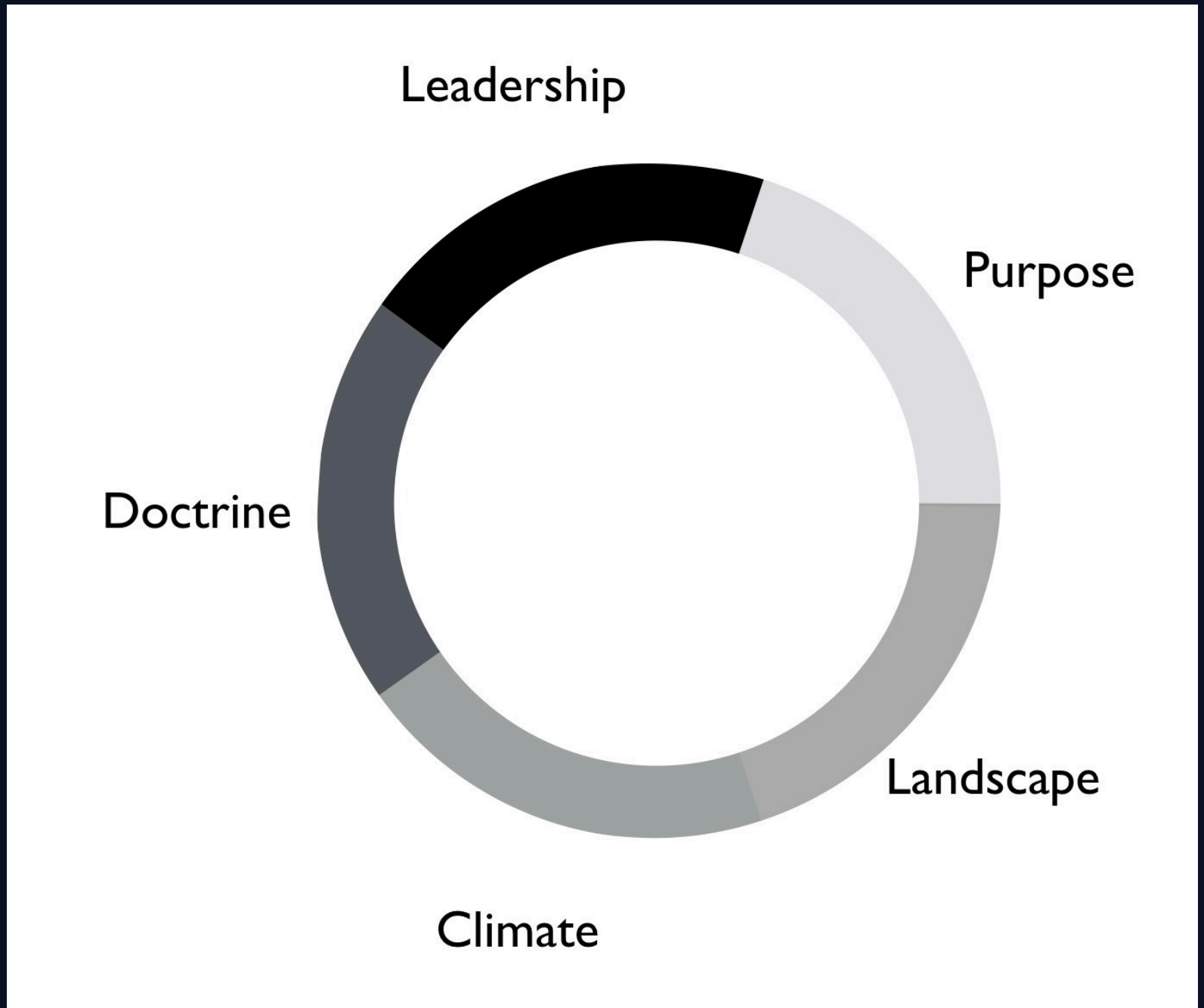
"Strategy without tactics is the slowest route to victory."

— attributed to Sun Tzu

PHOTO: MARTIN MAKARYAN, UNSPLASH

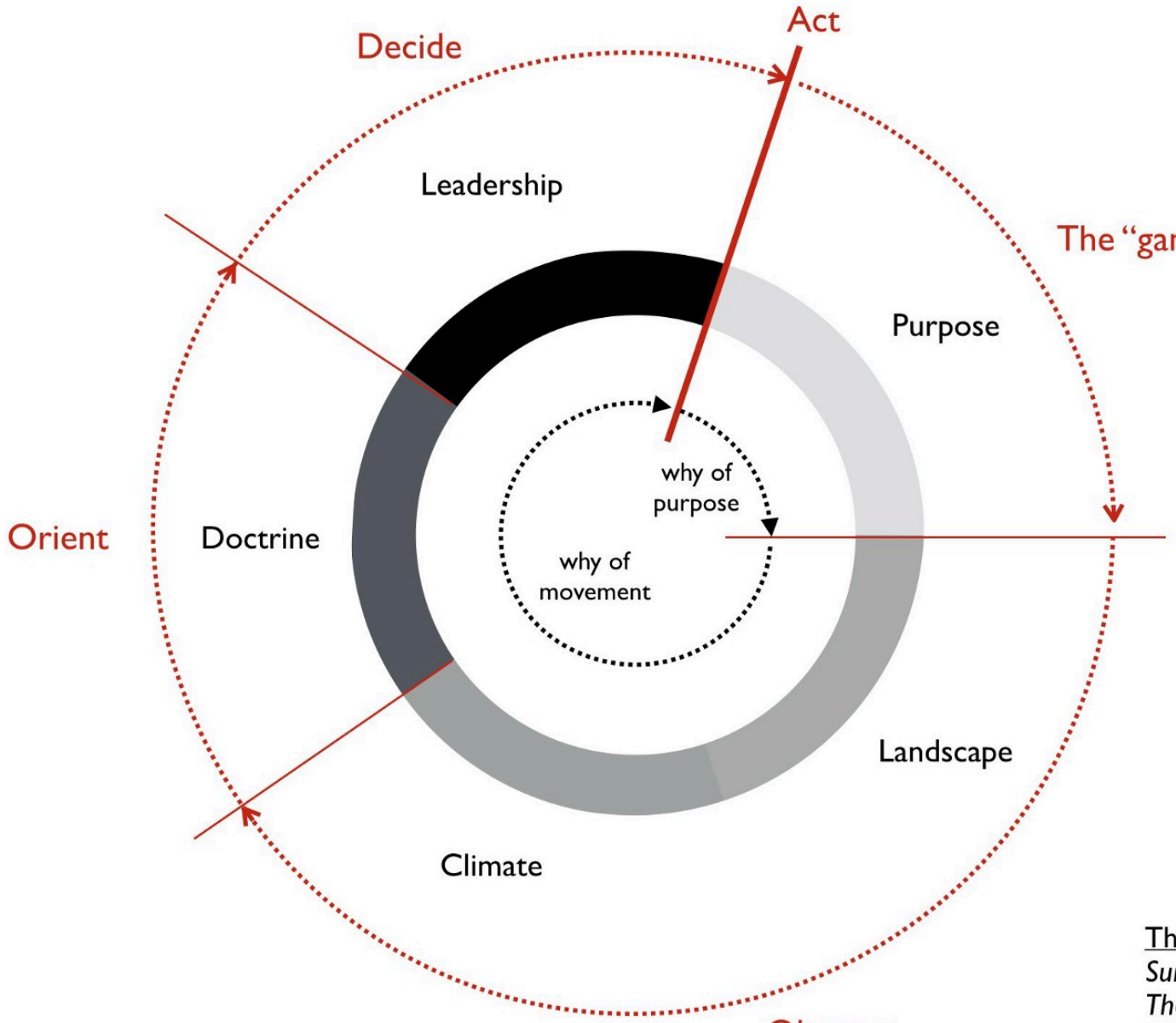
Sun Tzu’s five factors

- Purpose — why we compete
- Landscape — where we stand
- Climate — what changes around us
- Doctrine — training and standard ways of operating
- Leadership — context-dependent strategic decisions


Leadership
Purpose
Doctrine
Landscape
Climate

John Boyd's OODA loop: observe, orient, decide, act

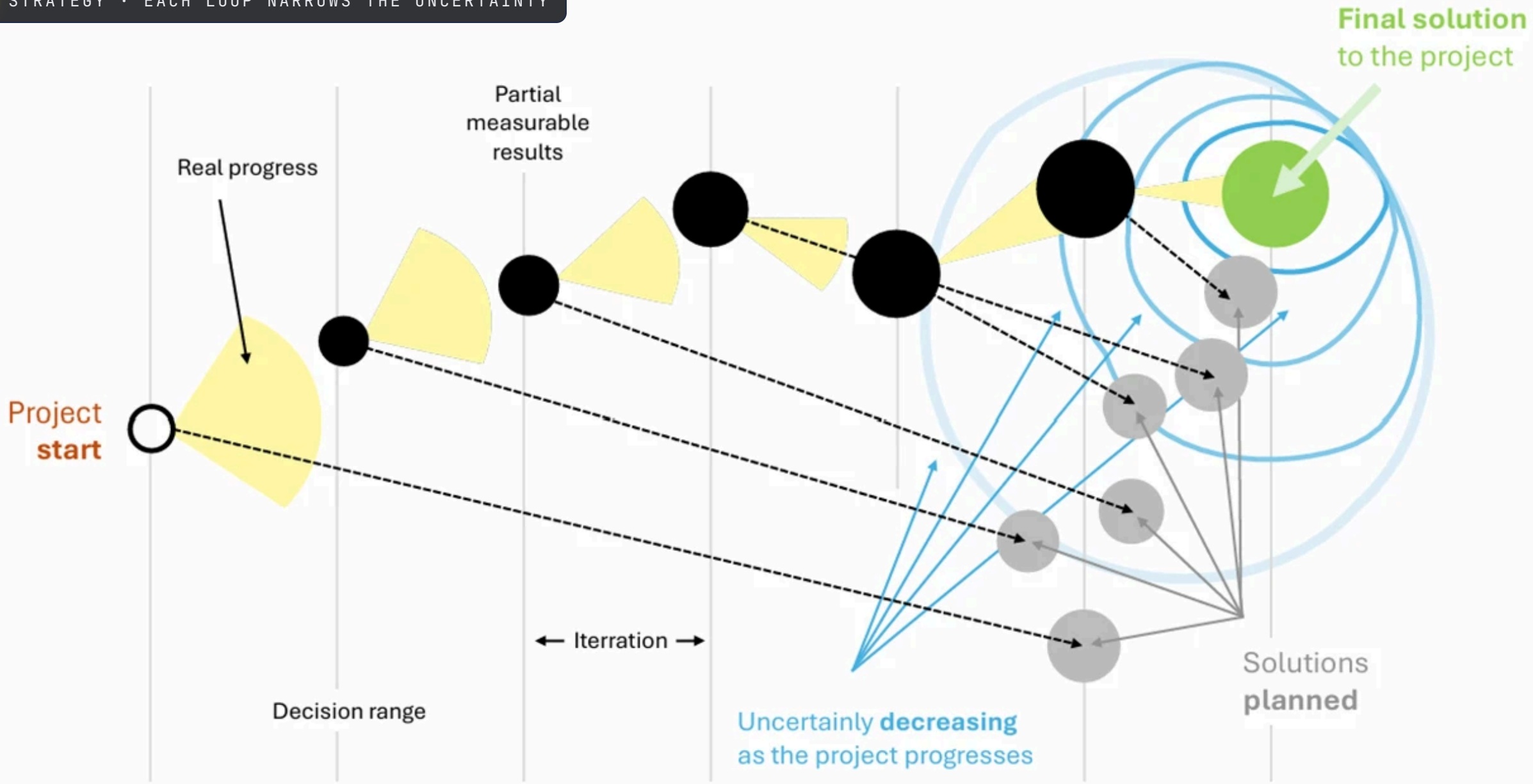
- Observe: look at the landscape and at what is changing around it, the climate
- Orient: make sense of it with the principles you trust, the doctrine
- Decide: choose the move, which is leadership
- Act: make the move, then observe again, because the landscape has changed
- Boyd was a fighter pilot. The side that goes round the loop faster wins, even with less
- Wardley puts the five factors on this loop. That is his strategy cycle, next slide



The Strategy Cycle
Sun Tzu's five factors
The two types of why
John Boyd's OODA loop

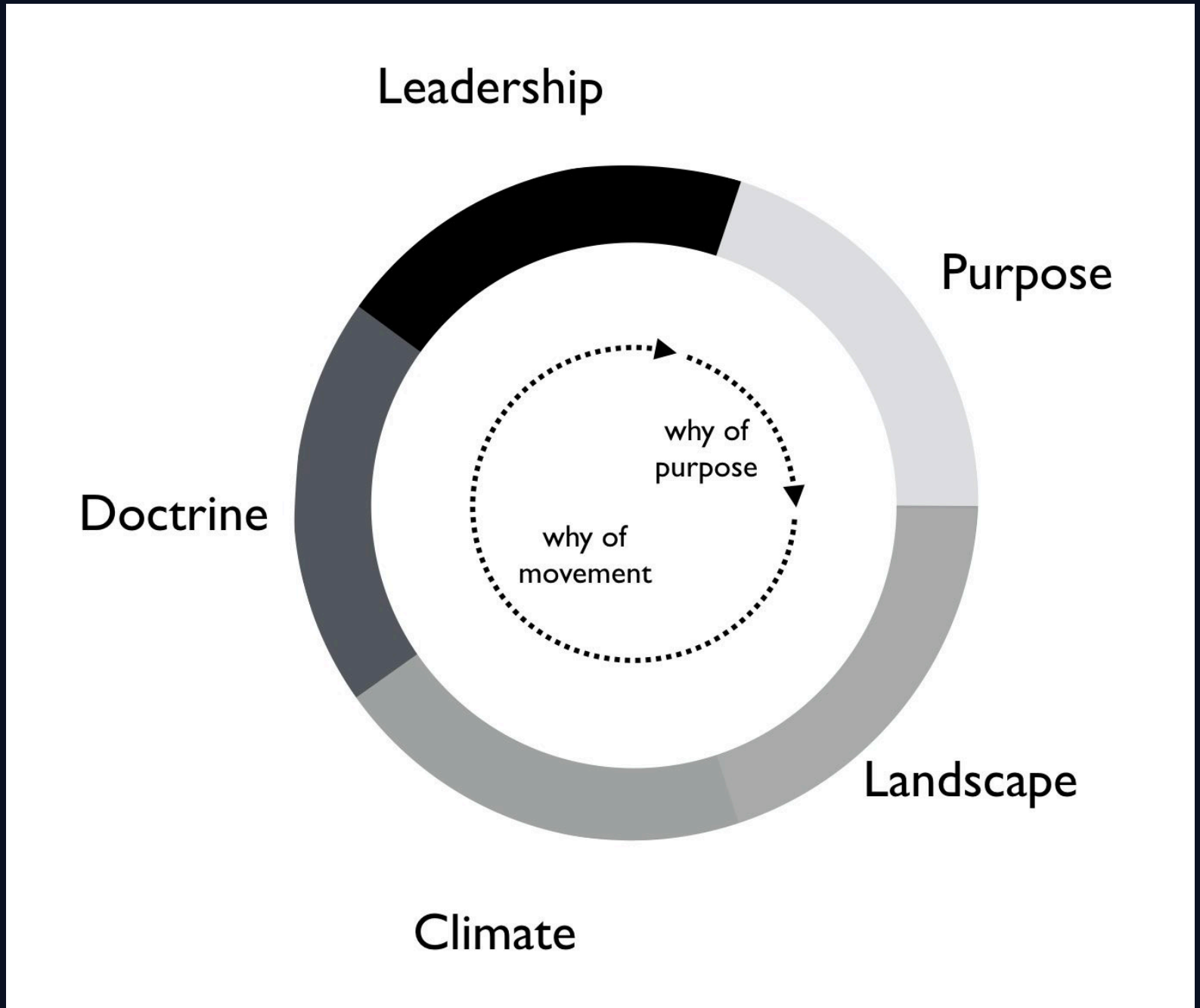
Category	**Wardley's Doctrine** (universally useful patterns that a user can apply)			
Communication	Be transparent *(a bias towards open)*	Focus on high situational awareness *(understand what is being considered)*	Use a common language *(necessary for collaboration)*	Challenge assumptions *(speak up and question)*
Development	Know your users *(e.g. customers, shareholders, regulators, staff)*	Focus on user needs	Think fast, inexpensive, restrained and elegant (FIRE, formerly FIST)	Remove bias and duplication
	Use appropriate methods *(e.g. agile vs lean vs six sigma)*	Focus on the outcome not a contract *(e.g. worth based development)*	Be pragmatic *(it doesn't matter if the cat is black or white as long as it catches mice)*	Use standards where appropriate
	Use appropriate tools *(e.g. mapping, financial models)*			
Operation	Manage inertia *(e.g. existing practice, political capital, previous investment)*	Optimise flow *(remove bottlenecks)*	Think small *(as in know the details)*	Effectiveness over efficiency
	Do better with less *(continual improvement)*	Set exceptional standards *(great is just not good enough)*	Manage failure	
Structure	Provide purpose, mastery & autonomy	Think small *(as in teams)*	Distribute power and decision making	Think aptitude and attitude
	Design for constant evolution	There is no one culture *(e.g. pioneers, settlers and town planners)*	Seek the best	
Learning	Use a systematic mechanism of learning *(a bias towards data)*	A bias towards action *(learn by playing the game)*	A bias towards the new *(be curious, take appropriate risks)*	Listen to your ecosystems *(acts as future sensing engines)*
Leading	Be the owner *(take responsibility)*	Move fast *(an imperfect plan executed today is better than a perfect plan executed tomorrow)*	Think big *(inspire others, provide direction)*	Strategy is iterative not linear *(fast reactive cycles)*
	Strategy is complex *(there will be uncertainty)*	Commit to the direction, be adaptive along the path *(crossing the river by feeling the stones)*	There is no core *(everything is transient)*	Be humble *(listen, be selfless, have fortitude)*
	Exploit the landscape			

Phase I
Phase II
Phase III
Phase IV


Final solution
to the project
Partial
measurable
results
Real progress
Project
start
Iterration
Decision range
Uncertainly decreasing
as the project progresses
Solutions
planned

Wardley’s two kinds of why

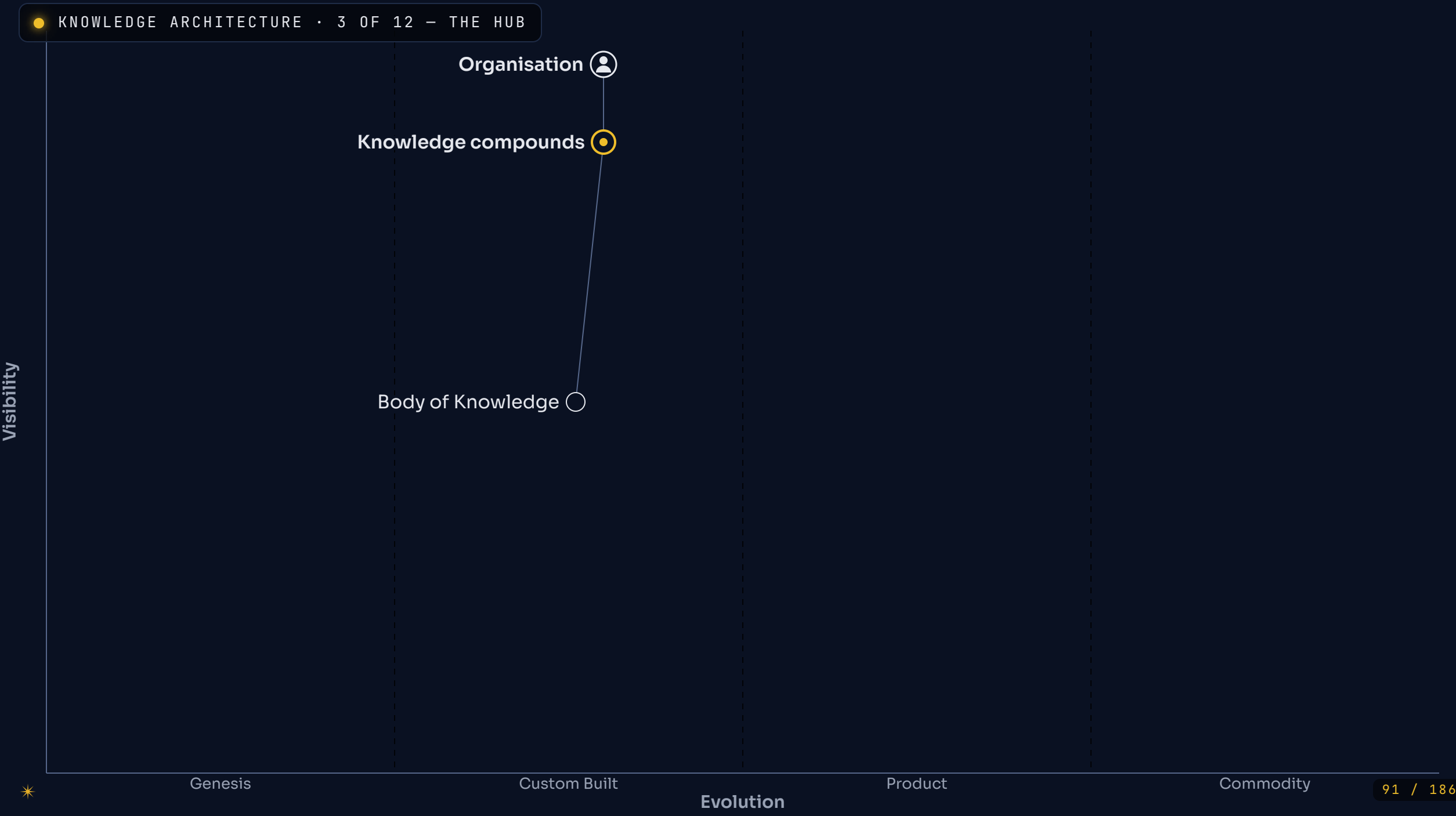
- Why of purpose: why we play. In chess: to win
- Why of movement: why this move over that one
- Leaders often give the first why and skip the second
- A map lets us argue the why of movement

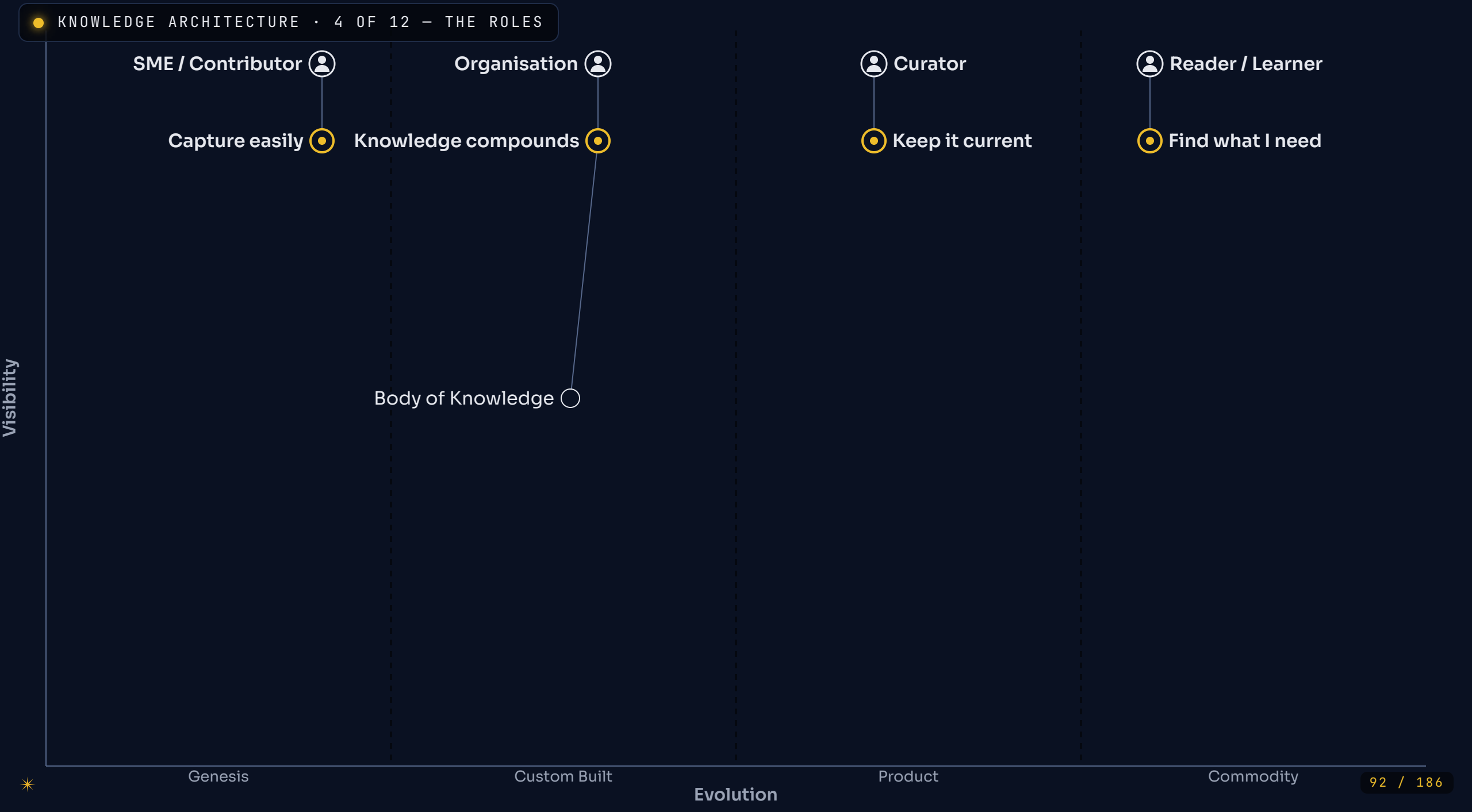

Leadership
Purpose
Doctrine
why of purpose
why of movement
Landscape
Climate

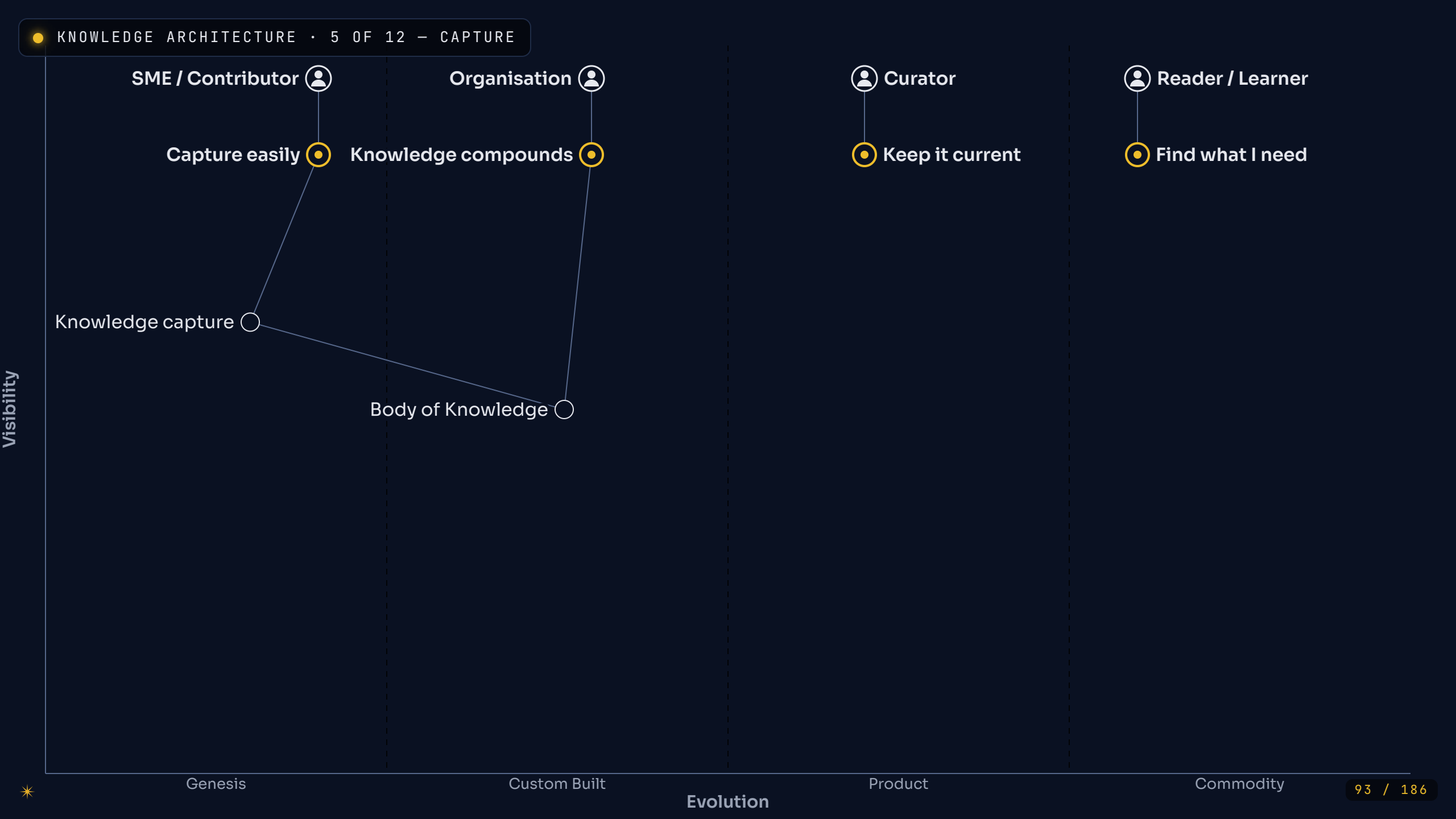
Organisation
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

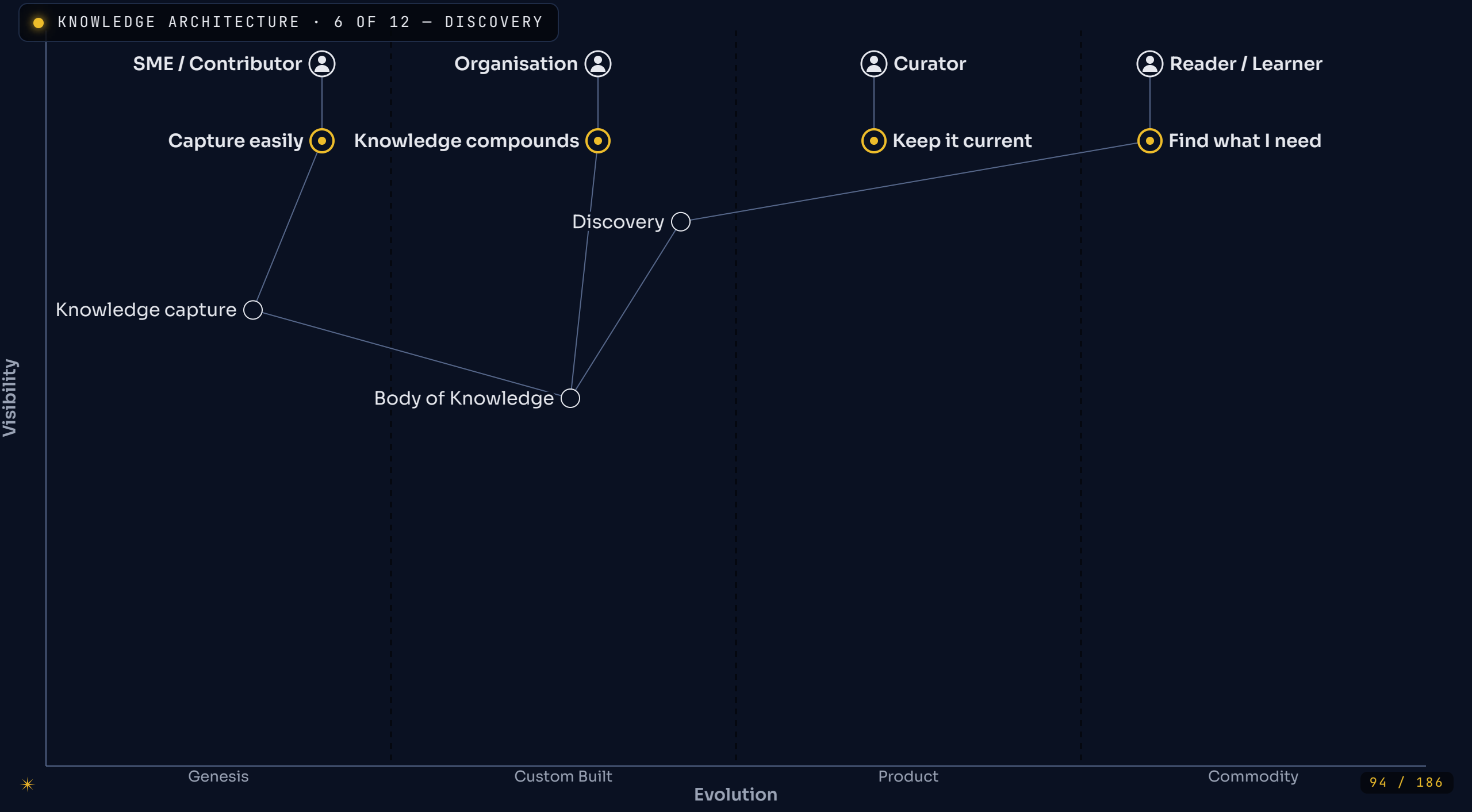
Organisation
Knowledge compounds
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

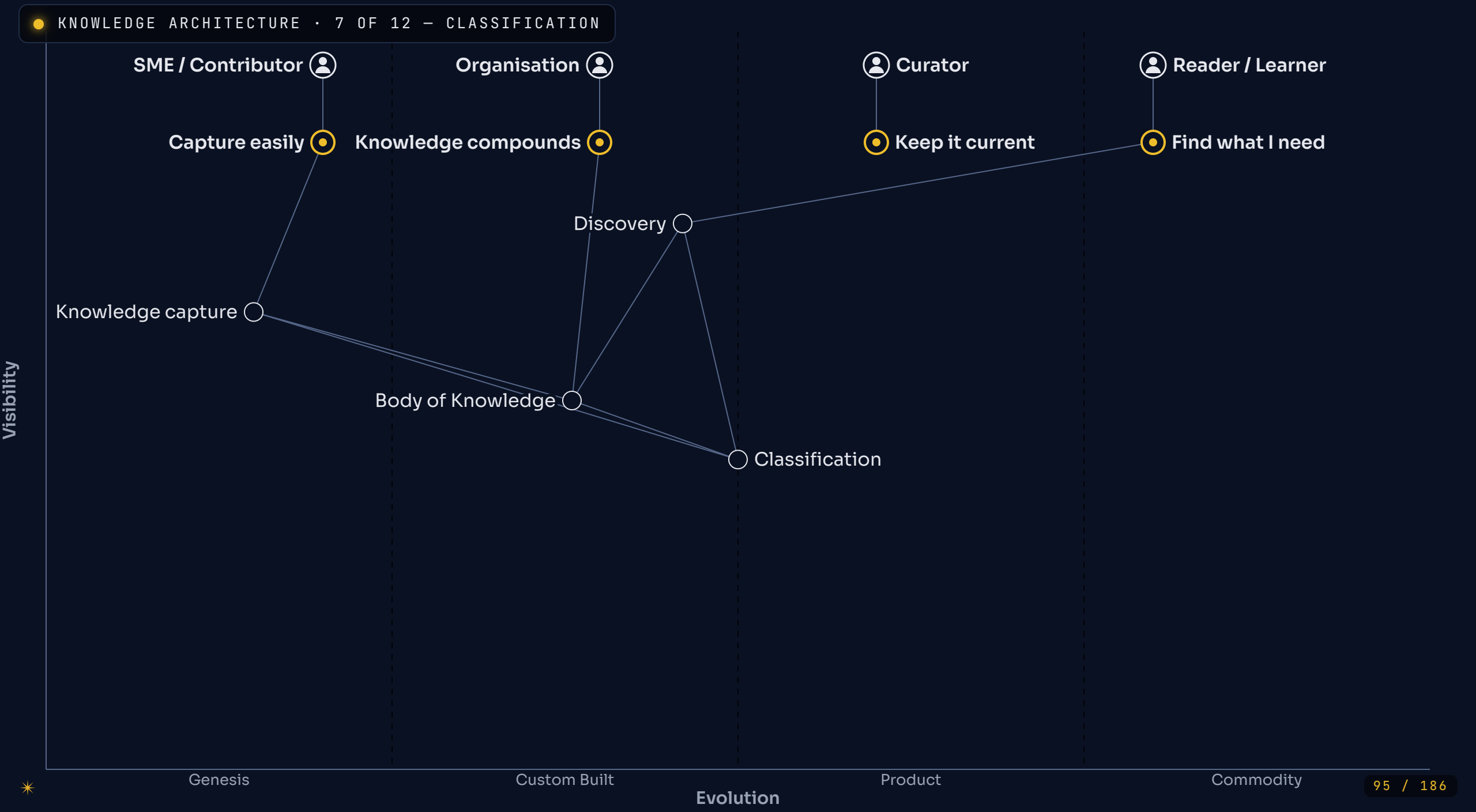
KNOWLEDGE ARCHITECTURE · 3 OF 12 — THE HUB

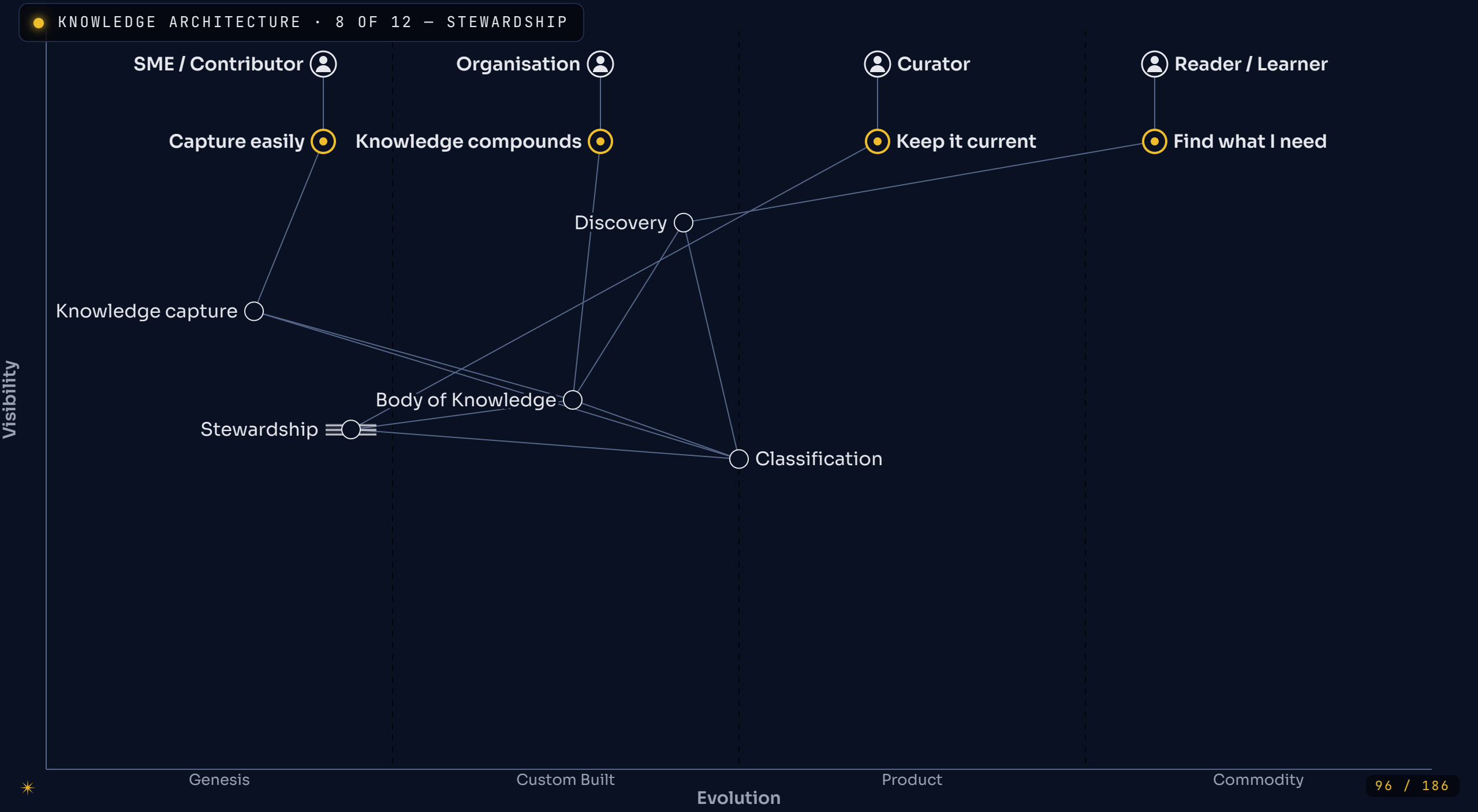


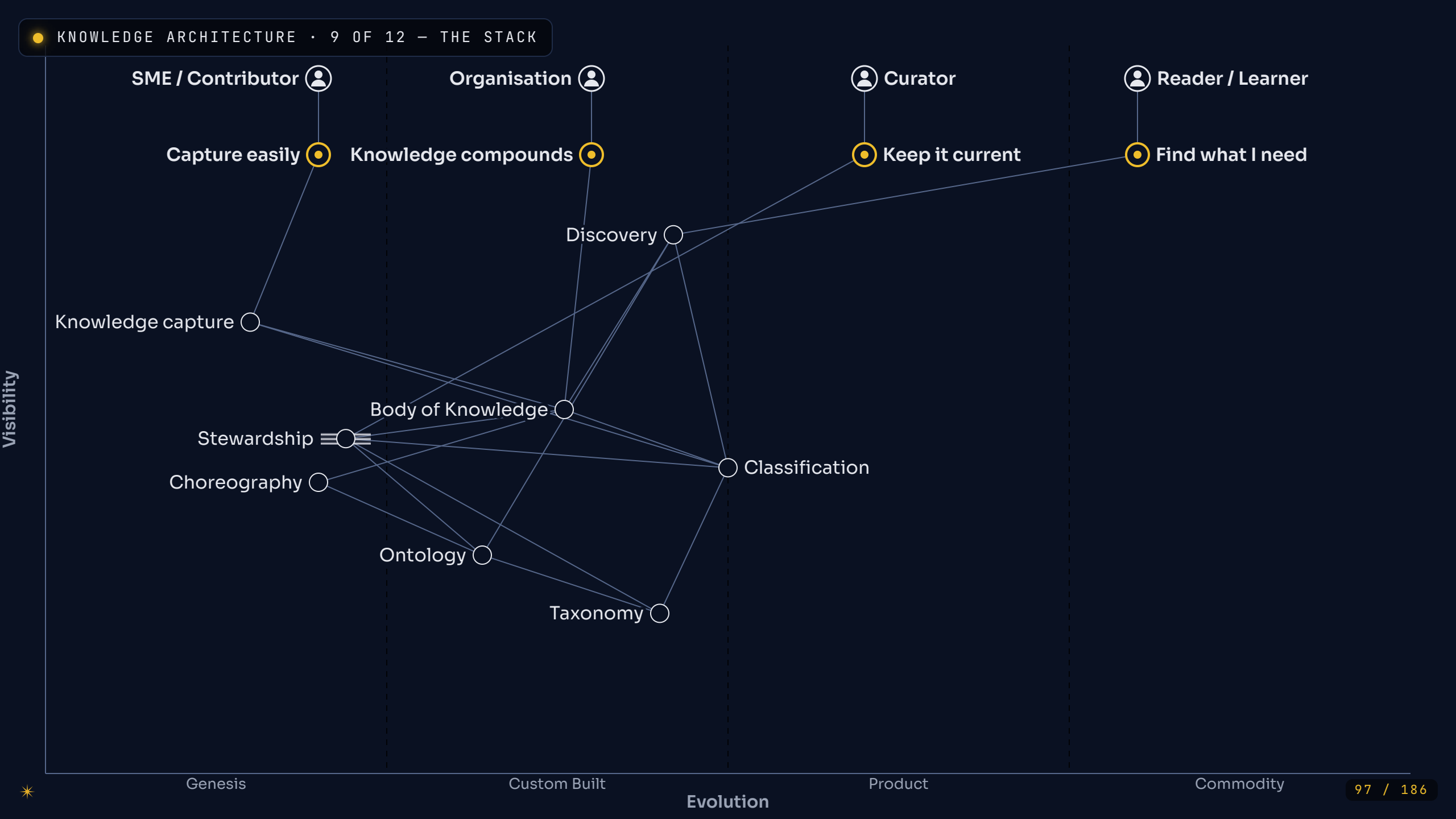

SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

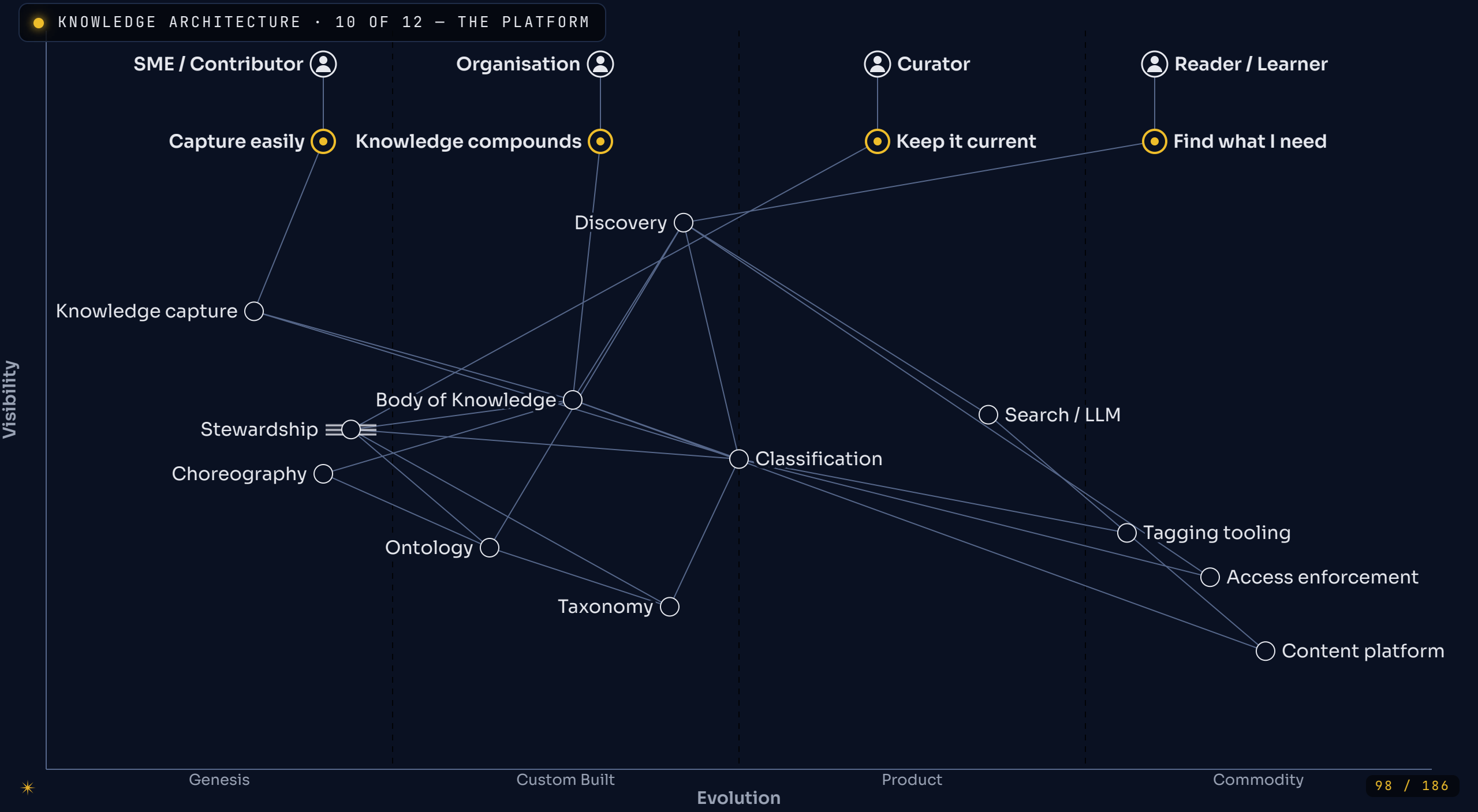

SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Knowledge capture
Body of Knowledge
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

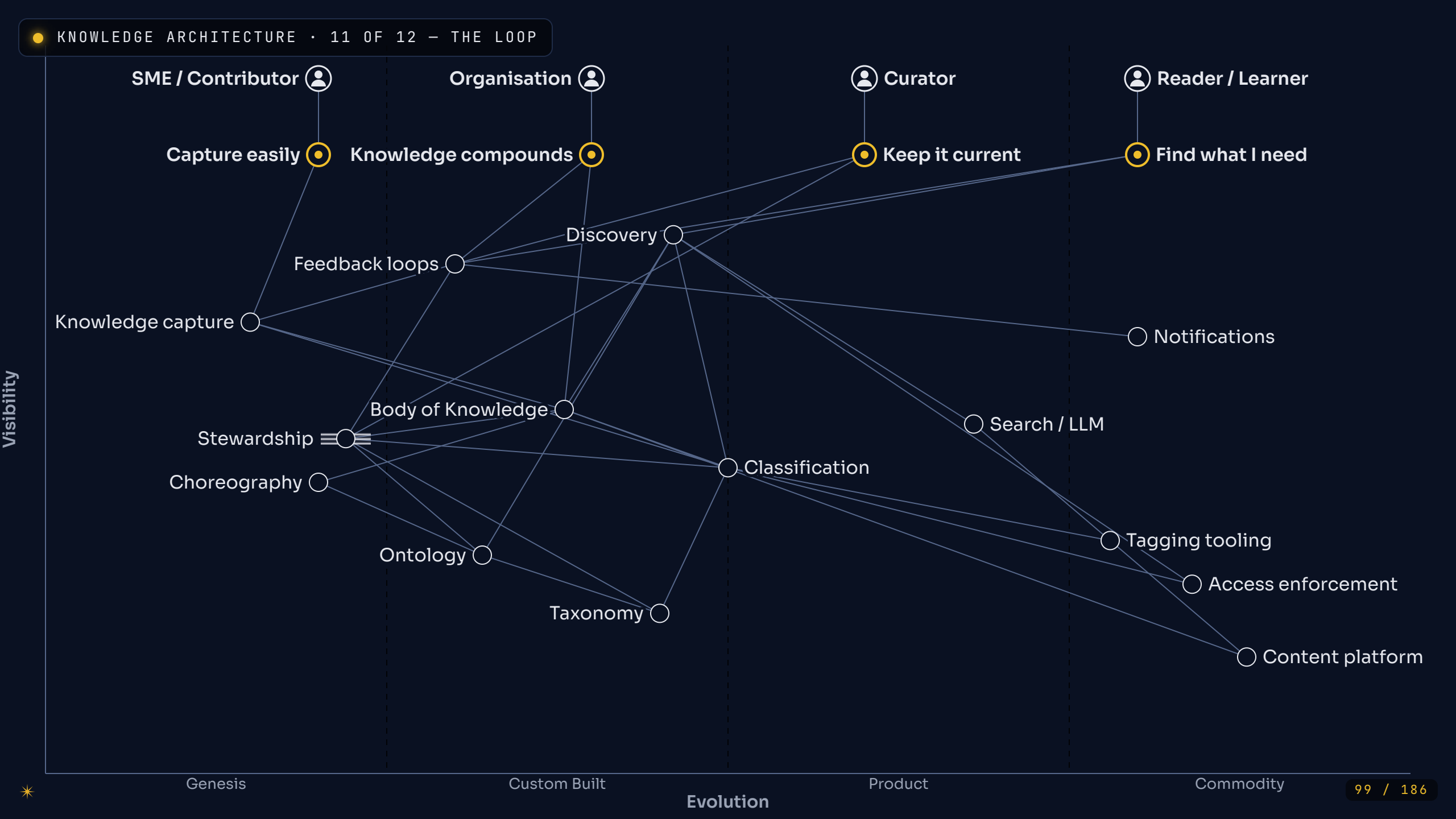

SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Knowledge capture
Body of Knowledge
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

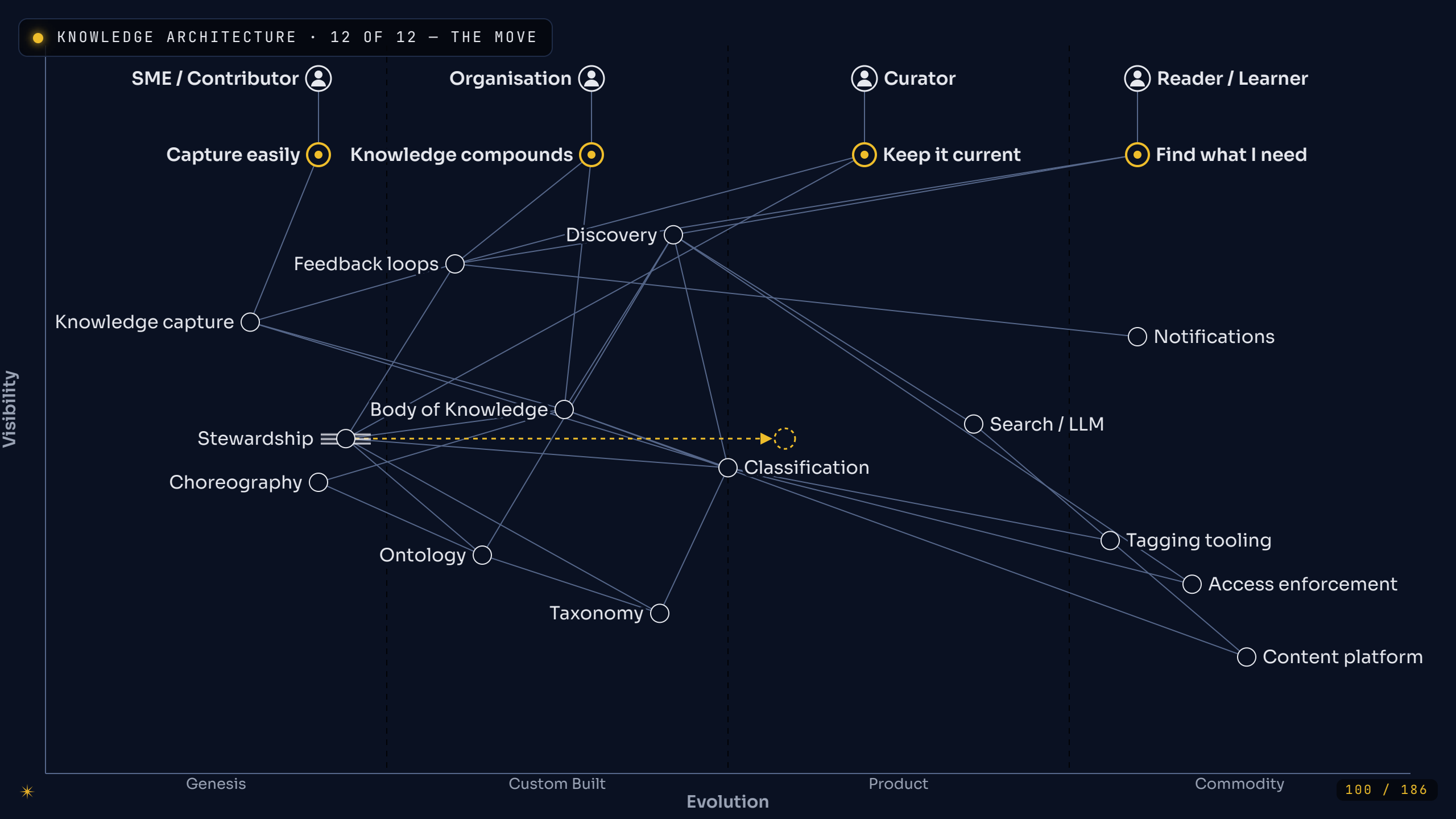

SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Knowledge capture
Body of Knowledge
Classification
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Knowledge capture
Body of Knowledge
Stewardship
Classification
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Knowledge capture
Body of Knowledge
Stewardship
Classification
Choreography
Ontology
Taxonomy
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Knowledge capture
Body of Knowledge
Stewardship
Choreography
Search / LLM
Classification
Tagging tooling
Ontology
Access enforcement
Taxonomy
Content platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Feedback loops
Knowledge capture
Notifications
Body of Knowledge
Search / LLM
Stewardship
Classification
Choreography
Tagging tooling
Ontology
Access enforcement
Taxonomy
Content platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


SME / Contributor
Organisation
Curator
Reader / Learner
Capture easily
Knowledge compounds
Keep it current
Find what I need
Discovery
Feedback loops
Knowledge capture
Notifications
Body of Knowledge
Search / LLM
Stewardship
Classification
Choreography
Tagging tooling
Ontology
Access enforcement
Taxonomy
Content platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

A map with no question

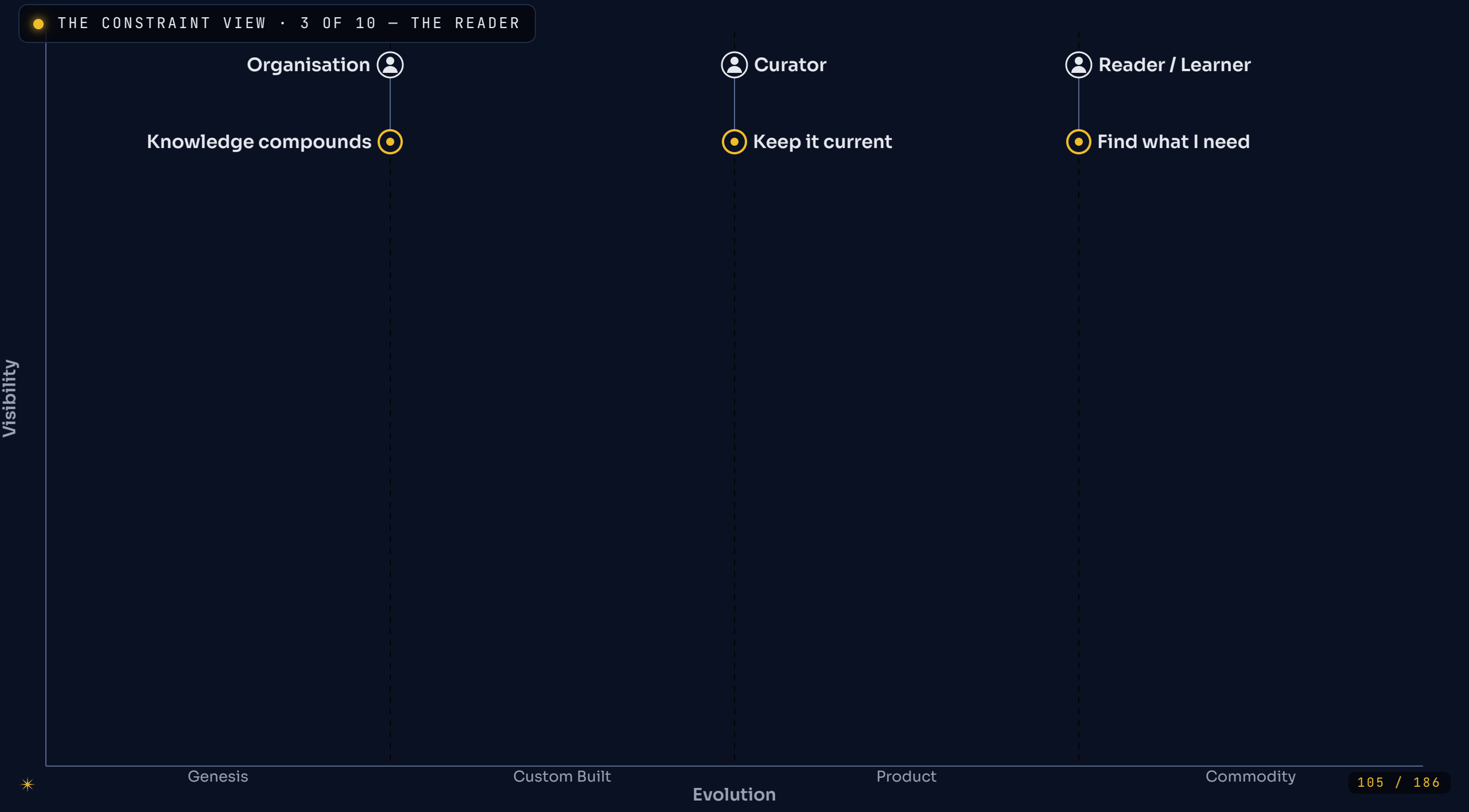
- Twenty components, thirty lines
- Everything is true, nothing is decided
- Ask: which question does this map answer?
- Keep what changes the answer, fold the rest

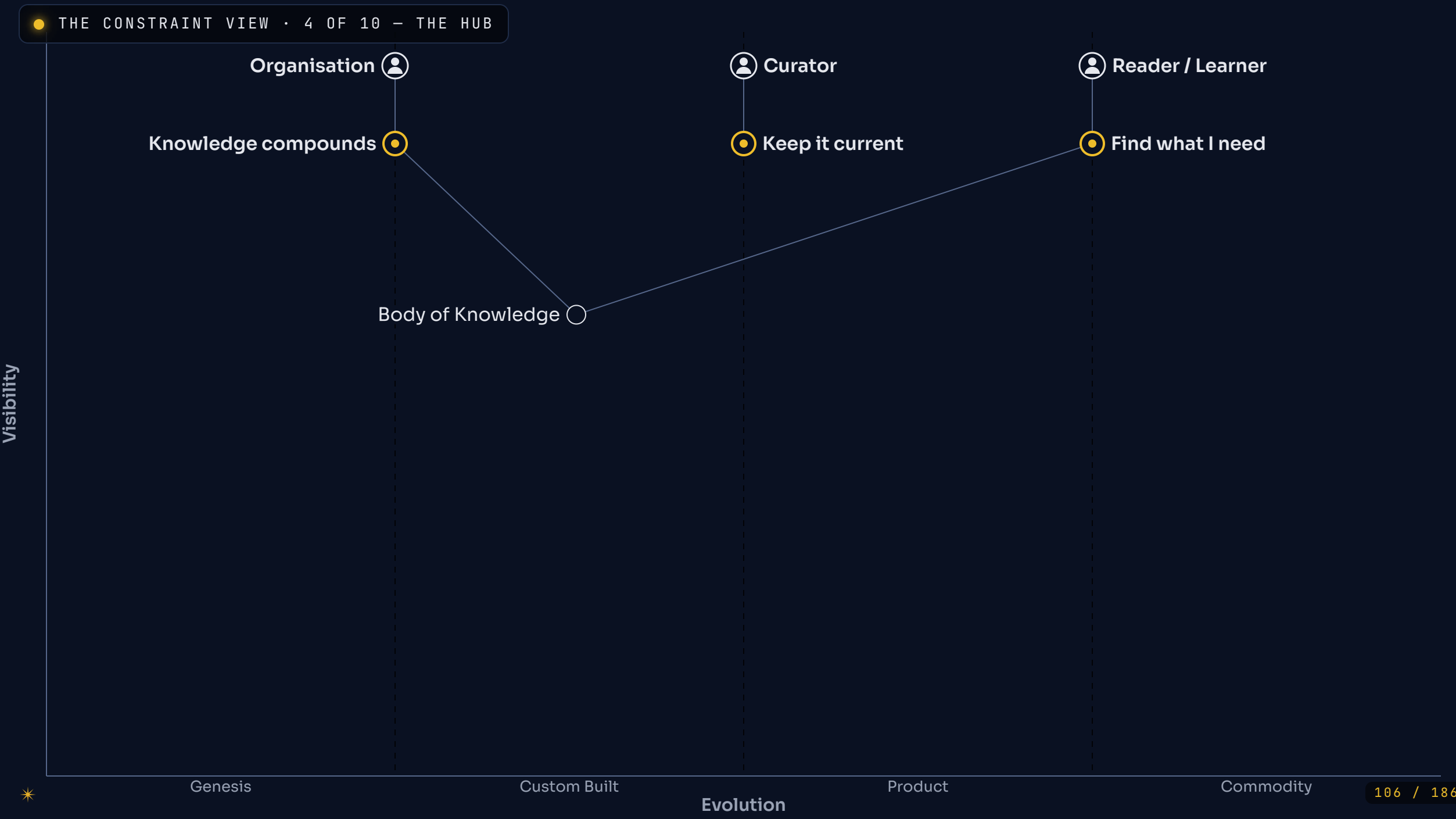
Four ways to make a map readable

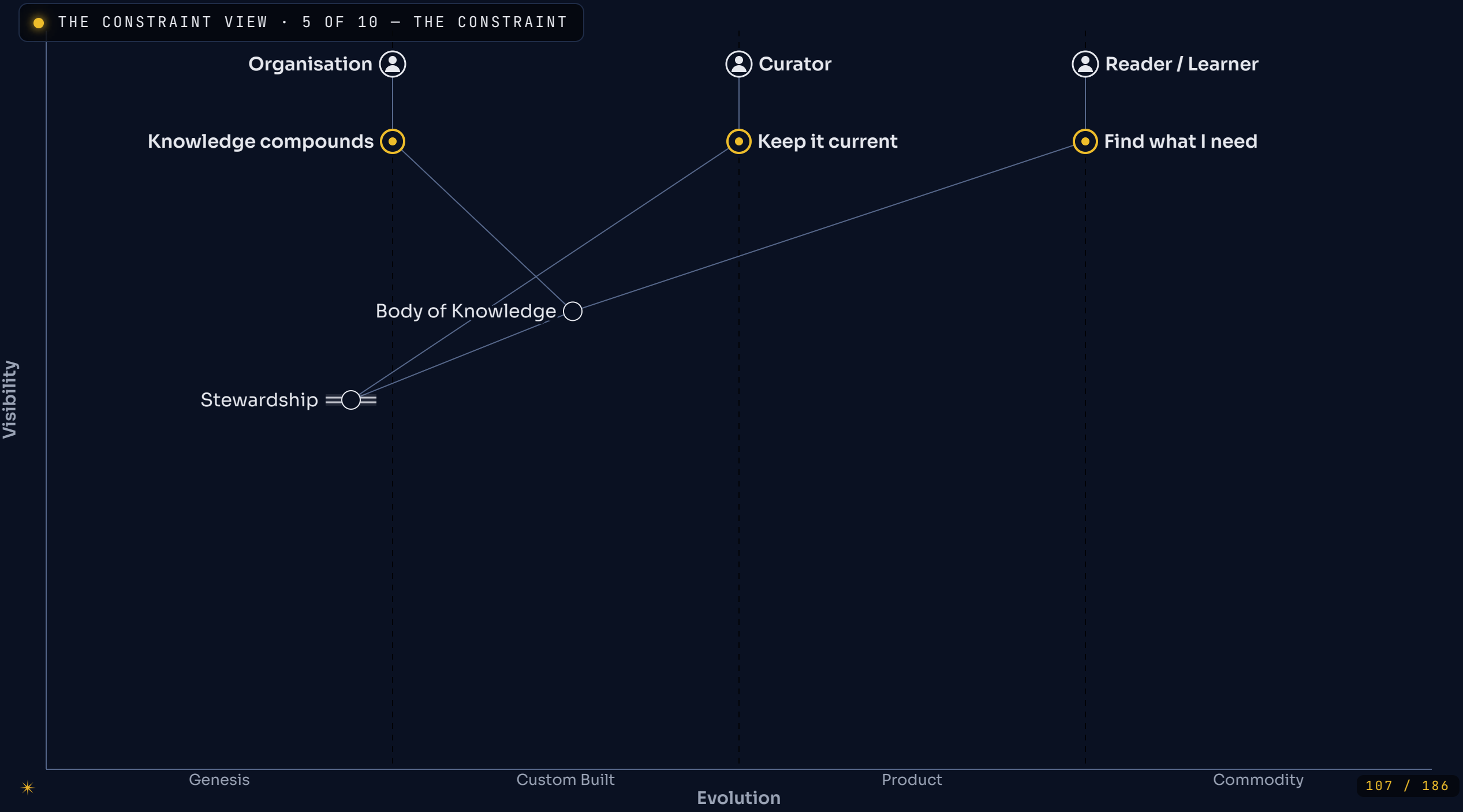
- One question per map: keep what changes the answer
- One map per user, when their needs really differ
- Map around the constraint: the chain to it, nothing else
- Fold a cluster into a submap
- Next: the same map, seen from its constraint

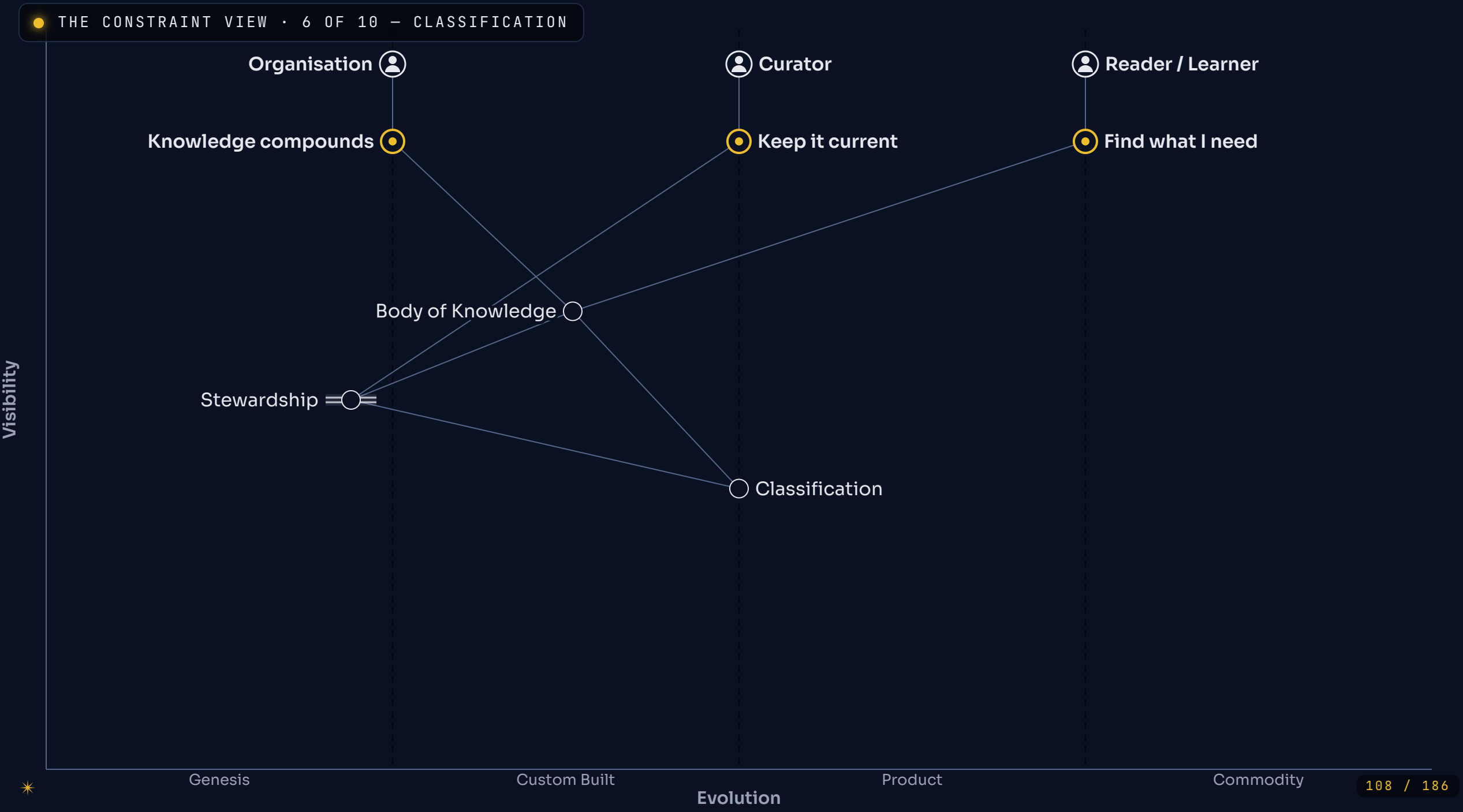
Organisation
Knowledge compounds
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

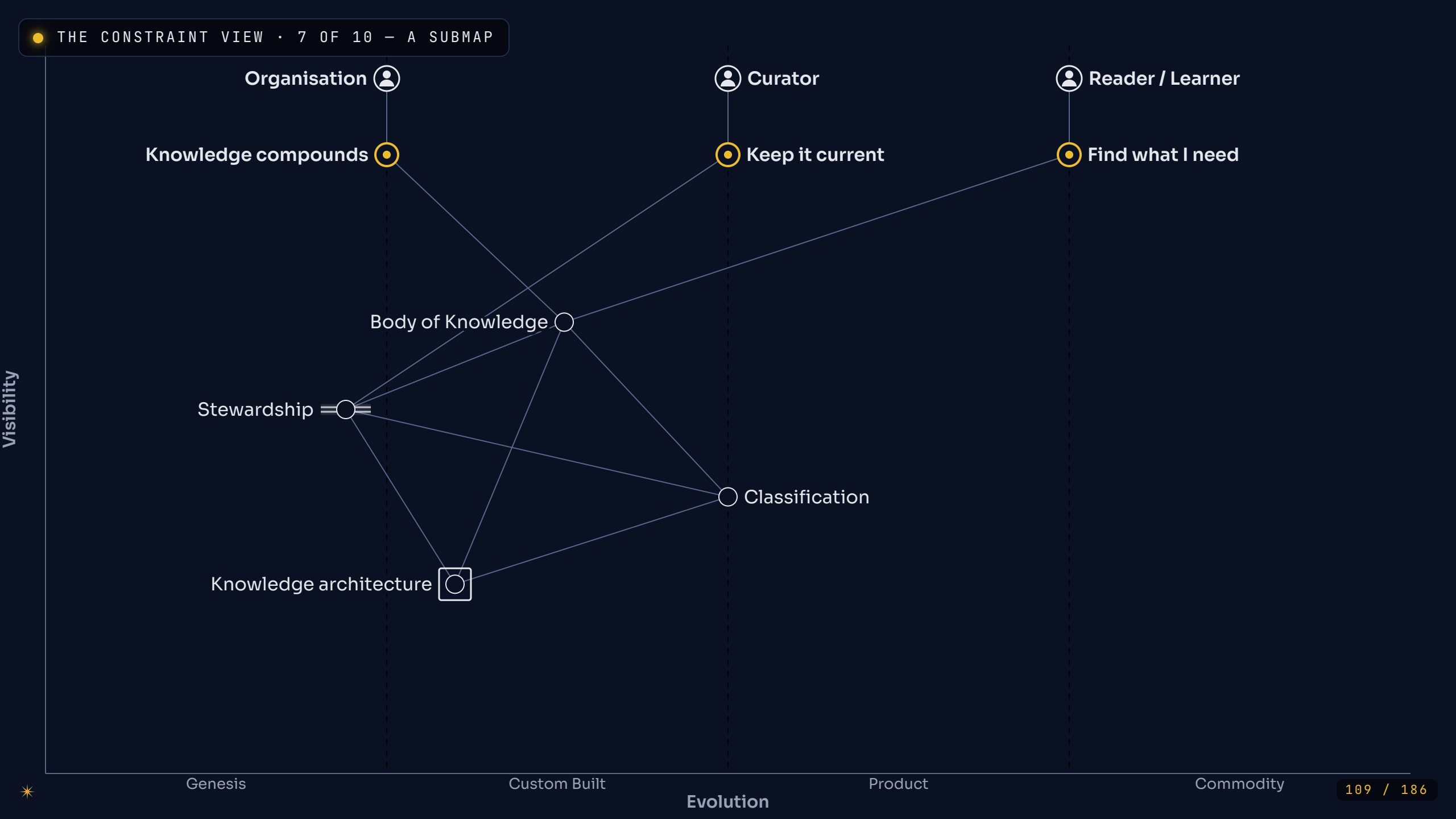
Organisation
Curator
Knowledge compounds
Keep it current
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

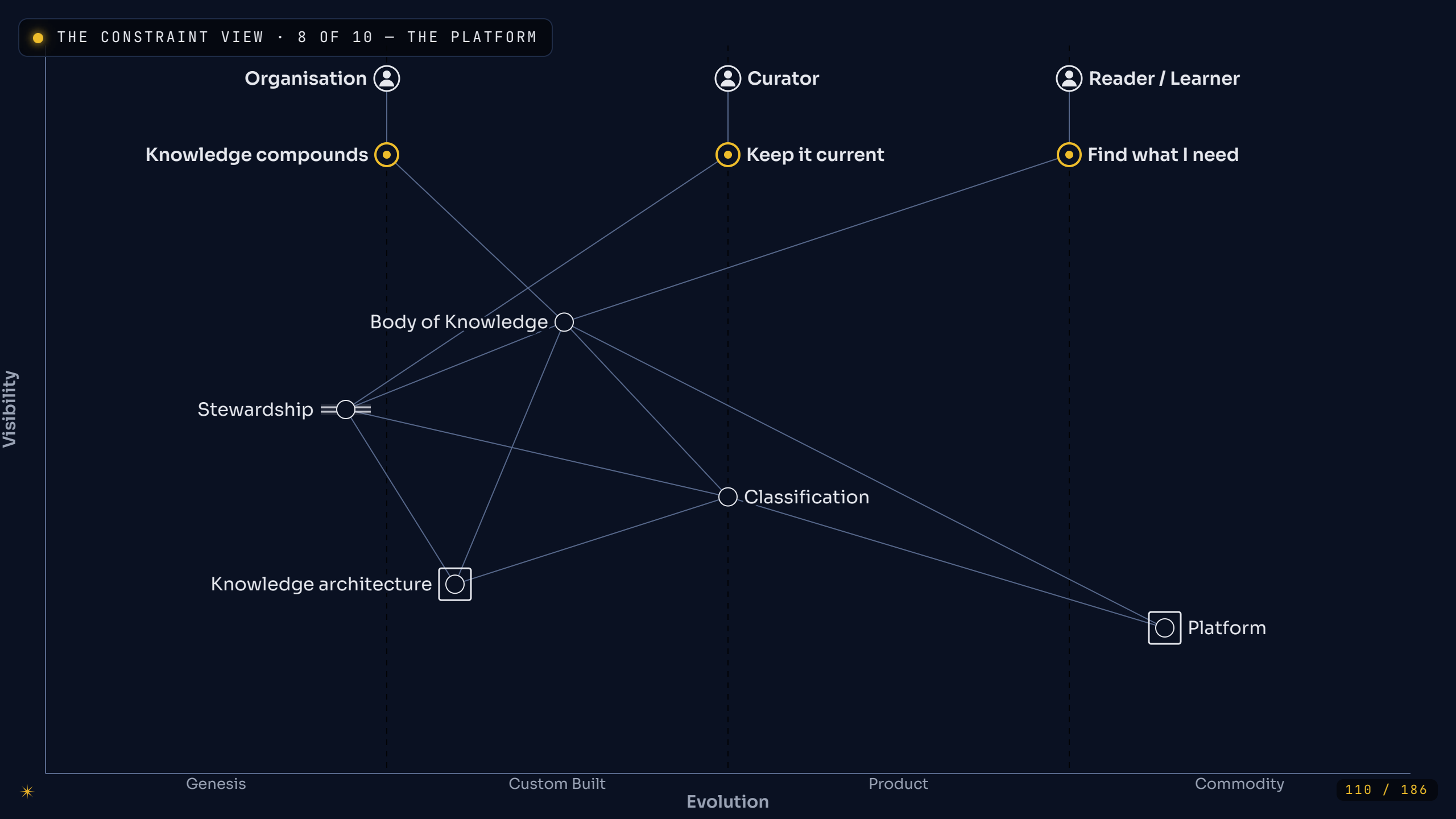

Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

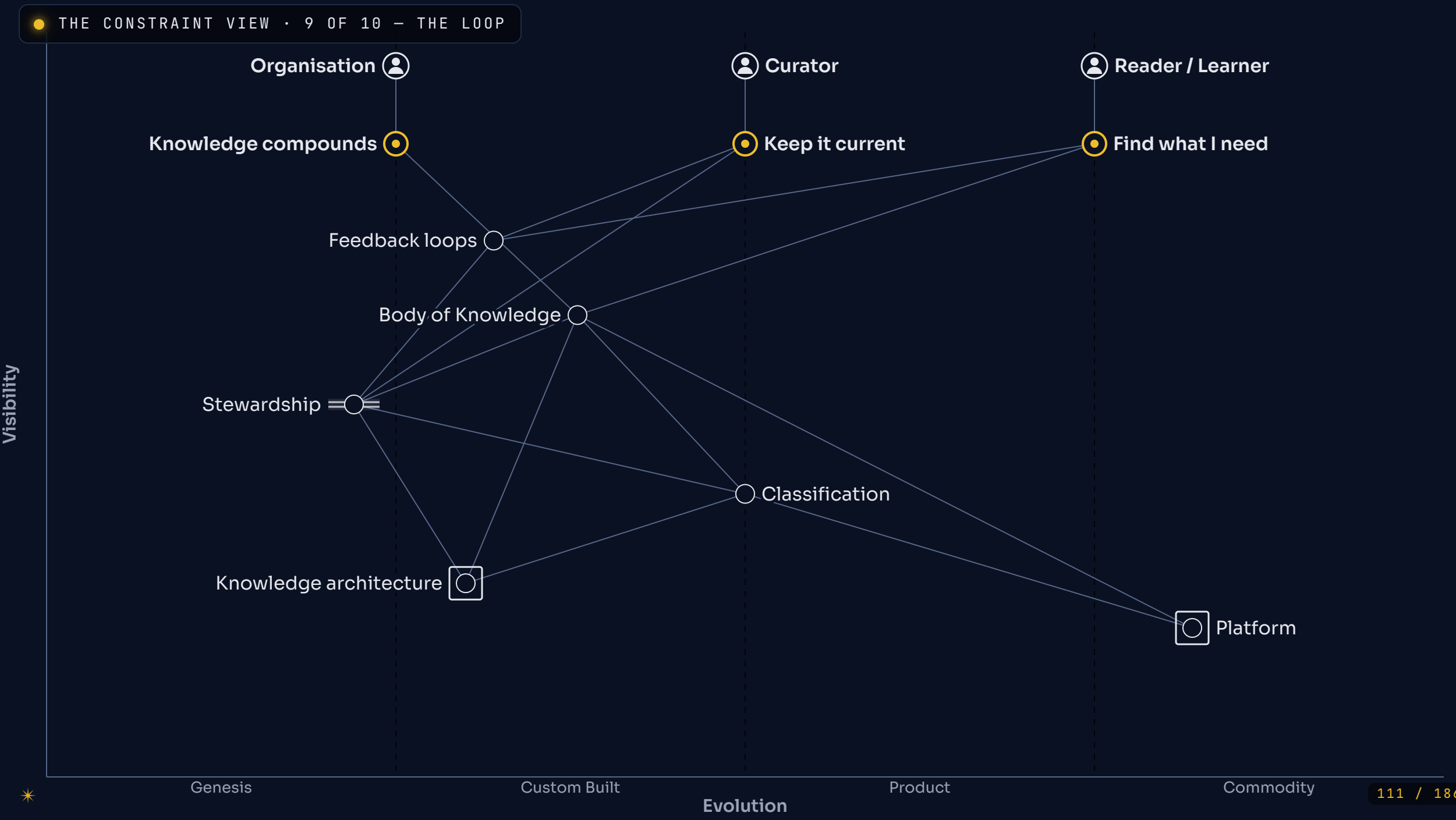

Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

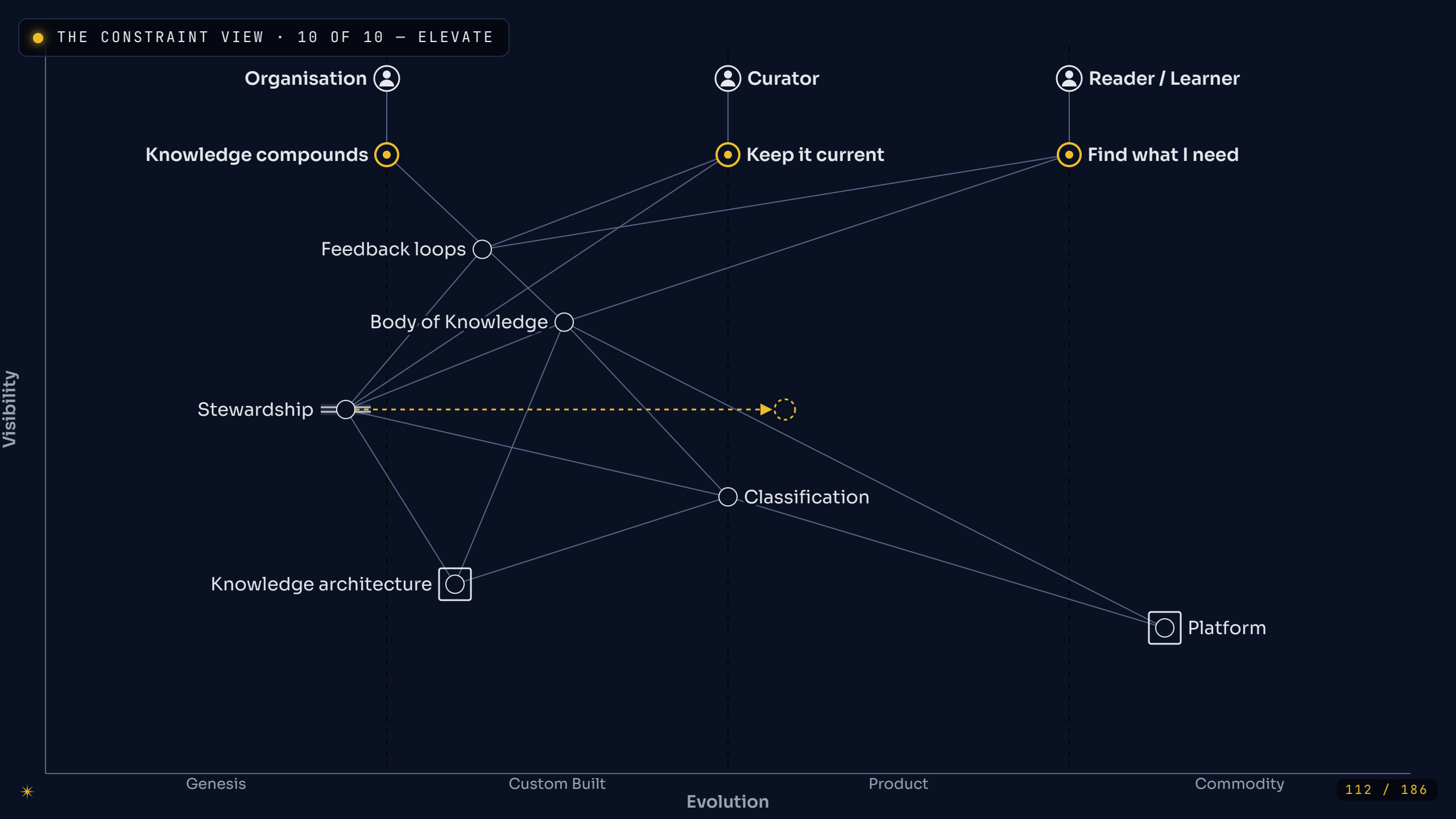

Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Stewardship
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Stewardship
Classification
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Stewardship
Classification
Knowledge architecture
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Body of Knowledge
Stewardship
Classification
Knowledge architecture
Platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Feedback loops
Body of Knowledge
Stewardship
Classification
Knowledge architecture
Platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Curator
Reader / Learner
Knowledge compounds
Keep it current
Find what I need
Feedback loops
Body of Knowledge
Stewardship
Classification
Knowledge architecture
Platform
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Stewardship holds the flow

- Every need runs through stewardship
- Exploit: curate what readers ask for; agents pre-tag
- Subordinate: pace capture to curation
- Elevate: shared stewardship, the evolve arrow

Searching where it is easy

"Why look here?" "Because the light is better." Storm P, 1926.

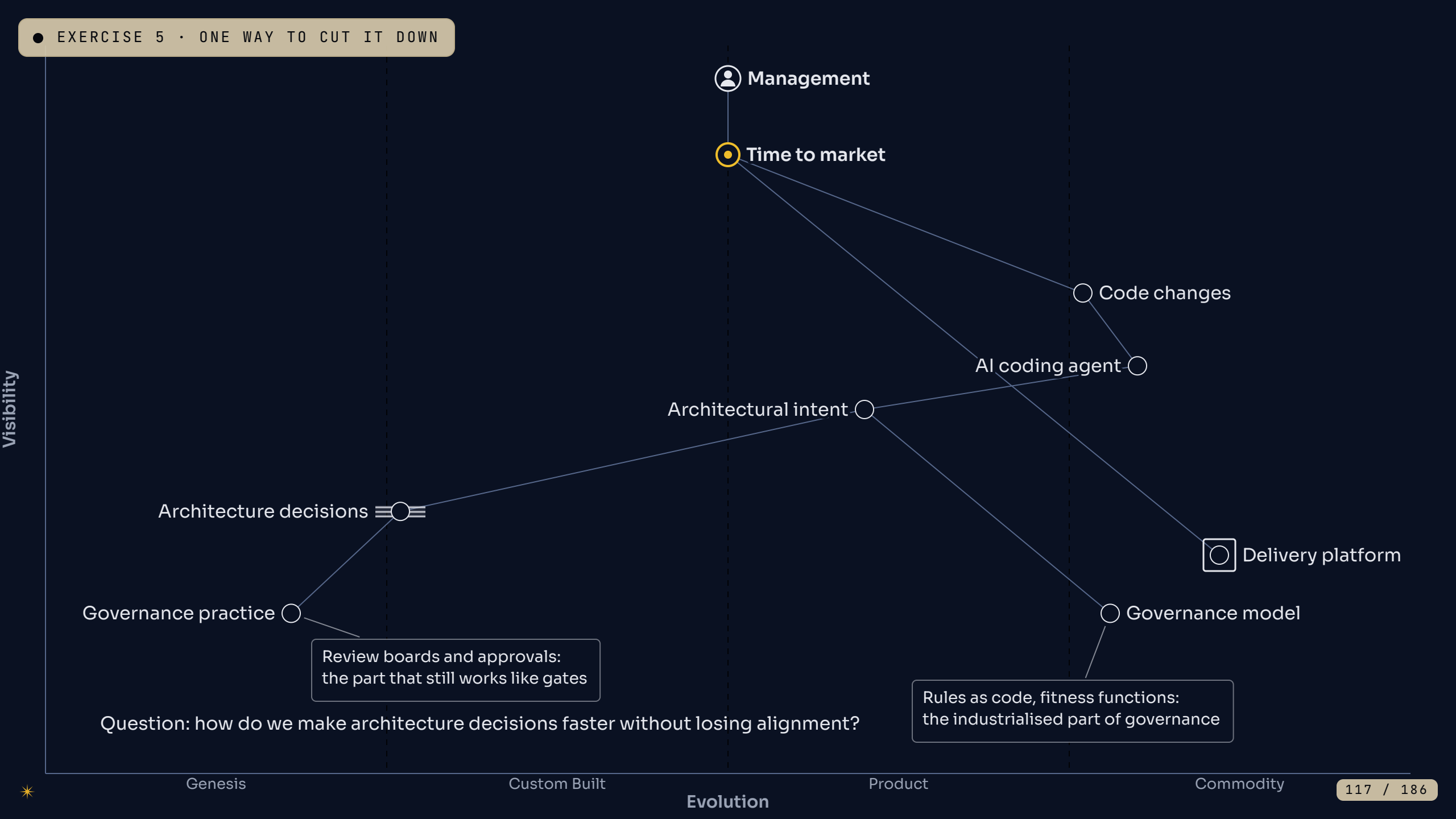
Exercise 5: simplify and move

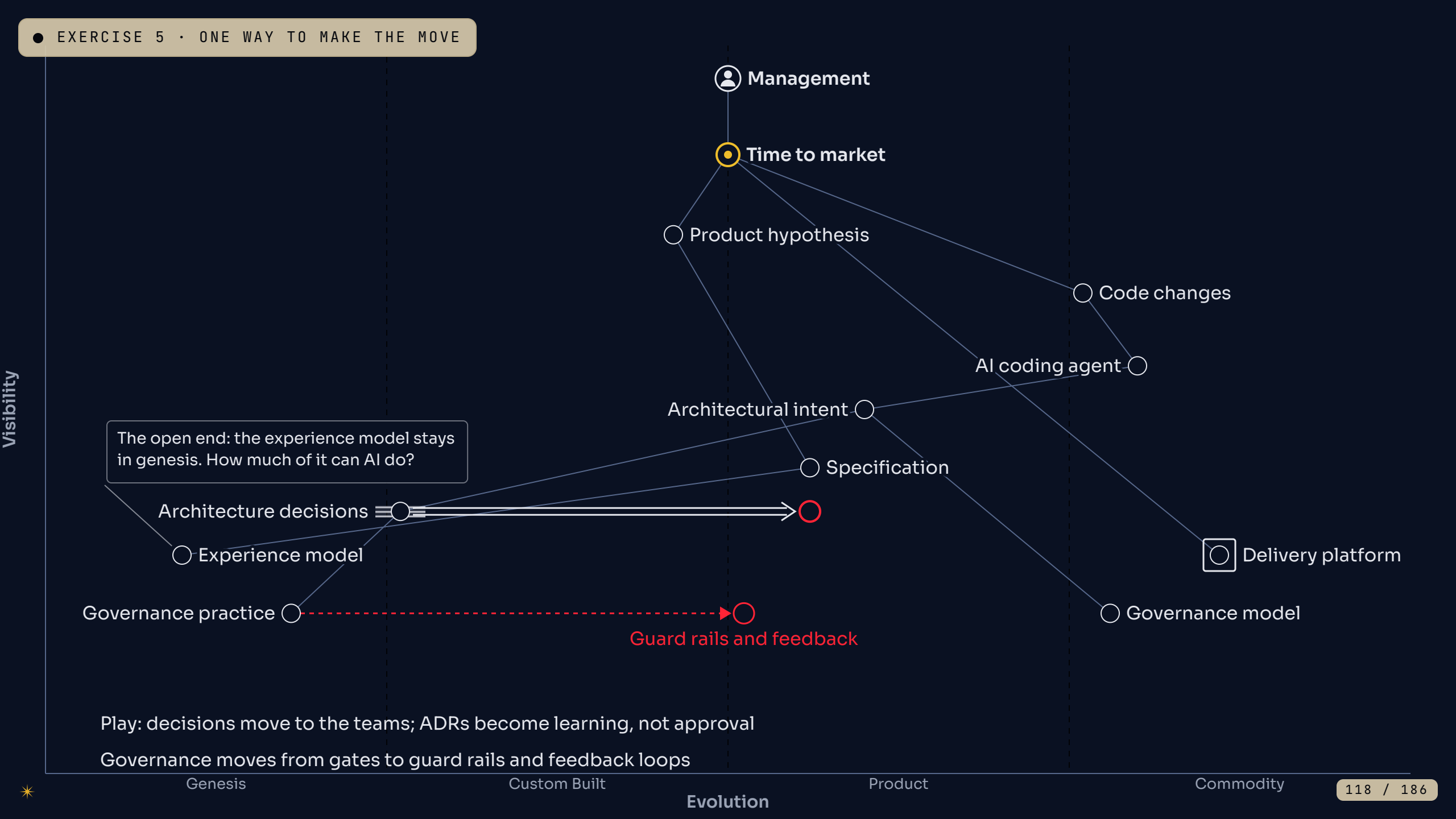
- **First: Cut it down**
  - Write the question your map answers
  - Remove what doesn't change the answer
  - Aim for 12 components or fewer
- **Then: Make a move**
  - Draw one strategic move on your simplified map
  - Natural evolution (dashed) or intended move (solid arrow)
  - Pick a gameplay pattern that speeds it up or holds it back

Exercise 5: discuss your simplification and move

- Is the story clearer after simplifying?
- Which component did you move, and where does it end up?
- Who gains from that move? Who loses?
- Would you dare to make that move this quarter?

EXERCISE 5 · ONE WAY TO CUT IT DOWN





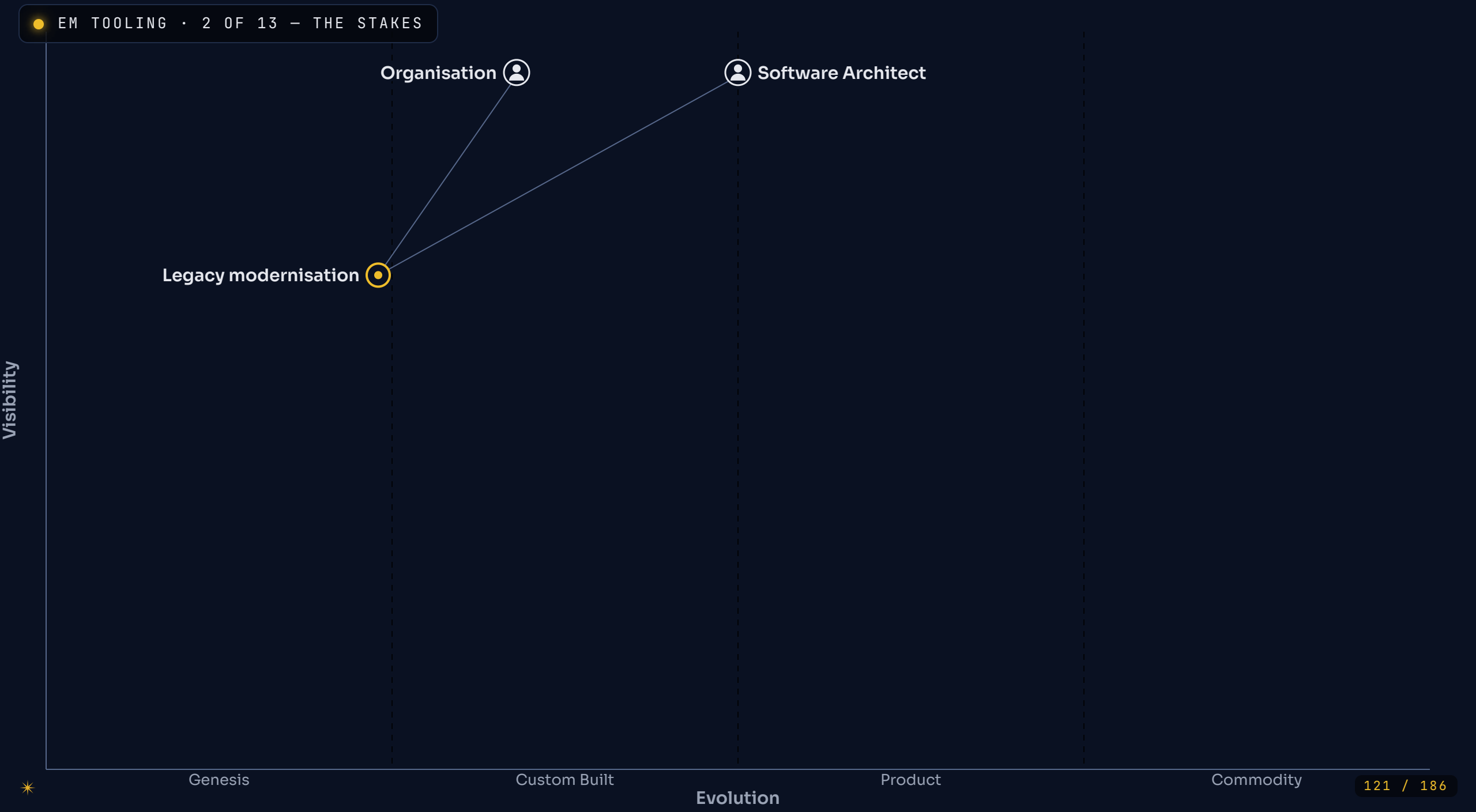
Play: decisions move to the teams; ADRs become learning, not approval

Governance moves from gates to guard rails and feedback loops

05

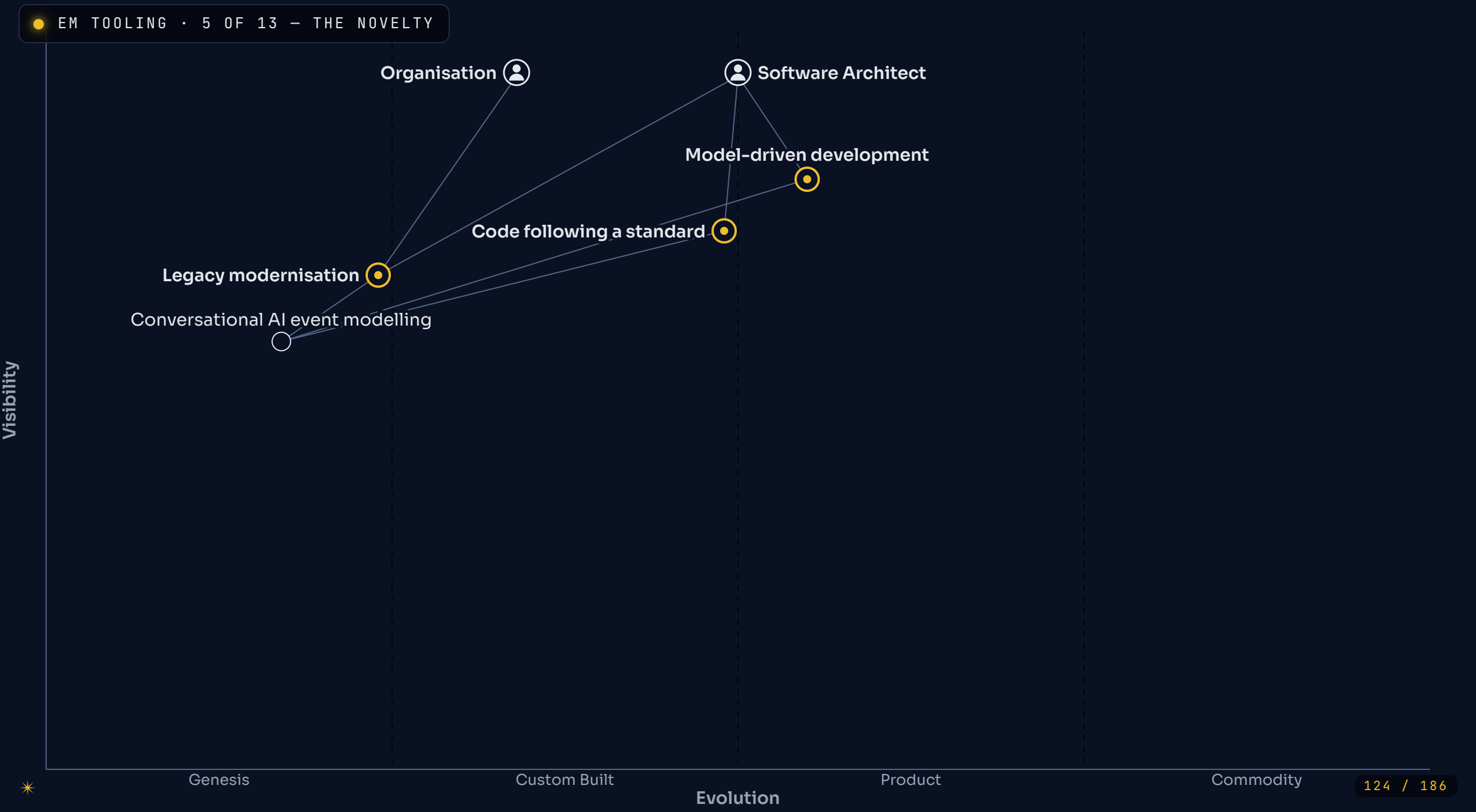
A real map

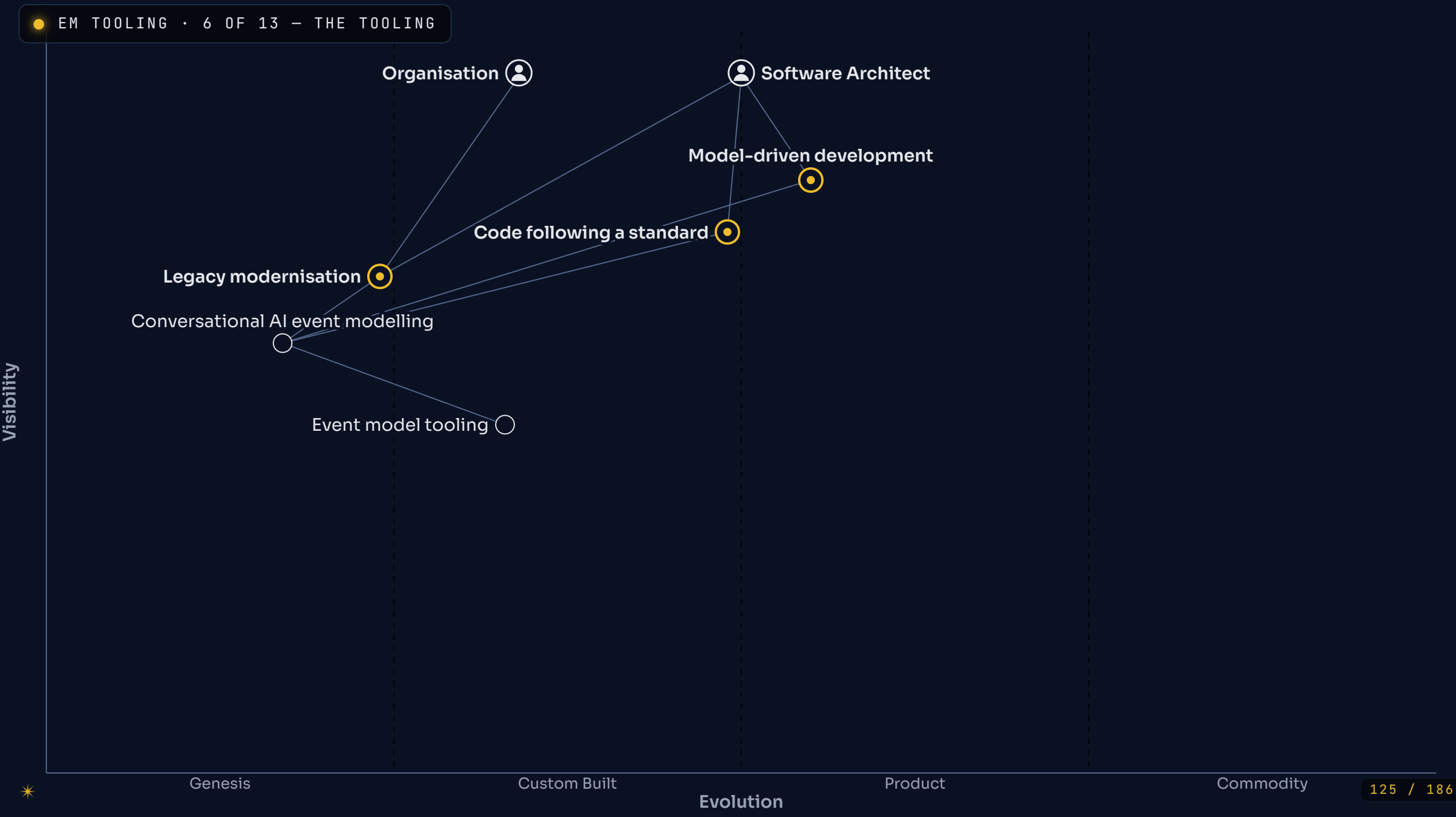
Organisation
Software Architect
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

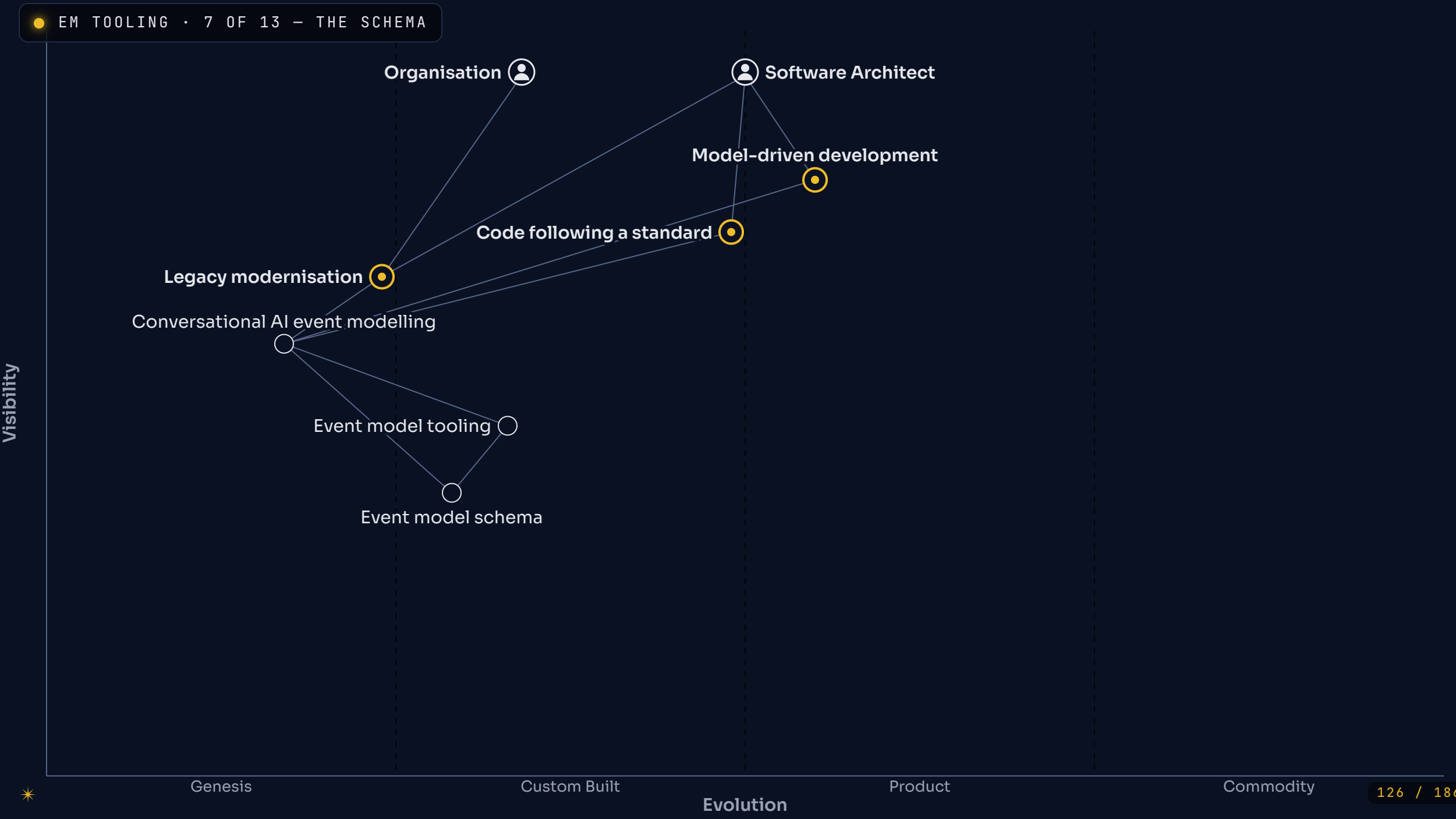

Organisation
Software Architect
Legacy modernisation
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

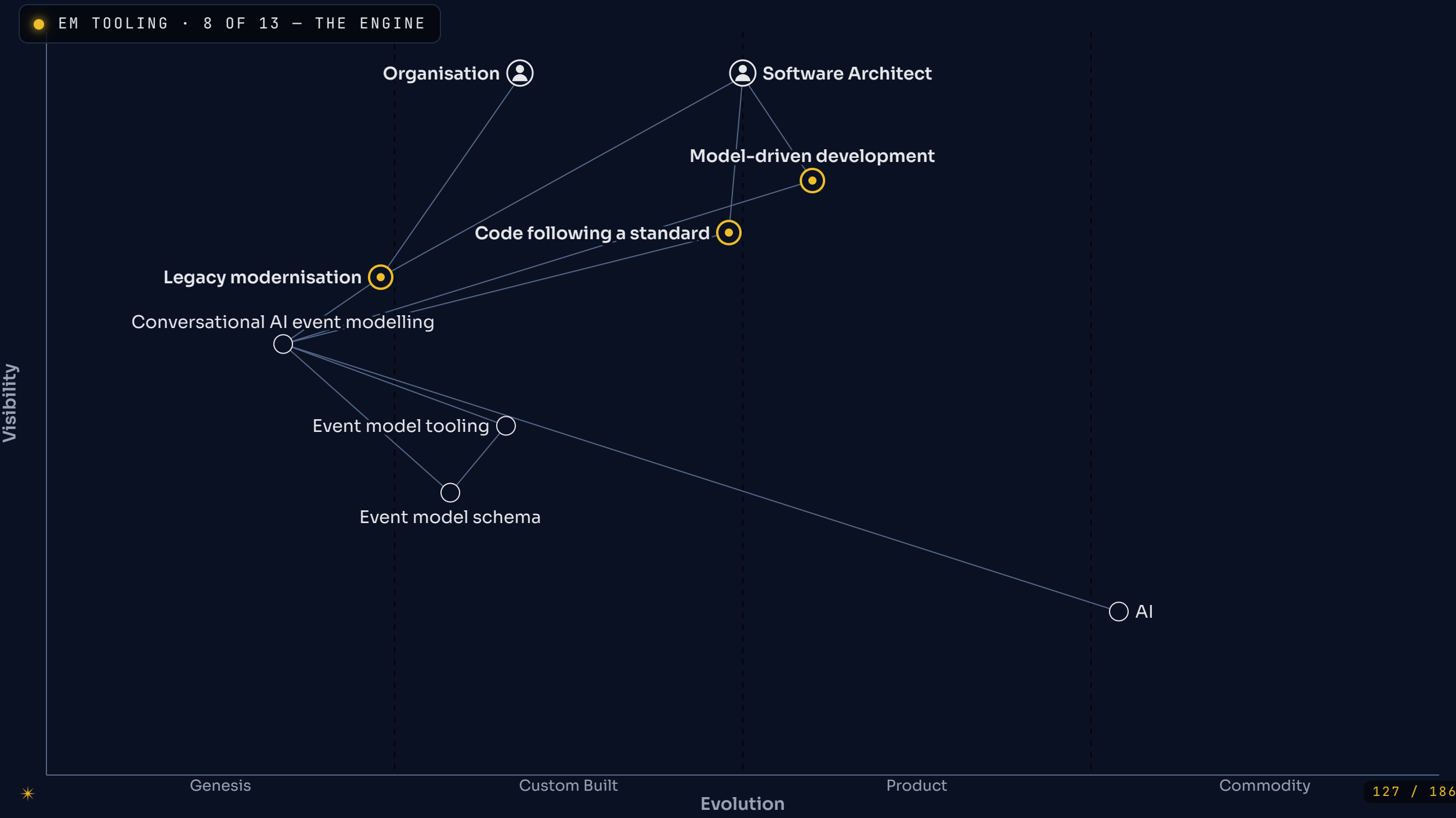
Organisation
Software Architect
Model-driven development
Legacy modernisation
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

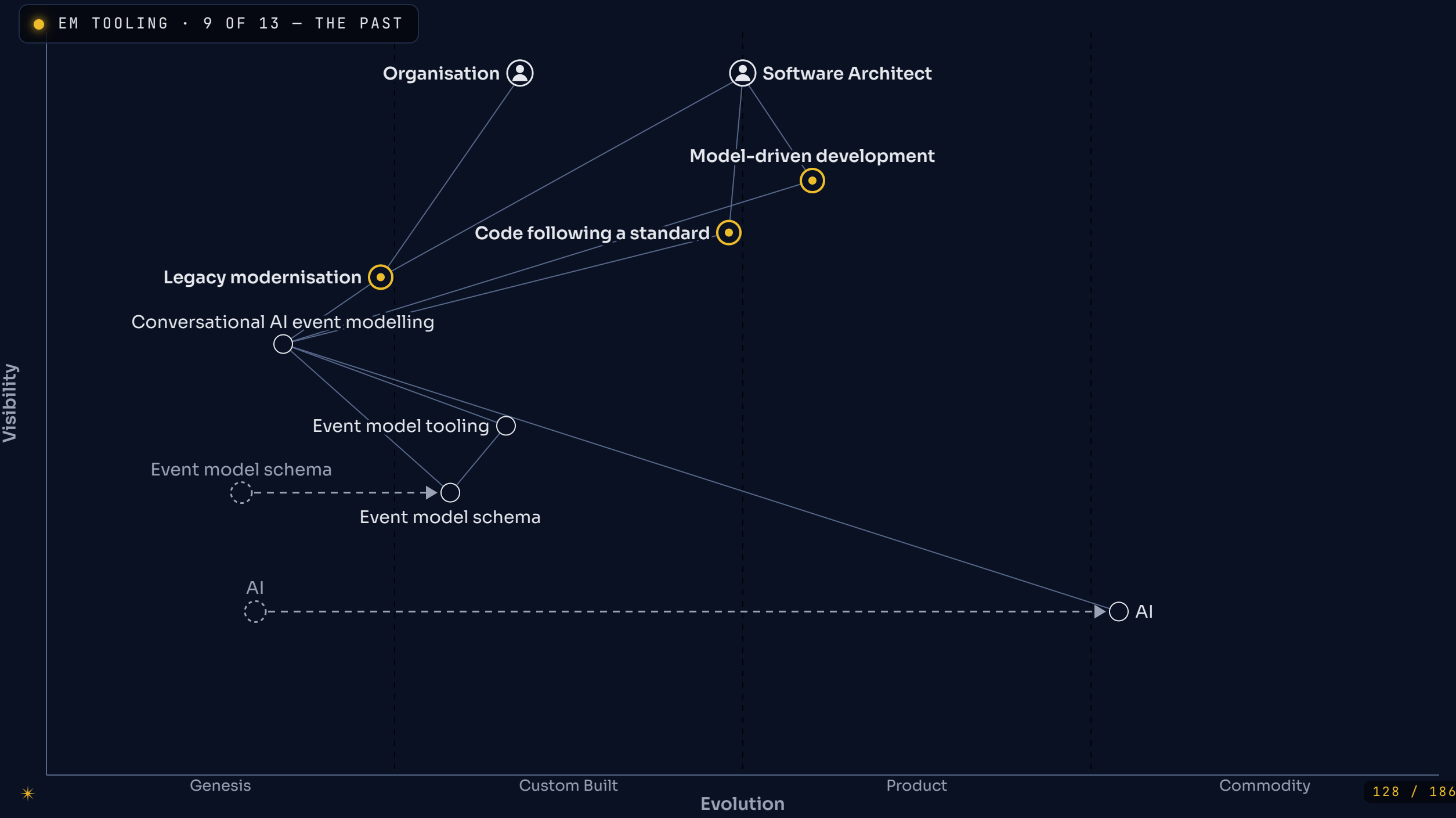
Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

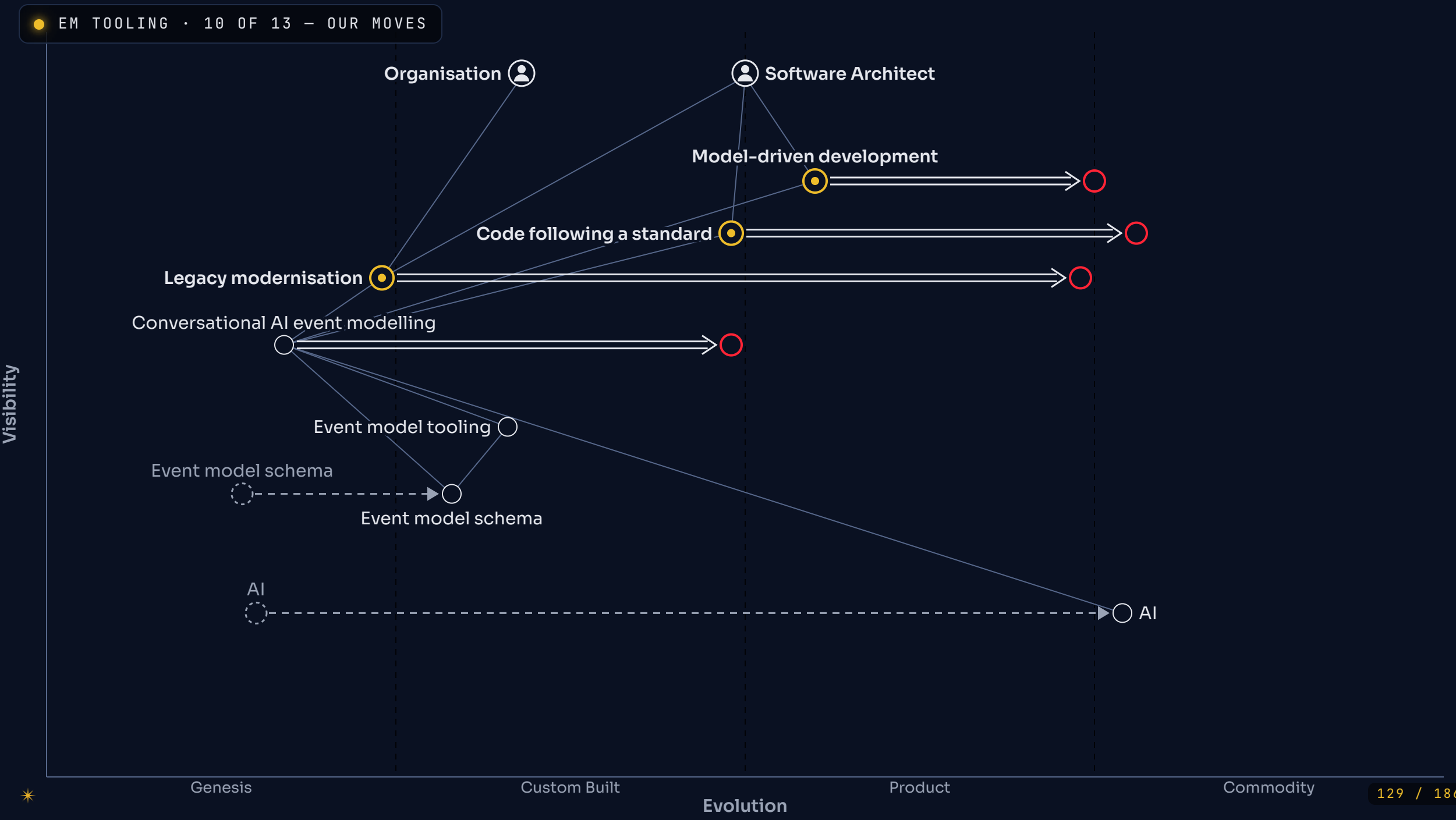

Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

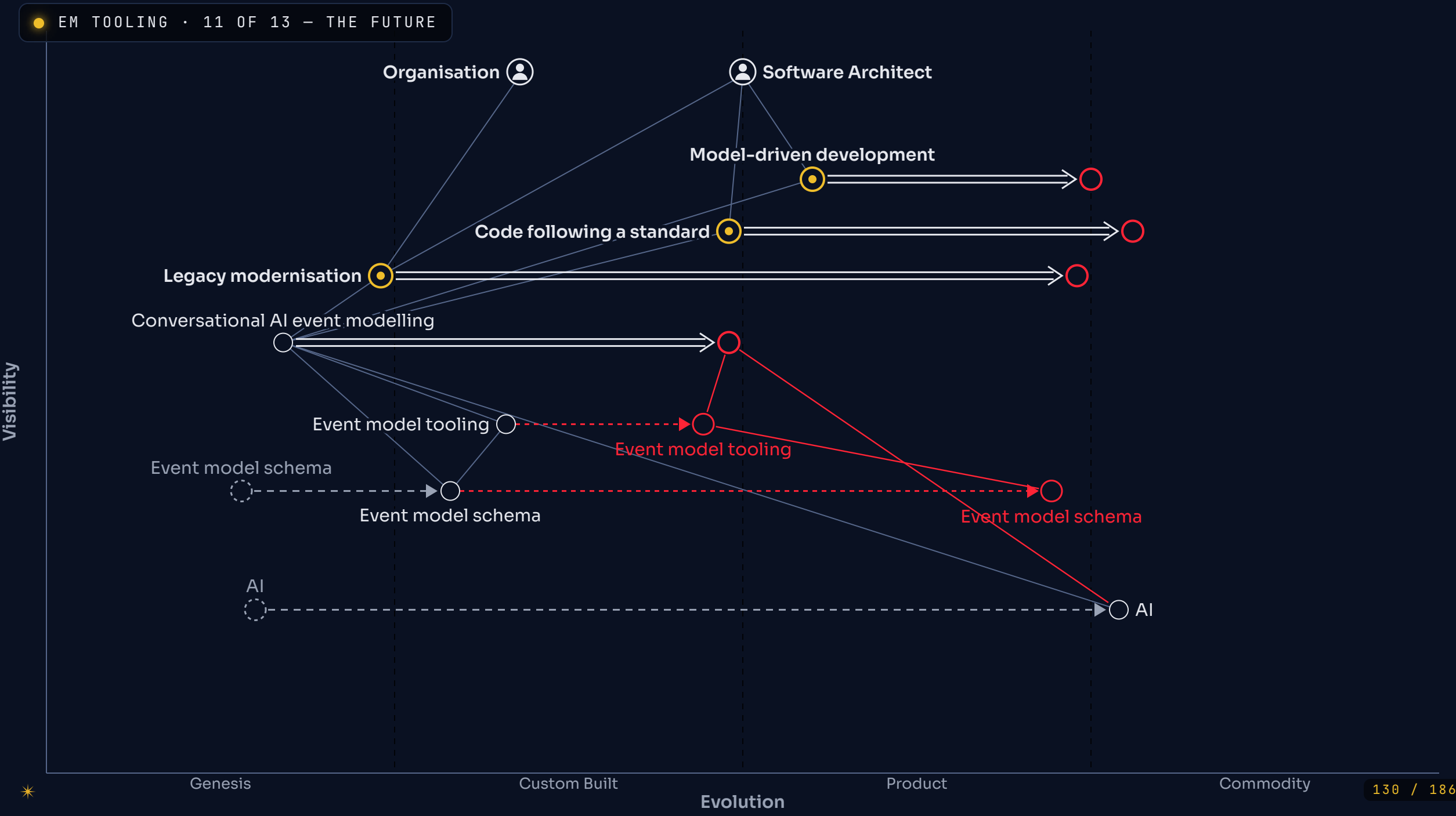

Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

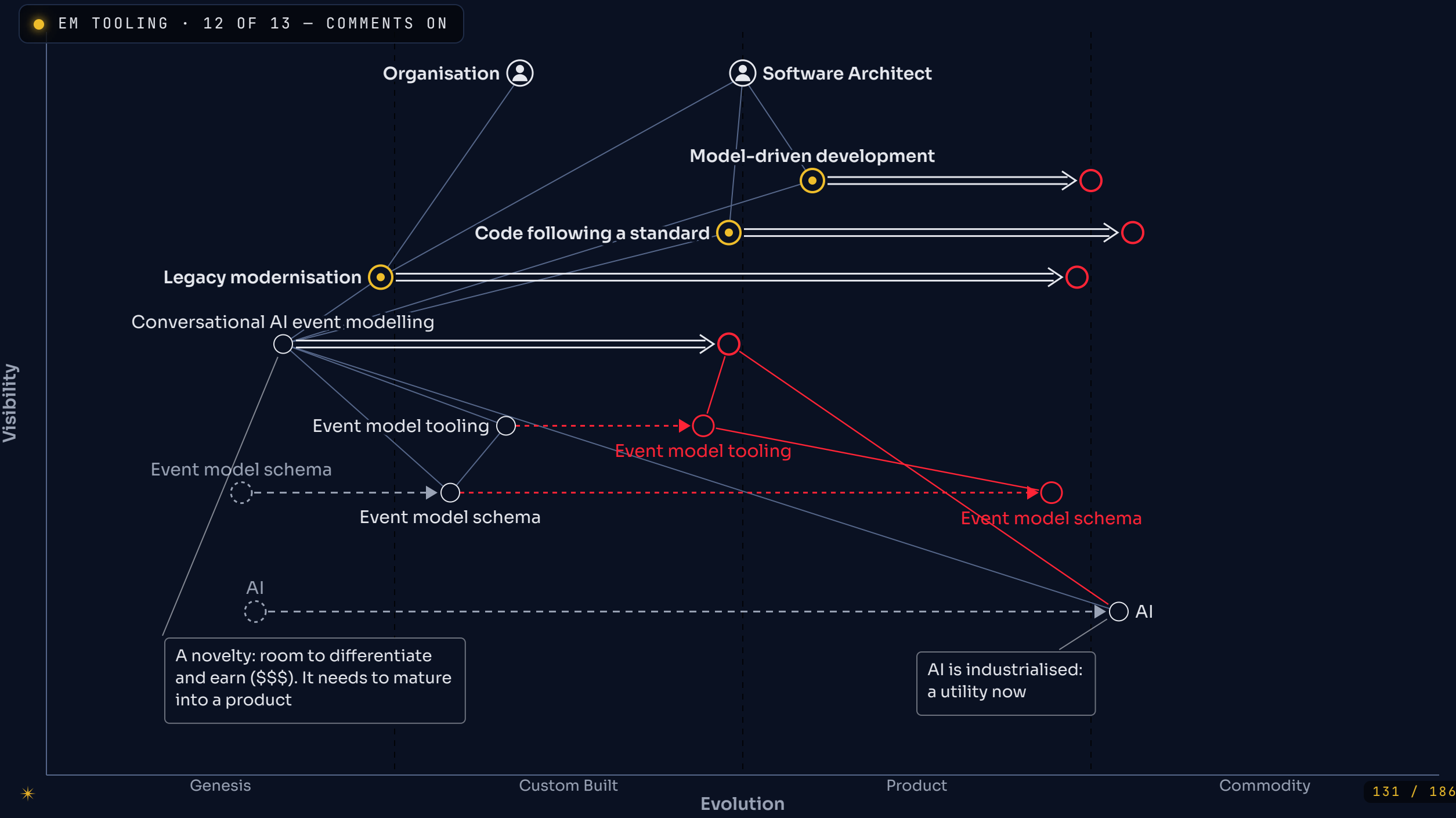

Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model schema
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

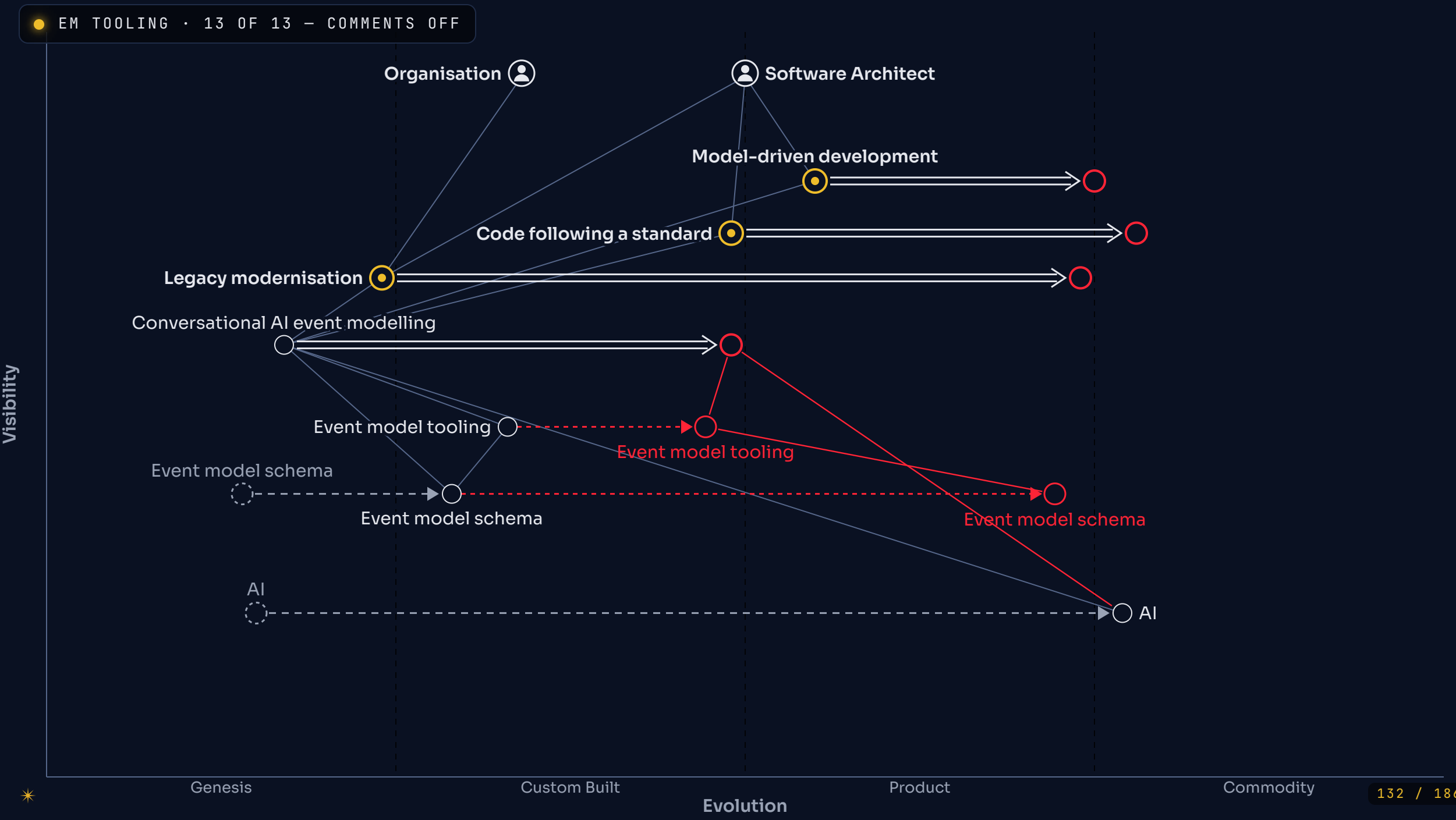

Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model schema
AI
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model schema
Event model schema
AI
AI
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model schema
Event model schema
AI
AI
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model tooling
Event model schema
Event model schema
Event model schema
AI
AI
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model tooling
Event model schema
Event model schema
Event model schema
AI
AI
A novelty: room to differentiate and earn ($$$). It needs to mature into a product
AI is industrialised: a utility now
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Organisation
Software Architect
Model-driven development
Code following a standard
Legacy modernisation
Conversational AI event modelling
Event model tooling
Event model tooling
Event model schema
Event model schema
Event model schema
AI
AI
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Where this can go

- Evolve the schema — it becomes the standard
- Mature the tooling past custom built
- Treat AI as a utility — it already is
- The worth is in the conversation

One day of research news, 17 October 2026

- Martin Rosén-Lidholm reads the agentic engineering papers every day and writes a digest
- About today's batch he wrote: "one of the strongest Software Civil Engineering signals we've seen. Multiple papers independently converge on the same thesis from different angles."
- The thesis: code generation is cheap, so the work moves to formal workflows, verified approvals and clear specifications
- That is the map you just drew. Next: his six pillars, then his five papers on one map

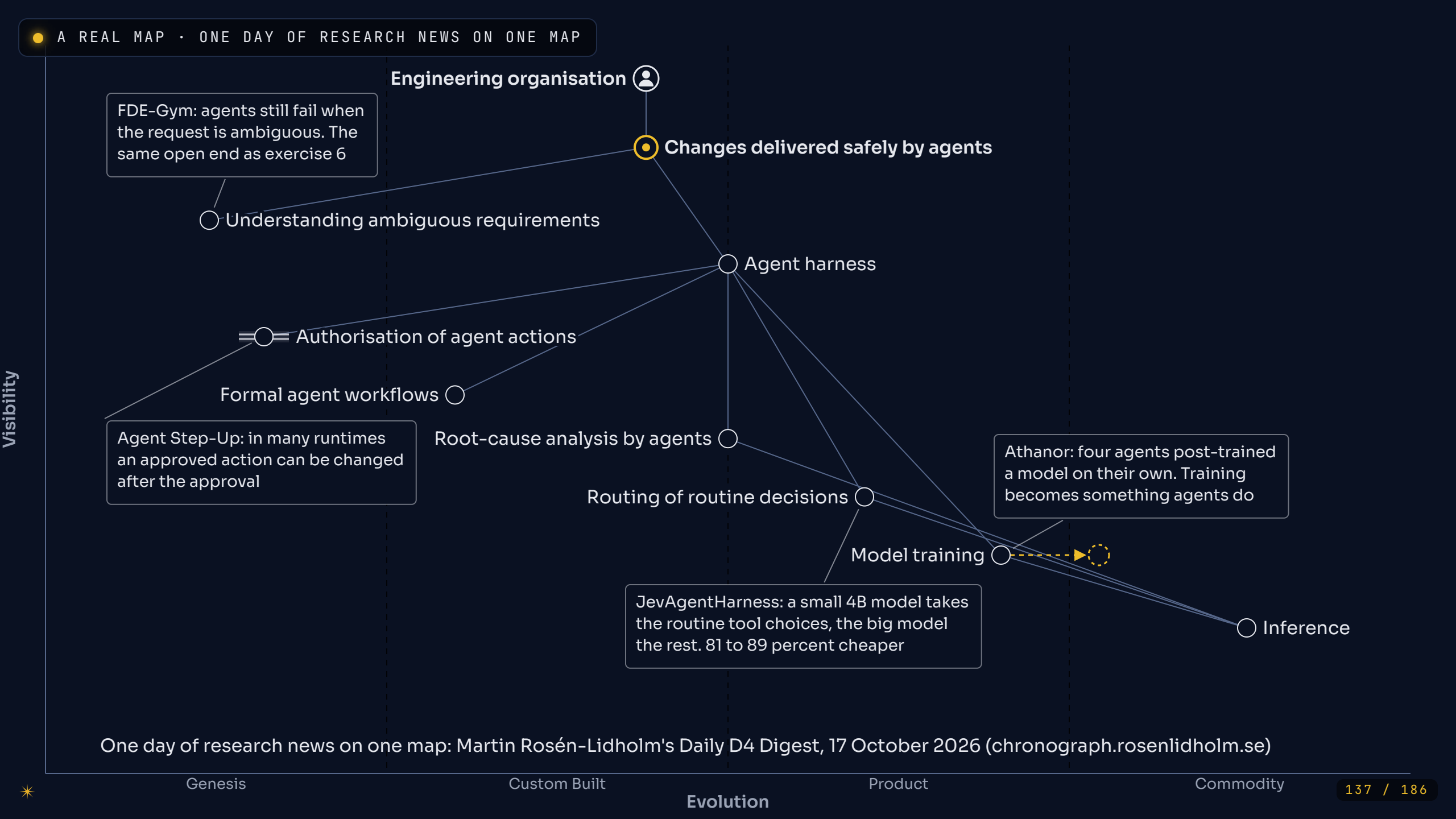
Six pillars of Software Civil Engineering

PILLAR, FROM CIVIL ENGINEERING	SOFTWARE TODAY	WHAT MARTIN PROPOSES
Formal specification and verification	user stories, ad hoc documents	the event model as the specification, checked by evaluations
Standardised material datasheets	no performance profiles for frameworks or infrastructure	provider profiles: known latency, throughput, limits
Building codes and norms	fragments: OWASP, well-architected frameworks	a policy model of rules that must always hold

Six pillars of Software Civil Engineering

PILLAR, FROM CIVIL ENGINEERING	SOFTWARE TODAY	WHAT MARTIN PROPOSES
Simulation and modelling	almost none for application logic	run the behaviour as pure functions first
Professional licensure and liability	no personal accountability	an open gap, nothing proposed yet
Formal education and examination	varies, no proof of competence	specification as an examined skill

Source: Martin Rosén-Lidholm, Software Civil Engineering: From Craft to Discipline, 22 March 2026, chronograph.rosenlidholm.se


A REAL MAP · ONE DAY OF RESEARCH NEWS ON ONE MAP
Engineering organisation
FDE-Gym: agents still fail when the request is ambiguous. The same open end as exercise 6
Changes delivered safely by agents
Understanding ambiguous requirements
Agent harness
Authorisation of agent actions
Formal agent workflows
Agent Step-Up: in many runtimes an approved action can be changed after the approval
Root-cause analysis by agents
Athanor: four agents post-trained a model on their own. Training becomes something agents do
Routing of routine decisions
Model training
JevAgentHarness: a small 4B model takes the routine tool choices, the big model the rest. 81 to 89 percent cheaper
Inference
Visibility
Genesis
Custom Built
Product
Commodity
Evolution
One day of research news on one map: Martin Rosén-Lidholm's Daily D4 Digest, 17 October 2026 (chronograph.rosenlidholm.se)

06

Wrap-up · the whole evening

BACK TOGETHER

What did the maps say?

- One insight per table, one sentence
- Where did the groups disagree?
- What moves right first?

07

Extra exercise, if there is time · your own domain

EXERCISE · 10 MIN

Draw one chain

- Pick a real user and one real need
- Add three things the need depends on
- Stack them: most visible at the top
- Place each on the evolution axis

EXERCISE · 10 MIN

Add the truth

- Mark what you would buy tomorrow
- Mark what hurts to change
- Mark one thing that should move right
- Find the constraint. Focus your map on it, and remove what is not strictly needed

EXERCISE · 15 MIN

Swap and stress-test

- Swap maps with another pair
- Ask: what did we leave off the chain?
- Ask: where would we place this differently?
- One decision per map: buy, build or evolve

08

Extra, if there is time · buy vs build

Kettles and whiteboards

- The kettle: a product. Buy it, never build
- The whiteboard: rent, don't make
- Build only where customers pay you to differ
- If you build a kettle, you are the map's problem

09

Extra, if there is time · inertia

What resists moving right

- Good facilitation is a craft. It is hard to turn into a product
- When something won't move, ask who would lose if it did
- Turning a craft into a product can kill it, or let it scale
- Help it evolve, or work around it

10

Extra, if there is time · the pipeline

Several ways to do one thing

- One need, several ways to meet it
- Each way sits at its own stage of evolution
- Each way evolves at its own pace
- We can speed a move up: open source, standards
- Or slow it down: patents, lock-in

Your team
legacy skills hold it back
Market evolution: the centre of gravity drifts right on its own
Compute
Hand-built
Managed hosting
IaaS
Serverless
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

11

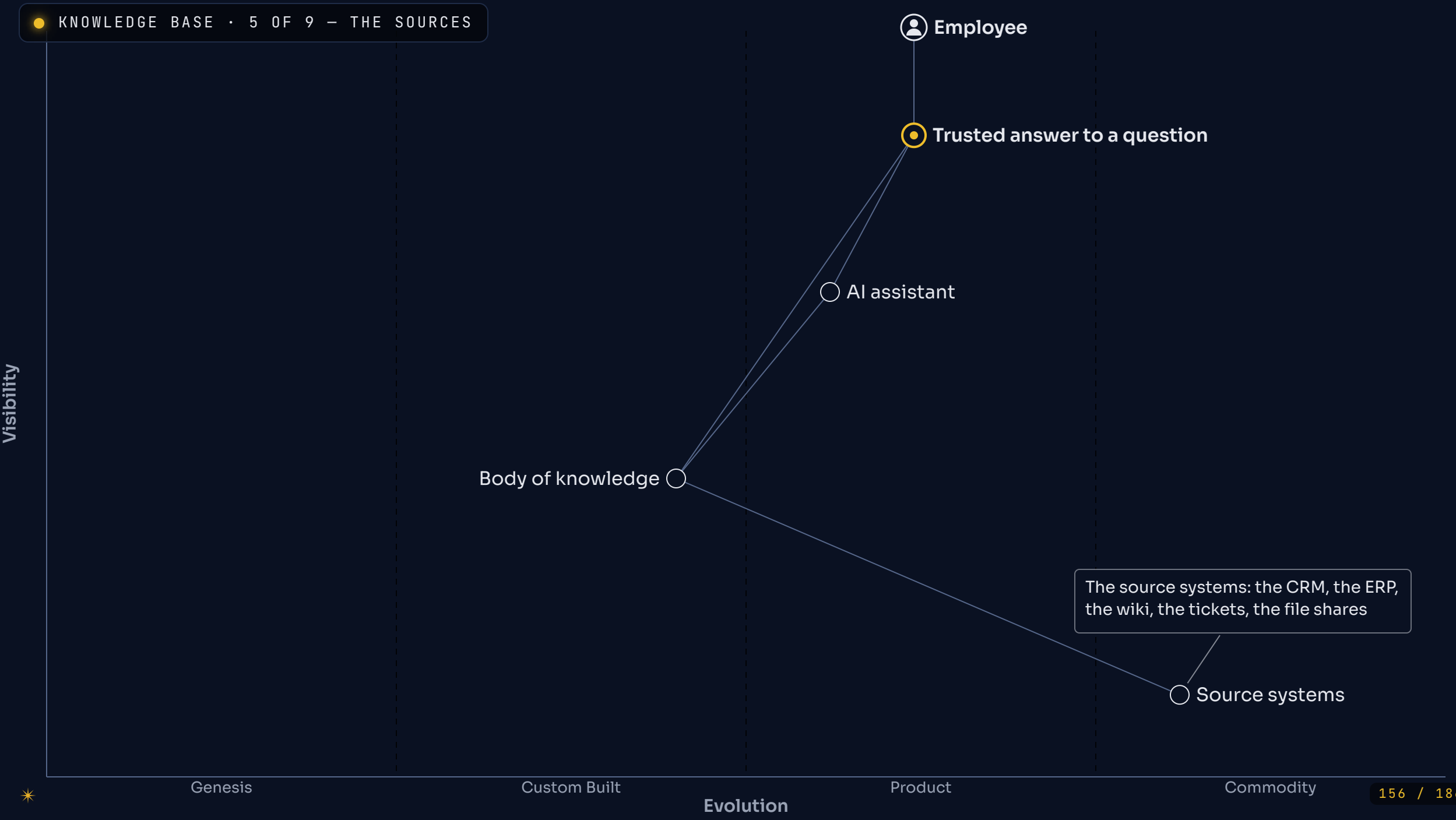
Extra, if there is time · knowledge base

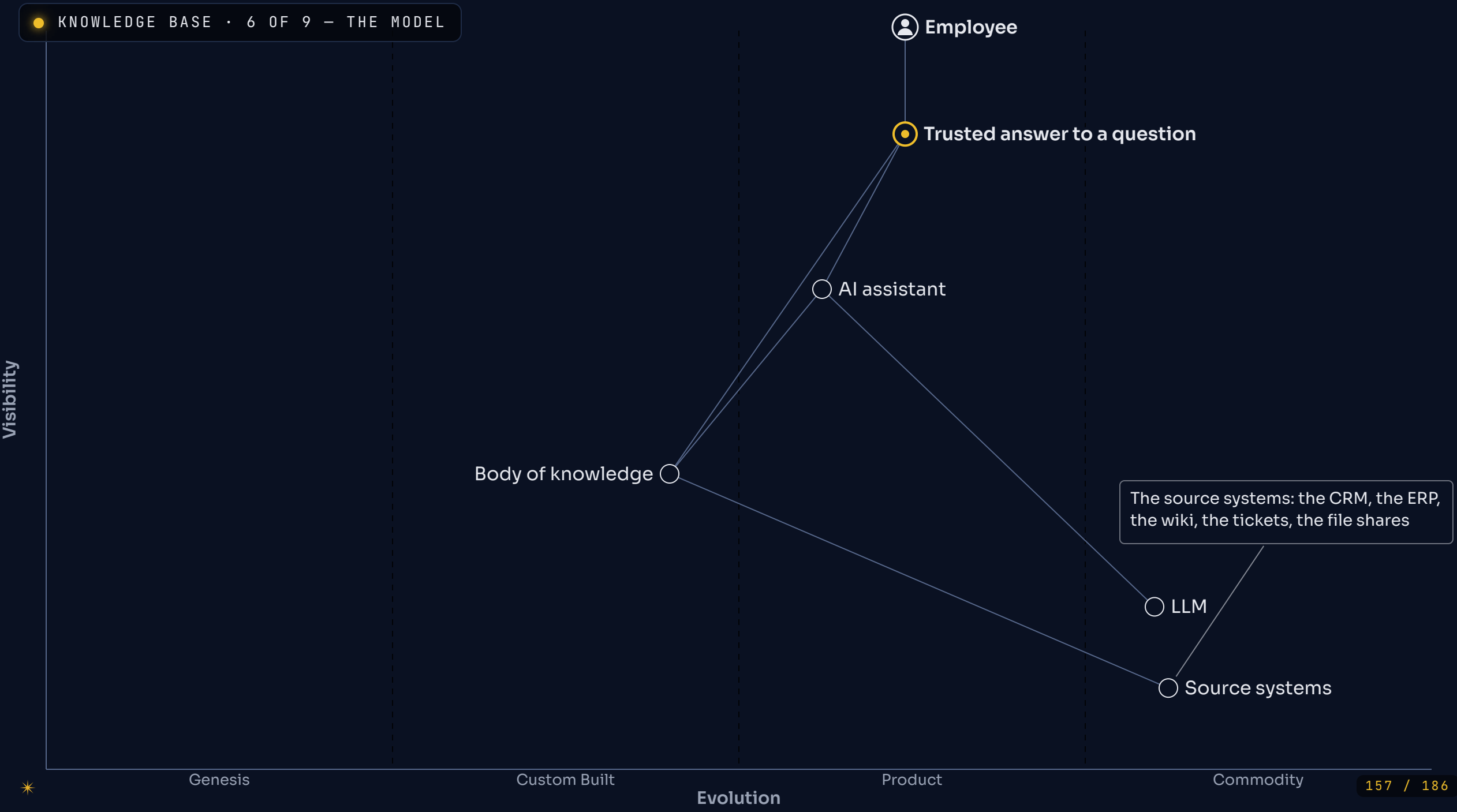
KNOWLEDGE BASE · 1 OF 9 — THE USER

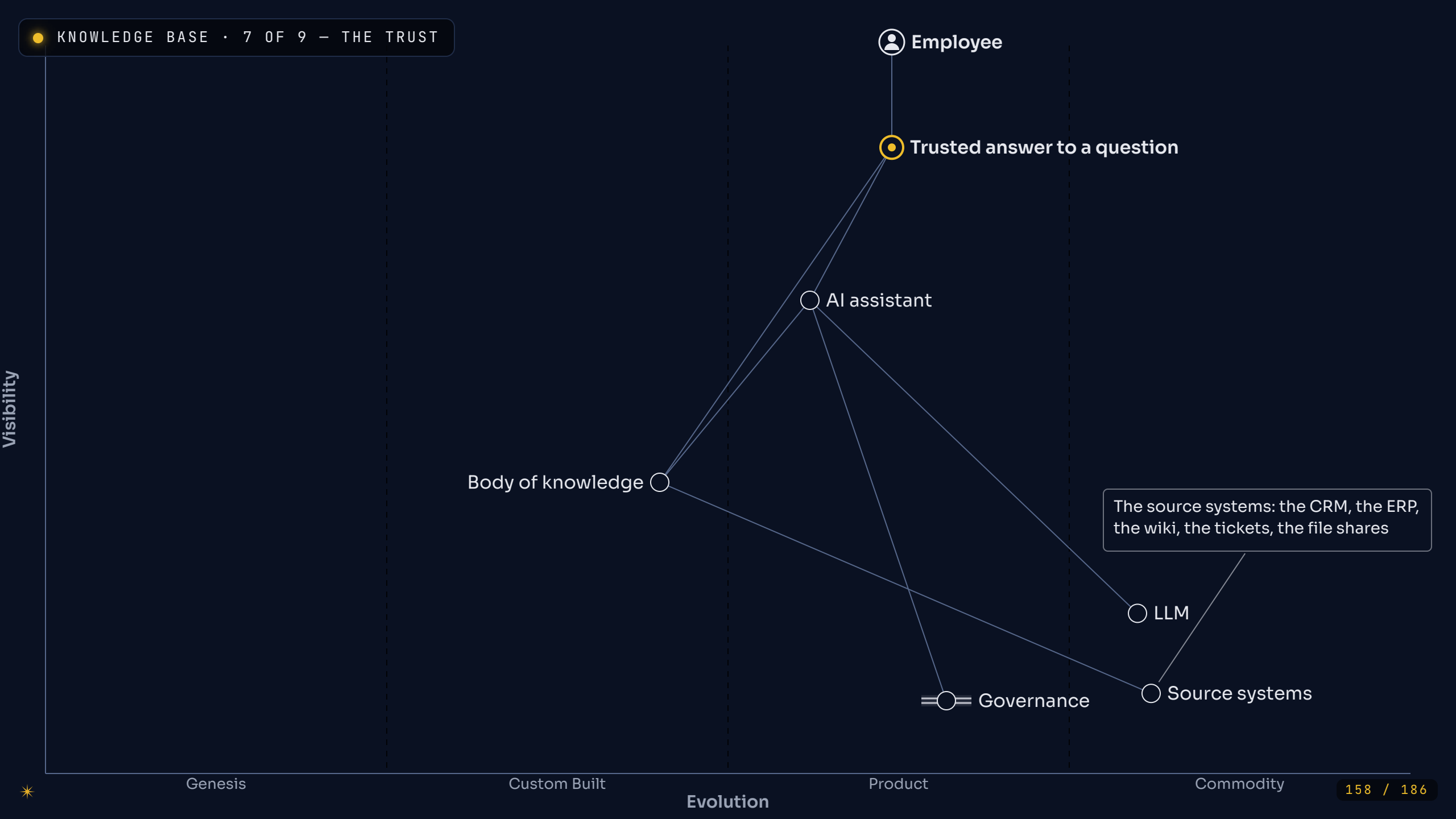
KNOWLEDGE BASE · 2 OF 9 — THE NEED

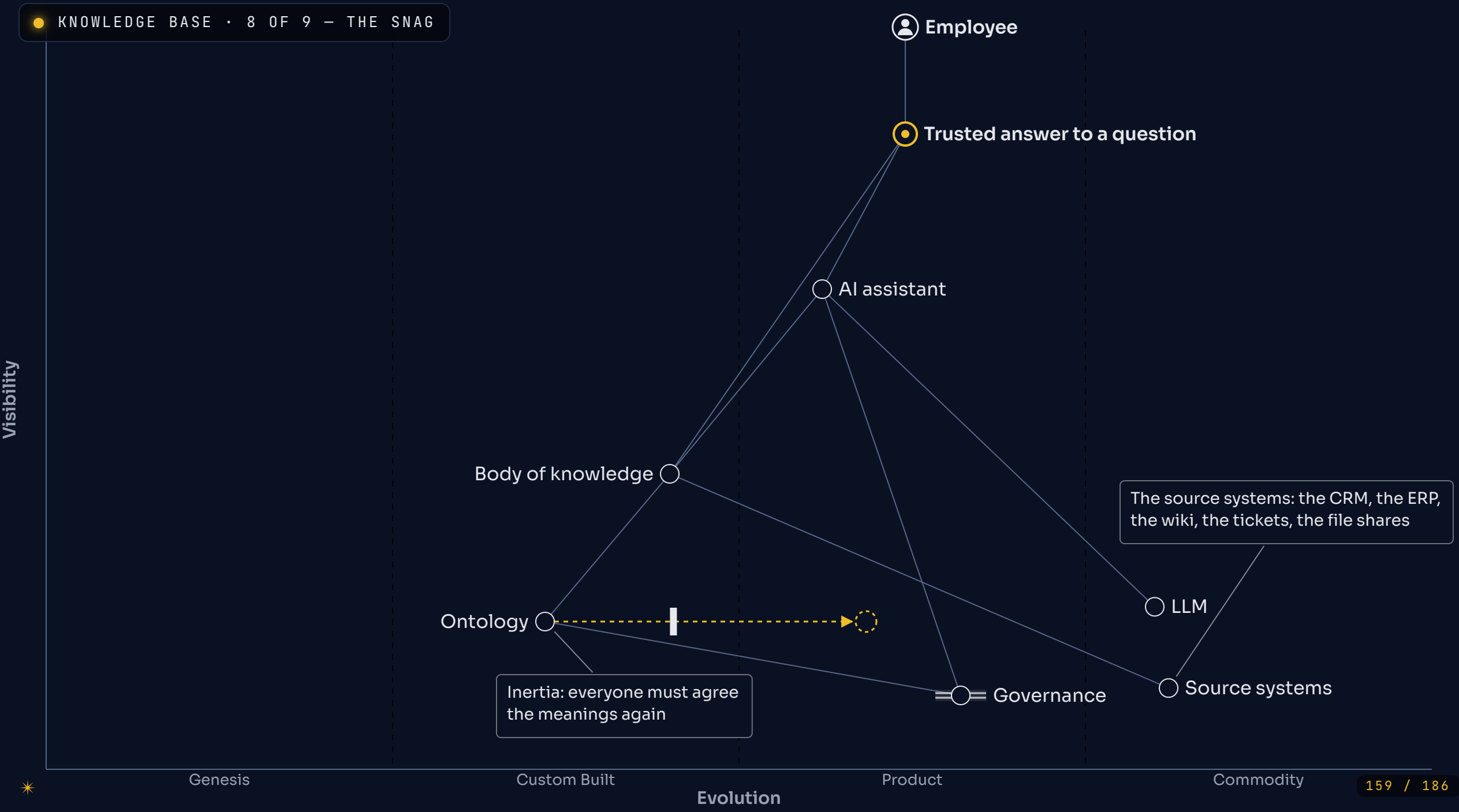
KNOWLEDGE BASE · 3 OF 9 — THE ASSISTANT

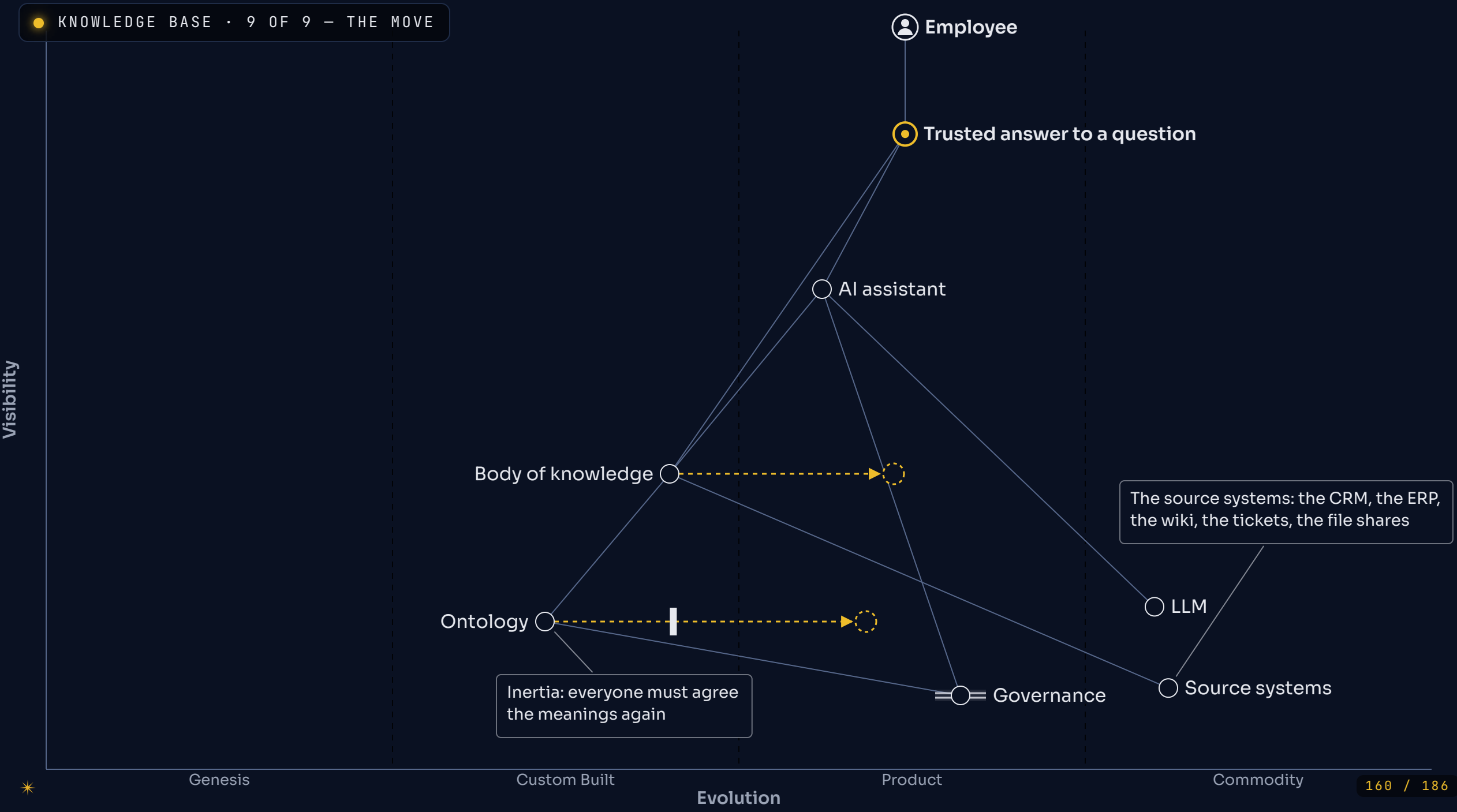
Employee
Trusted answer to a question
AI assistant
Body of knowledge
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Employee
Trusted answer to a question
AI assistant
Body of knowledge
The source systems: the CRM, the ERP, the wiki, the tickets, the file shares
Source systems
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Employee
Trusted answer to a question
AI assistant
Body of knowledge
The source systems: the CRM, the ERP, the wiki, the tickets, the file shares
LLM
Source systems
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Employee
Trusted answer to a question
AI assistant
Body of knowledge
The source systems: the CRM, the ERP, the wiki, the tickets, the file shares
LLM
Governance
Source systems
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Employee
Trusted answer to a question
AI assistant
Body of knowledge
The source systems: the CRM, the ERP, the wiki, the tickets, the file shares
LLM
Ontology
Inertia: everyone must agree the meanings again
Governance
Source systems
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Employee
Trusted answer to a question
AI assistant
Body of knowledge
The source systems: the CRM, the ERP, the wiki, the tickets, the file shares
LLM
Ontology
Inertia: everyone must agree the meanings again
Governance
Source systems
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Trust is the product

- Buy the assistant, the model, the plumbing
- Build the body of knowledge — where you differ
- The ontology resists: everyone must agree the meanings again
- The constraint sits at governance — map there

12

Extra, if there is time · submaps

Knowledge architecture

Visibility

Genesis

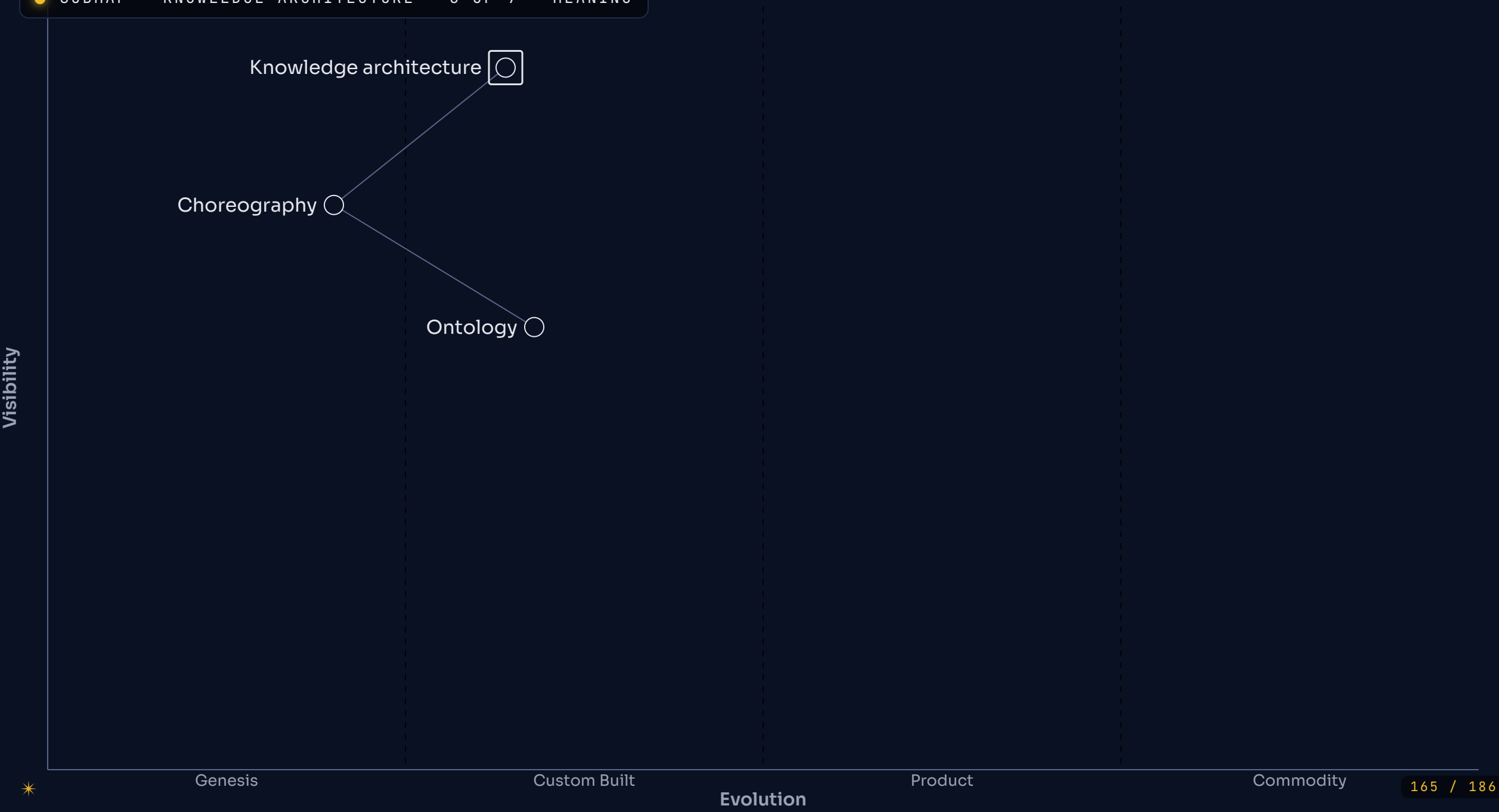
Custom Built

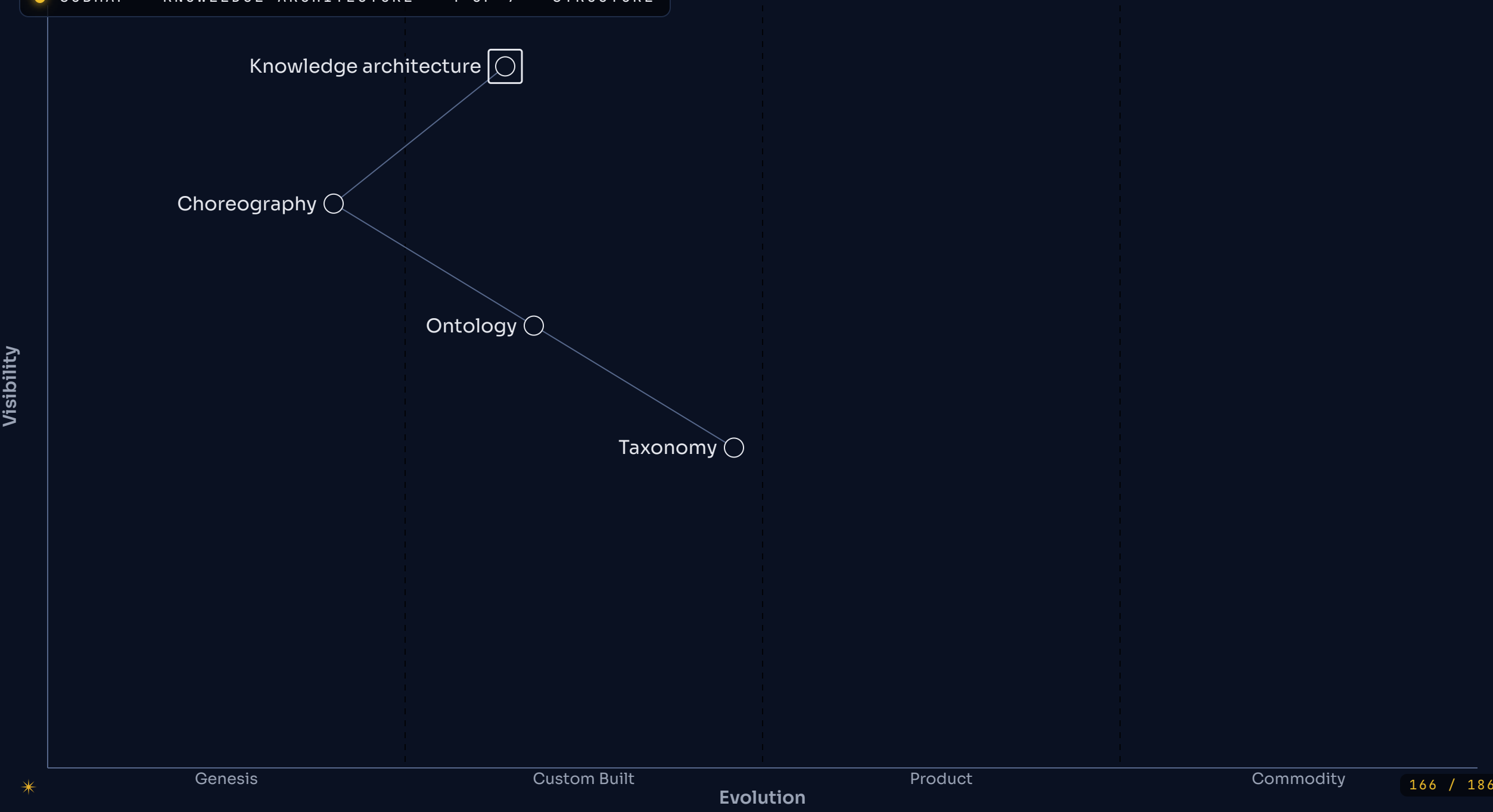
Product

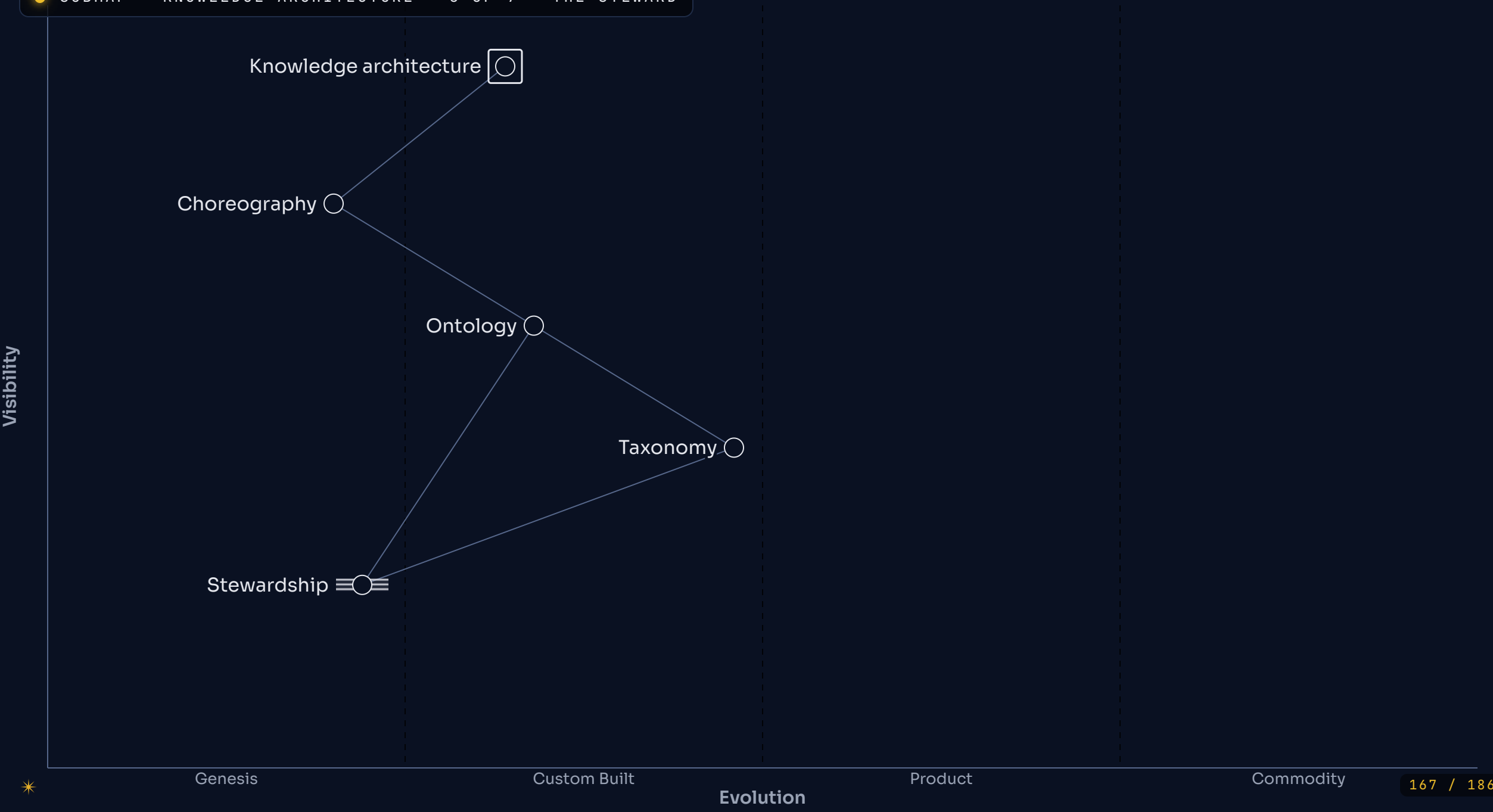
Commodity

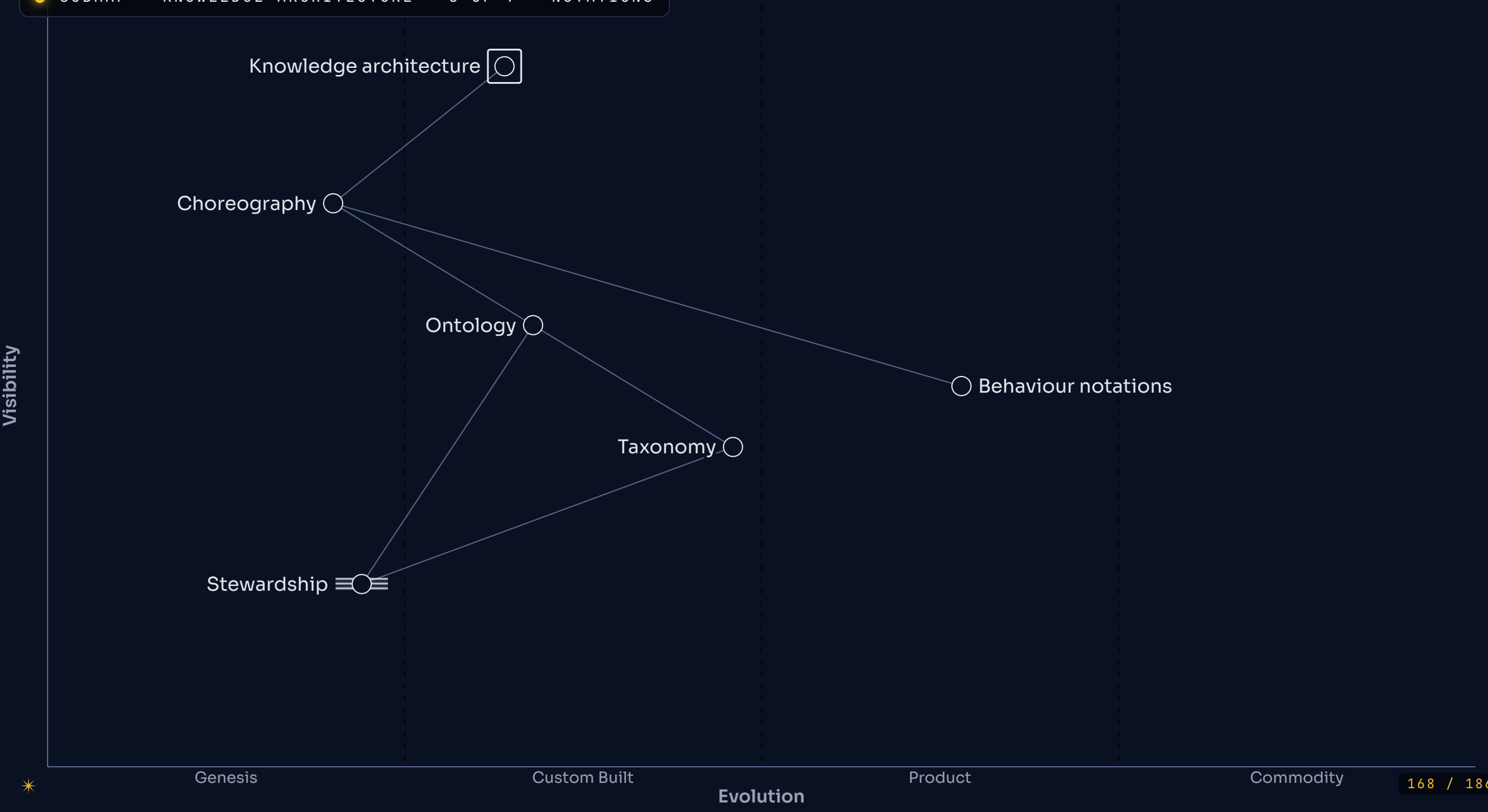
Evolution

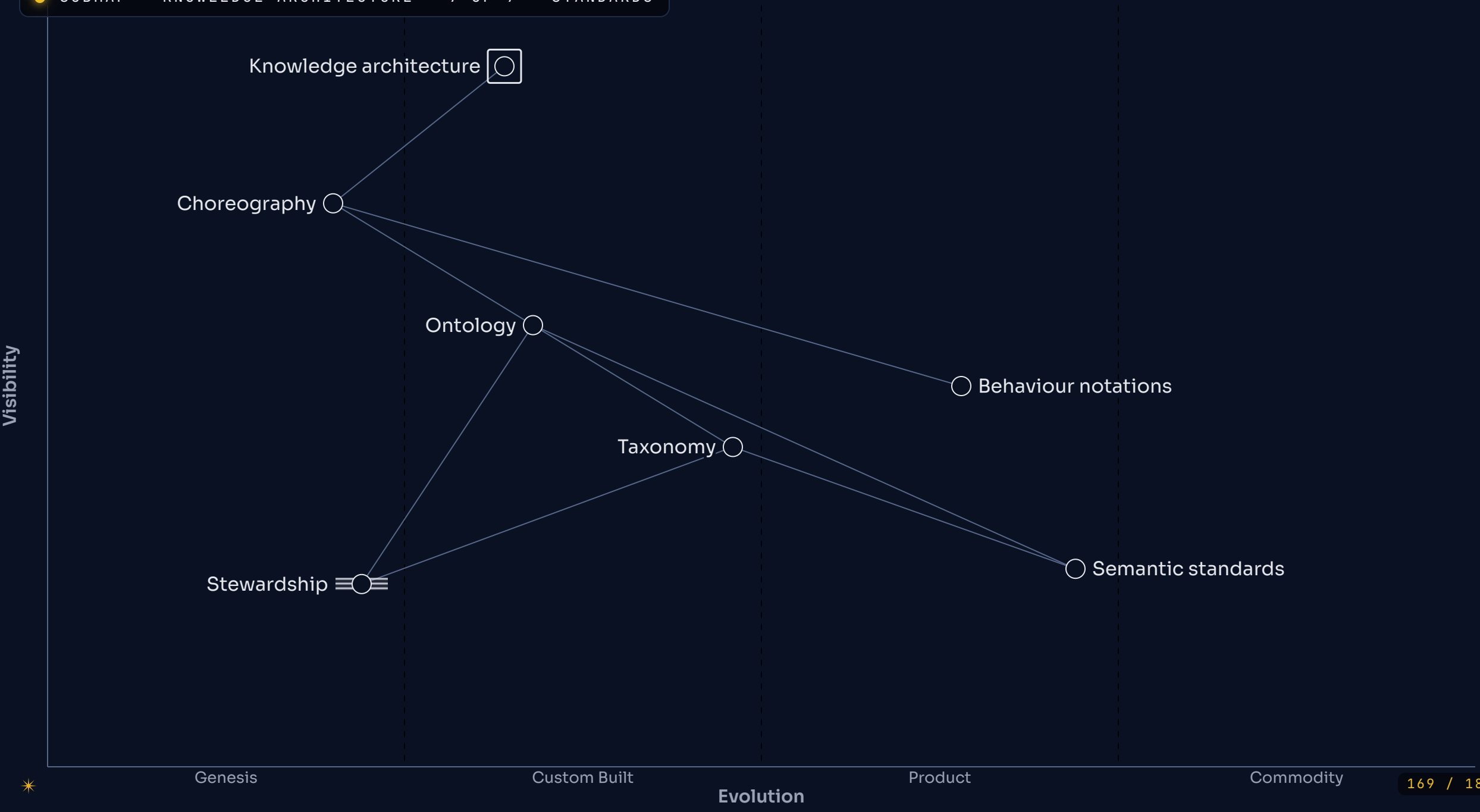
Knowledge architecture
Choreography
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Knowledge architecture
Choreography
Ontology
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Knowledge architecture
Choreography
Ontology
Taxonomy
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Knowledge architecture
Choreography
Ontology
Taxonomy
Stewardship
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Knowledge architecture
Choreography
Ontology
Behaviour notations
Taxonomy
Stewardship
Visibility
Genesis
Custom Built
Product
Commodity
Evolution


Knowledge architecture
Choreography
Ontology
Behaviour notations
Taxonomy
Semantic standards
Stewardship
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Structure, meaning, behaviour

- Taxonomy: what exists and how it is grouped
- Ontology: what things and their relations mean
- Behaviour: what happens over time
- Users feel behaviour, yet it is the least evolved
- We polish taxonomy because it is easy to see
- Stewardship looks after all three layers

13

Appendix · cheat sheet

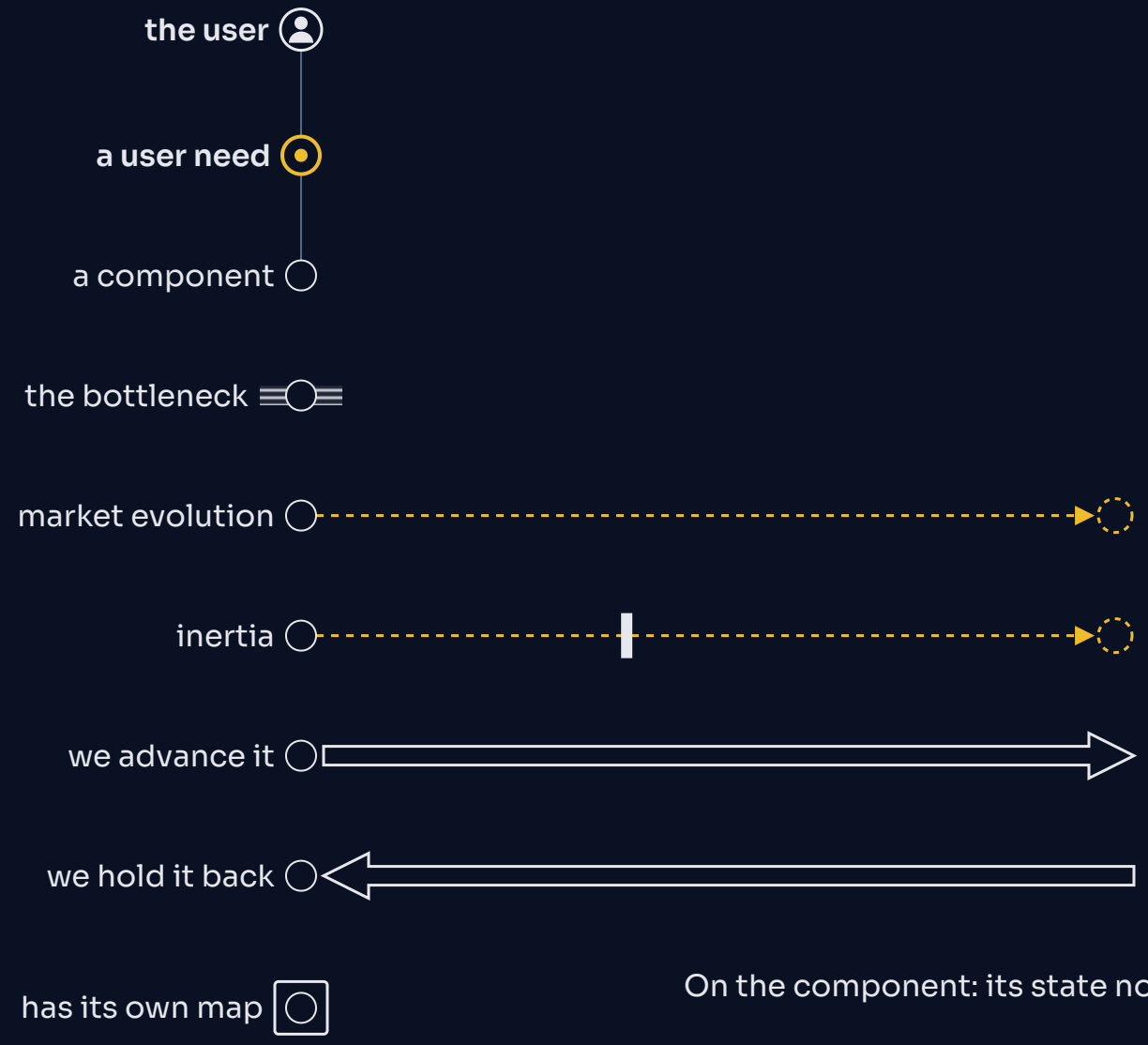
Evolution: four sets of stage labels

ACTIVITIES	GENESIS	CUSTOM BUILT	PRODUCT (+ RENTAL)	COMMODITY (+ UTILITY)
Practices	Novel	Emerging	Good	Best
Data	Unmodelled	Divergent	Convergent	Modelled
Knowledge	Concept	Hypothesis	Theory	Accepted

Evolution runs left to right, as on the map.

Users and needs: start from the user

- The anchor is a user: a person, a team or an organisation
- Their needs sit just below, in their words
- Everything else is there to meet a need
- Ask "how?" to go down the chain, "why?" to go up
- Several users? Where their needs differ, look closer



On the component: its state now

On the evolution line: forces on its move

we build it

we buy it

we outsource it

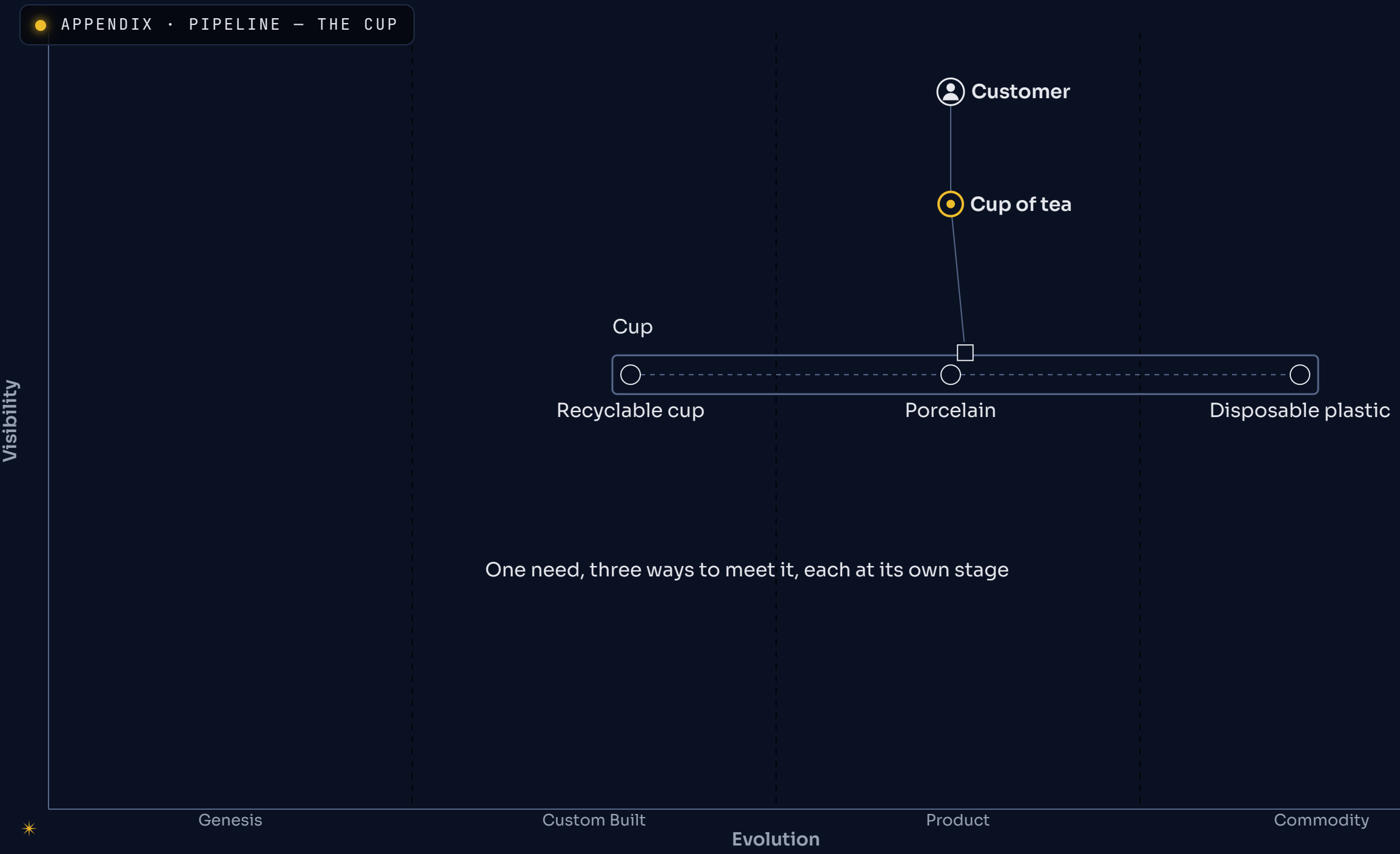
a market

Sourcing: how we get it

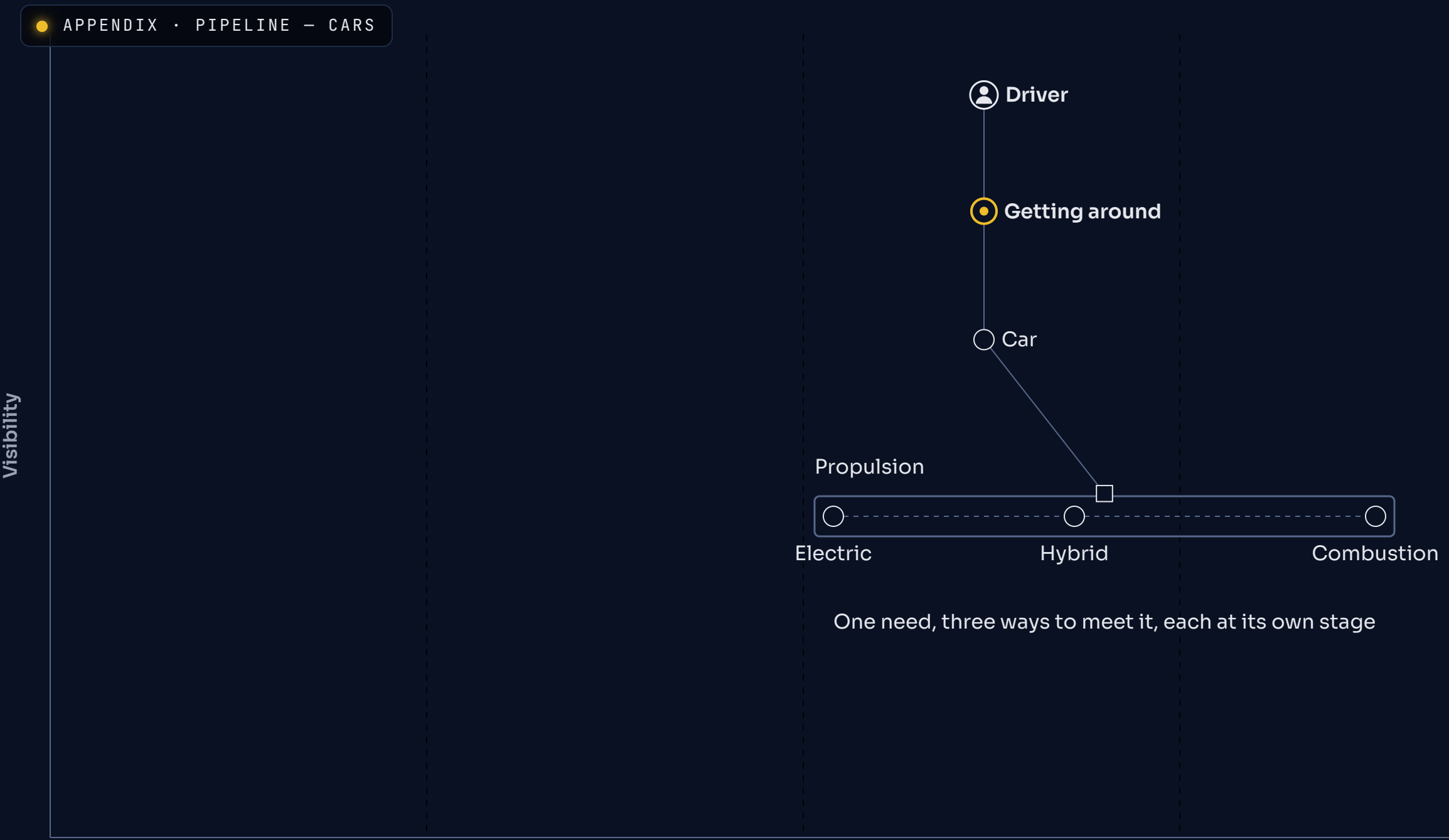
A marketplace others trade in

a pipeline

One thing, several ways, each at its own stage



One need, three ways to meet it, each at its own stage


Driver
Getting around
Car
Propulsion
Electric
Hybrid
Combustion
One need, three ways to meet it, each at its own stage
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

Submaps: a map inside a component

- A square around a component: it has its own map
- Keep the top map to the decision at hand
- Open a submap only for the part your question is about
- Inside, the same rules: users, needs, evolution

Constraint vs inertia

- Constraint: a capacity limit. Work piles up in front of it
- Inertia: the old way is kept although a better one exists
- Constraint sits on the component: the striped bar
- Inertia sits on its evolution line: the solid bar
- Ask who would lose if it moved: that is the inertia

Goldratt’s Theory of Constraints: five steps

- Identify the constraint
- Exploit it: get the most out of what is there
- Subordinate everything else to it
- Elevate it: add capacity, only now
- Repeat: the constraint has moved

Wardley’s climate: patterns you can't stop

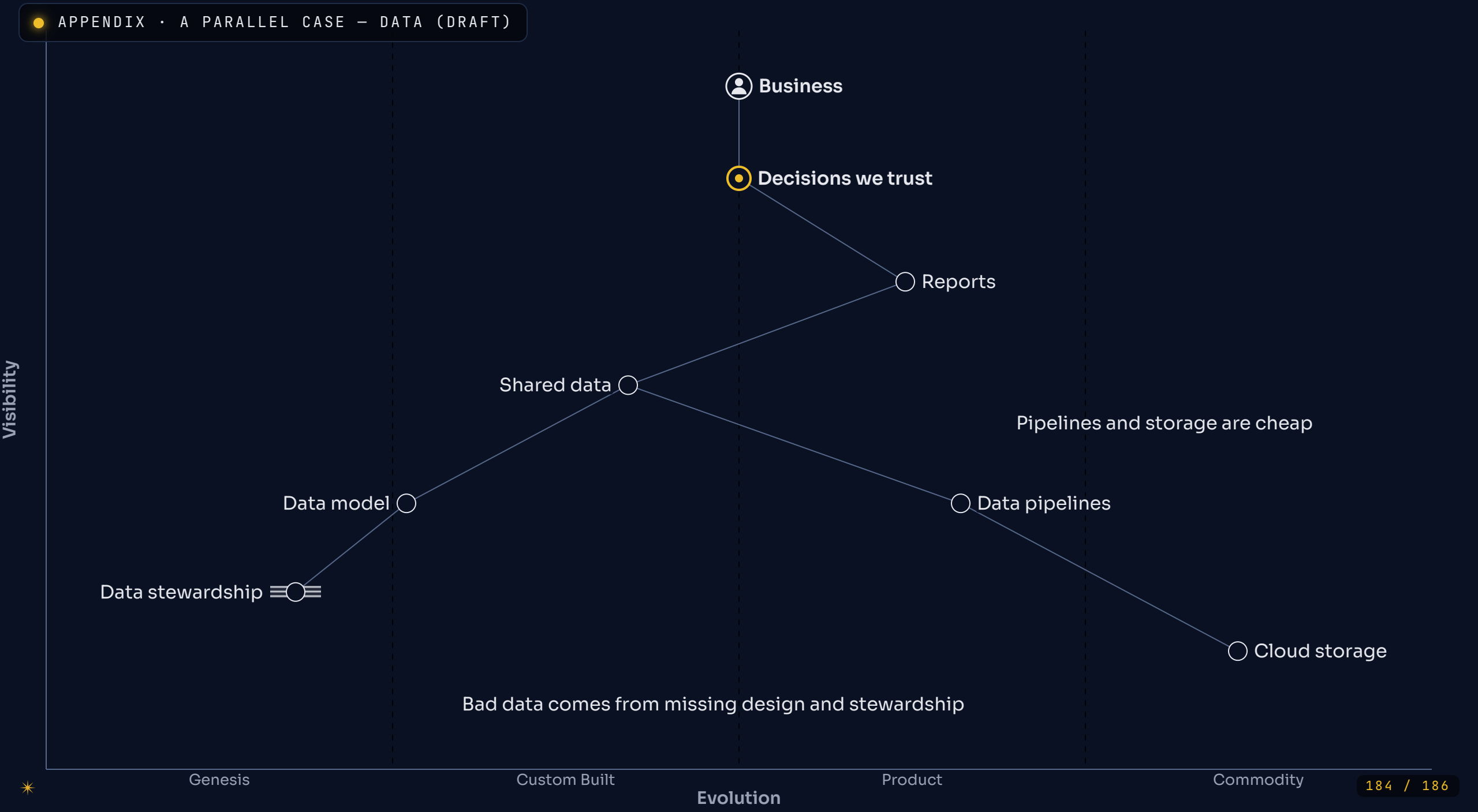
- Everything evolves through supply and demand competition
- Characteristics change as components evolve
- No choice over evolution
- Practices co-evolve with the activities they support
- Efficiency enables innovation
- Higher-order systems create new sources of worth
- Past success breeds inertia

Wardley’s doctrine: principles that work anywhere

- Focus on user needs
- Know your users
- Use a common language
- Challenge assumptions
- Remove duplication and bias
- Use appropriate methods
- Think small: small teams, small parts

Wardley’s gameplay: moves you can choose

- Open approaches: commoditise a layer someone else owns
- ILC: innovate, leverage, commoditise
- Standards game: set the rules others build on
- Exploit inertia: move where a rival can't follow
- Co-creation: build it with users, not for them


Business
Decisions we trust
Reports
Shared data
Pipelines and storage are cheap
Data model
Data pipelines
Data stewardship
Cloud storage
Bad data comes from missing design and stewardship
Visibility
Genesis
Custom Built
Product
Commodity
Evolution

If you want to read on

- First: Simon Wardley's own book, free to read at medium.com/wardleymaps and on Leanpub
- All the others map with Wardley maps. Kaiser and Tune write for a DDD audience
- Erik Schön's annex to The Art of Strategy collects 168 Wardley maps from 90 mappers, free to read. One of them is my logistics map

GALAXITECH

THANK YOU

QUESTIONS

galaxi.tech · cj@galaxi.tech
linkedin.com/in/cjjohansen

`github.com/cjjohansen/DDD-Belgium-Wardley-Mapping-2026`

Book a workshop